$A000 VGC
$D400 SID
$BA00 XMC

MAPPING THE NOVAVM

A Complete Address-by-Address Reference

For the e6502 Virtual Computer | 2026 Edition

64KB Address Space | 12 Hardware Controllers | 512KB Expansion | 14-Voice Sound

Mapping the NovaVM

A Complete Address-by-Address Reference — 2026 Edition

Copyright © 2026 Barry Walker. All Rights Reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

e6502 is an open source project.

This reference covers the NovaVM hardware as implemented in the e6502 emulator. EhBASIC internals are derived from Enhanced BASIC 2.22p5 by Lee Davison, used under the terms of its original licence.

Typeset with LaTeX and the `lmodern` font family.
Document class: `book`, 11 pt, A4.
Build toolchain: `latexmk` with `pdflatex`.

Contents

1 Zero Page **1**
- 1.1 Boot and Warm Start ($00–$02) . . . 1
- 1.2 Unallocated ($03–$09) . . . 2
- 1.3 USR() Function Vector ($0A–$0C) . . . 2
- 1.4 Terminal Settings ($0D–$10) . . . 2
- 1.5 Temporary Integer ($11–$12) . . . 3
- 1.6 Unallocated ($13–$5A) . . . 3
- 1.7 Interpreter Flags ($5B–$64) . . . 4
- 1.8 String Descriptor Stack ($65–$70) . . . 5
- 1.9 Utility Pointers and FAC Temporaries ($71–$78) . . . 5
- 1.10 Memory Management Pointers ($79–$86) . . . 6
- 1.11 Line and Data Pointers ($87–$9F) . . . 8
- 1.12 Garbage Collection and Array Workspace ($A0–$AB) . . . 10
- 1.13 Floating-Point Accumulator 1 ($AC–$B2) . . . 11
- 1.14 Floating-Point Accumulator 2 ($B3–$B9) . . . 12
- 1.15 BASIC Byte Fetch Subroutines ($BA–$D7) . . . 12
- 1.16 Pseudo-Random Number Generator ($D8–$DB) . . . 13
- 1.17 Interrupt Handler Vectors ($DC–$E1) . . . 14
- 1.18 HELP Scratch ($E2) . . . 15
- 1.19 Unused ($E3–$EE) . . . 15
- 1.20 Decimal Conversion Buffer ($EF–$FF) . . . 15

2 The Stack **17**
- 2.1 Stack Mechanics . . . 17
- 2.2 Stack Frame Formats . . . 18
  - 2.2.1 JSR/RTS — Subroutine Call (2 bytes) . . . 18
  - 2.2.2 BRK/IRQ/NMI — Interrupt Frame (3 bytes) . . . 18
  - 2.2.3 EhBASIC FOR/NEXT Frame (~18 bytes) . . . 18
  - 2.2.4 EhBASIC GOSUB Frame (5 bytes) . . . 19
- 2.3 Stack Pointer Initialisation . . . 19
- 2.4 Reading the Stack Pointer . . . 19

3 The Vector Table **21**
- 3.1 Vector Entries . . . 21
- 3.2 Reserved Range ($0218–$027F) . . . 23
- 3.3 Reading the Complete Vector Table . . . 24

4 BASIC Program Space **25**
- 4.1 Memory Layout Overview . . . 25
- 4.2 Tokenised Line Format . . . 26
  - 4.2.1 Worked Example: `10 PRINT "HELLO"` . . . 27

4.3 Token Table . . . 27
4.3.1 Primary Tokens ($80–$FF) . . . 27
4.3.2 Extended Tokens ($01 + sub-token) . . . 29
4.4 Simple Variable Storage . . . 30
4.4.1 Variable Name Encoding . . . 30
4.4.2 Numeric Variables . . . 31
4.4.3 String Variables . . . 31
4.5 Array Storage . . . 31
4.5.1 Array Header . . . 31
4.5.2 Array Element Data . . . 31
4.6 String Heap . . . 32
4.6.1 Garbage Collection . . . 32
4.7 Free Memory . . . 32
4.8 Walking the Data Structures . . . 33
4.8.1 Walking the Program Line List . . . 33
4.8.2 Dumping a Tokenised Line . . . 33
4.8.3 Showing the Memory Map . . . 33

5 The Virtual Graphics Controller 35
5.1 Display Modes . . . 35
5.2 Core Registers ($A000–$A00F) . . . 36
5.3 Command Interface ($A010–$A01E) . . . 39
5.3.1 Graphics Commands . . . 39
5.3.2 Sprite Commands . . . 43
5.3.3 Memory I/O Commands . . . 45
5.3.4 Copper (Raster) Commands . . . 46
5.4 IRQ Control Register ($A01F) . . . 48
5.5 Help System Registers ($A020–$A030) . . . 48
5.6 Compiler Controller ($A031–$A03E) . . . 49
5.7 ROM Swap Register ($A03F) . . . 50
5.8 Sprite Registers ($A040–$A0BF) . . . 50

6 The Network Interface Controller 53
6.1 Command Register . . . 53
6.2 Status and Control Registers . . . 54
6.3 Port Registers . . . 55
6.4 DMA Registers . . . 55
6.5 Per-Slot Status Registers . . . 56
6.6 Hostname Buffer . . . 56
6.7 Complete Examples . . . 57
6.7.1 Client: Connect, Send, and Receive . . . 57
6.7.2 Server: Listen, Accept, and Echo . . . 57

7 The Wavetable Synthesizer 59
7.1 Voice Registers ($A140–$A17F) . . . 59
7.2 Global Registers ($A180–$A187) . . . 61
7.3 Instrument Enumeration ($A1A0–$A1DF) . . . 62
7.4 Complete Examples . . . 63
7.4.1 Loading a Soundfont . . . 63
7.4.2 Playing a Note . . . 63
7.4.3 Three-Note Chord with Panning . . . 63
7.4.4 Setting Up Reverb and Chorus . . . 64

8 Screen Memory **65**
8.1 Character RAM ($AA00–$B1CF) . . . 65
8.2 Color RAM ($B1D0–$B99F) . . . 66
8.3 Screen Coordinate Math . . . 66
8.3.1 Worked Example . . . 67
8.4 BASIC Examples . . . 67
8.4.1 Direct Screen Write . . . 67
8.4.2 Screen Dump . . . 67
8.4.3 Fast Text Rendering . . . 67
8.4.4 Colour Cycling Effect . . . 68

9 The File I/O Controller **69**
9.1 Register Map . . . 69
9.2 Core Registers . . . 70
9.3 Command Reference . . . 72
9.3.1 File Operations . . . 72
9.3.2 SID Playback . . . 73
9.3.3 Music Engine . . . 73
9.3.4 MIDI Playback . . . 75
9.3.5 Soundfont . . . 75
9.3.6 Disk Operations . . . 76
9.4 File Metadata Buffer ($BAB0–$BB1F) . . . 77
9.5 Status and Error Code Summary . . . 77
9.5.1 Status Codes (FioStatus, $B9A1) . . . 77
9.5.2 Error Codes (FioErrCode, $B9A2) . . . 77
9.6 BASIC Examples . . . 77
9.6.1 Saving and Loading a Program . . . 77
9.6.2 Directory Listing . . . 78
9.6.3 Playing a SID File . . . 78
9.6.4 Loading and Playing a MIDI File . . . 79

10 The Expansion Memory System **81**
10.1 Expansion Memory Controller ($BA00–$BA3F) . . . 81
10.1.1 Register Map . . . 82
10.1.2 Detailed Register Descriptions . . . 82
10.1.3 Command Reference . . . 85
10.2 Timer Controller ($BA40–$BA4F) . . . 89
10.3 Music Status Registers ($BA50–$BA62) . . . 90
10.4 DMA Controller ($BA63–$BA82) . . . 91
10.4.1 Register Map . . . 91
10.4.2 Detailed Register Descriptions . . . 92
10.4.3 BASIC Examples . . . 94
10.5 Blitter Controller ($BA83–$BAA2) . . . 94
10.5.1 Register Map . . . 94
10.5.2 Detailed Register Descriptions . . . 95
10.5.3 BASIC Examples . . . 97
10.6 XMC Windows ($BC00–$BFFF) . . . 98

11 The ROM **101**
11.1 ROM Layout Overview . . . 101
11.2 Key Entry Points . . . 102
11.3 Monitor Code ($FF80–$FFF9) . . . 102
11.4 SID Register Window . . . 103
11.5 Hardware Vectors ($FFFA–$FFFF) . . . 103

12 The SID Sound Chips **105**
12.1 Voice Registers . . . 105
12.2 Filter Registers ($D415–$D418) . . . 108
12.3 Extended Per-Voice Volume ($D41D–$D41F) . . . 109
12.4 SID 2 ($D420–$D43F) . . . 109
12.5 BASIC Examples . . . 109
12.6 Register Summary . . . 110

A VGC Bitmap & Sprite Shape RAM **113**
A.1 Graphics Bitmap . . . 113
A.1.1 Address Calculation . . . 113
A.1.2 Row Start Offsets . . . 114
A.1.3 Drawing Commands vs. Direct Access . . . 114
A.1.4 BASIC Example: Read/Write Pixels via MemRead/MemWrite . . . 114
A.2 Sprite Shape RAM . . . 114
A.2.1 Address Calculation . . . 115
A.2.2 Shape Slot Assignments . . . 115
A.2.3 BASIC Example: Define a Shape via MemWrite . . . 115
A.2.4 Using DMA for Bulk Shape Loading . . . 115

B Expansion RAM (XRAM) **117**
B.1 Bank Structure . . . 117
B.2 Access Methods . . . 117
B.2.1 Byte Commands (GetByte / PutByte) . . . 118
B.2.2 Bulk Commands (Stash / Fetch) . . . 118
B.2.3 XMC Windows ($BC00–$BFFF) . . . 118
B.2.4 DMA Transfers (Space 5) . . . 118
B.3 Named Block Allocation . . . 119
B.4 Page Usage Tracking . . . 119

C WTS Sample Memory **121**
C.1 Loading Soundfonts . . . 121
C.2 SF2 Format Overview . . . 122
C.3 Instrument Mapping . . . 122
C.4 Memory Usage . . . 122
C.5 Limitations . . . 123

D Color Table **125**
D.1 Palette Reference . . . 125
D.2 Usage Notes . . . 126
D.3 BASIC Example . . . 126

E ASCII & Screen Code Table **127**
E.1 ASCII Table (0–127) 127
E.2 Control Characters Used by the Screen Editor 128
E.3 Extended Characters (128–255) 128
E.4 BASIC String Functions 128

Address Index **131**

Chapter 1

Zero Page

> "The zero page is prime real estate on the 6502. If your code isn't living there, it's commuting."
>
> — *Unattributed, ca. 1983*

Introduction

On the MOS 6502, the 256 bytes from $0000 to $00FF enjoy a privileged status. Instructions that reference zero-page addresses are one byte shorter and one cycle faster than their absolute-address equivalents. The CPU's limited register set makes these locations indispensable — they serve as extra registers, scratch space, and indirect-address pointers for nearly every significant operation.

In the NovaVM, the zero page is owned almost entirely by EhBASIC. The interpreter packs its warm-start vector, terminal state, memory-management pointers, floating-point accumulators, and even a small machine-language subroutine into this single page. Understanding these locations is the key to understanding how BASIC works under the hood — and to writing programs that can peek behind the curtain.

The map below is arranged in address order. Where a location serves double duty under different aliases (a common EhBASIC technique), all names are listed.

1.1 Boot and Warm Start ($00–$02)

When EhBASIC starts up, it writes a three-byte `JMP` instruction at the very bottom of the zero page. Thereafter, a warm start (the BASIC command `RUN` completing, or the user pressing the reset key) simply jumps to address $0000, which executes the `JMP` and re-enters the interpreter's main loop without losing the stored program.

$0000 LAB_WARM

R/W | Default: $4C

The first byte of the warm-start trampoline. Always contains `$4C`, the 6502 opcode for `JMP abs`. Do not alter this byte unless you are deliberately redirecting the warm-start path.

$0001 Wrmjpl

R/W | Default: —

Low byte of the warm-start jump target address.

$0002 Wrmjph

R/W | Default: —

High byte of the warm-start jump target address. Together with $0001, this forms the destination of the JMP at $0000.

You can redirect the warm start by POKEing a new address into $01–$02. For example, to jump to your own machine-language routine at $8000:

```
DOKE $01,$8000
```

Be very sure you know what you are doing — a bad vector here means a hard reset is the only way back.

1.2 Unallocated ($03–$09)

Addresses $03 through $09 are not assigned by EhBASIC. They sit in the gap between the warm-start trampoline and the USR() vector. These seven bytes are safe for your own machine-language routines or short data tables, provided you do not call any BASIC routine that might overwrite them during initialisation. In practice, they remain untouched after boot.

1.3 USR() Function Vector ($0A–$0C)

The USR() function in EhBASIC calls a user-supplied machine-language routine. Like the warm-start vector, the mechanism is a three-byte JMP instruction planted in the zero page.

$000A **Usrjmp** R/W | Default: $4C

The JMP opcode byte for the USR() dispatch. Always $4C.

$000B **Usrjpl** R/W | Default: —

Low byte of the USR() target address.

$000C **Usrjph** R/W | Default: —

High byte of the USR() target address. To point USR() at a routine at $4000:

```
DOKE $0B,$4000
X = USR(42)
```

The argument (42 in this case) is passed in FAC1 (see $00AC), and the routine should leave its return value there.

1.4 Terminal Settings ($0D–$10)

These four bytes control how EhBASIC interacts with the terminal — line width, cursor column tracking, and null padding.

$000D **Nullct** R/W | Default: $00

Null count: the number of $00 bytes transmitted after each carriage return. On real serial terminals this gave the print head time to return to the left margin. On the NovaVM this is unused and should be left at zero.

$000E **TPos** R/W | Default: $00

Terminal position: the current column of the cursor, maintained by EhBASIC's output routines. Starts at zero for the left margin and increments with each printed character. The interpreter checks this against $000F to decide when to wrap. Reading this byte tells you the cursor's horizontal position:

```
PRINT "Hello";
PRINT PEEK($0E)
```

This prints 5, the column after the five characters of "Hello".

$000F **TWidth** R/W | Default: $50

Terminal width. Set to 80 ($50) by the NovaVM at boot. EhBASIC wraps output and inserts a carriage return when $000E reaches this value.

```
POKE $0F,40 : REM narrow screen
```

Warning

Changing TWidth does not resize the physical display. It only affects EhBASIC's line-wrap logic. Setting it wider than 80 will cause output to overrun the visible screen.

$0010 **Iclim** R/W | Default: $50

Input column limit: the maximum number of characters EhBASIC will accept on a single INPUT line. Normally matches TWidth.

1.5 Temporary Integer ($11–$12)

$0011 **Itempl** R/W | Default: —

Low byte of a temporary 16-bit integer used internally during line-number parsing, number-base conversion, and various arithmetic operations. Also known as nums_1 when used for binary/hex string conversion.

$0012 **Itemph** R/W | Default: —

High byte of the temporary integer. Also nums_2. A third alias byte nums_3 occupies $13 for 24-bit conversions.

These locations are transient workspace. Their contents are unpredictable between BASIC statements and should not be relied upon for storage.

1.6 Unallocated ($13–$5A)

Addresses $13 through $5A — a full 72 bytes — are not assigned by EhBASIC. This is the largest contiguous free region on the zero page and an excellent place to store variables, counters, or short machine-language routines that benefit from fast zero-page addressing.

If you are writing a machine-language game loop or interrupt handler, the $13–$5A block gives you plenty of room for sprite coordinates, frame counters, velocity tables, and other frequently-accessed state — all at zero-page speed.

1.7 Interpreter Flags ($5B–$64)

This cluster of bytes controls the interpreter's parsing and evaluation state. Most are written and read only by internal BASIC routines; they are documented here for completeness and for advanced users writing machine-language extensions.

$005B Srchc / Temp3

R/W | Default: —

Search character used by the tokeniser and line-editing routines. Also serves as `Temp3`, a scratch byte during number-to-string conversion. Double duty as the low byte of `XOAw` (the EOR/OR/AND working word).

$005C Scnquo / Asrch

R/W | Default: —

Scan-between-quotes flag. When set, the tokeniser treats everything up to the next quotation mark as literal text rather than keywords. Also used as `Asrch` (alternate search character) and as the high byte of `XOAw`.

$005D Ibptr / Dimcnt / Tindx

R/W | Default: —

A three-way alias:

- **Ibptr** — pointer (offset) into the input buffer during line entry.
- **Dimcnt** — dimension count while processing `DIM` statements.
- **Tindx** — token index during tokenisation.

Only one use is active at a time; the interpreter knows which context applies.

$005E Defdim

R/W | Default: $00

Default DIM flag. When non-zero, an array reference that has not been explicitly dimensioned will be auto-dimensioned (to a default upper bound of 10). Set internally by the array-access routines.

$005F Dtypef

R/W | Default: $00

Data type flag. $FF means the current value is a string; $00 means numeric. The interpreter sets this after every expression evaluation so that subsequent operations (assignment, PRINT, comparison) know which type they are handling.

$0060 Oquote / Gclctd

R/W | Default: $00

Open-quote flag during DATA scanning and LIST output. Bit 7 signals that the scanner is inside a quoted string. The same byte doubles as `Gclctd`, the garbage-collected flag — set after the string heap has been compacted to prevent redundant re-collection.

$0061 Sufnxf

R/W | Default: $00

Subscript/FN flag. Bit 7 set indicates a `FN` call is in progress; the lower bits track subscript depth. This prevents the interpreter from confusing array subscripts with function arguments.

$0062 Imode

R/W | Default: $00

Input mode flag. $00 during an `INPUT` statement, $80 during `READ`. This tells the shared input-parsing code whether to prompt the user or consume the next `DATA` element.

$0063 **Cflag** R/W | Default: $00

Comparison evaluation flag. Accumulates the result of relational operators during expression parsing:

- Bit 0 — less than (<)
- Bit 1 — equal (=)
- Bit 2 — greater than (>)

Combinations like <= set bits 0 and 1 simultaneously.

$0064 **TabSiz** R/W | Default: $0A

TAB step size. Controls the column spacing of comma-separated items in `PRINT` statements. Default is 10.

```
POKE $64,20 : REM wider tab stops
PRINT 1,2,3
```

1.8 String Descriptor Stack ($65–$70)

EhBASIC maintains a small stack of string descriptors in the zero page. Each descriptor is three bytes: one byte for the string length and two bytes for the pointer to the string data. The stack holds up to three temporary strings at once.

$0065 **next_s** R/W | Default: —

Pointer to the next available slot on the descriptor stack. Starts at $68 (the base of `des_sk`) and advances by three for each temporary string pushed.

$0066 **last_sl** R/W | Default: —

Low byte of the address of the most recently pushed descriptor.

$0067 **last_sh** R/W | Default: $00

High byte of the last descriptor address. Since the descriptor stack lives entirely within the zero page, this byte is always $00.

$0068-$0070 **des_sk (Descriptor Stack)** R/W | Default: —

Nine bytes holding up to three string descriptors. These are working storage for intermediate string results — for example, the expression `A$+B$+C$` will push descriptors onto this stack as it concatenates. If more than three temporaries are needed, EhBASIC raises a `?FORMULA TOO COMPLEX` error.

1.9 Utility Pointers and FAC Temporaries ($71–$78)

$0071 **ut1_pl** R/W | Default: —

Utility pointer 1, low byte. A general-purpose 16-bit pointer used by block move operations, array manipulation, and string handling. Also known as `Temp_2` during block moves.

$0072 **ut1_ph** R/W | Default: —

Utility pointer 1, high byte.

$0073 **ut2_pl** R/W | Default: —

Utility pointer 2, low byte. A second general-purpose pointer, often used as the destination address when `ut1` is the source.

$0074 **ut2_ph** R/W | Default: —

Utility pointer 2, high byte.

$0075 **FACt_1** R/W | Default: —

FAC temporary mantissa byte 1. Used as scratch space during floating-point operations and multiply/divide routines.

$0076 **FACt_2 / dims_l** R/W | Default: —

FAC temporary mantissa byte 2. Also serves as `dims_l`, the low byte of the array dimension size during `DIM` processing.

$0077 **FACt_3 / dims_h** R/W | Default: —

FAC temporary mantissa byte 3, and the high byte of the array dimension size.

$0078 **TempB** R/W | Default: —

A single scratch byte used by various interpreter routines. Entirely transient.

1.10 Memory Management Pointers ($79–$86)

These seven pointer pairs are the backbone of EhBASIC's memory management. They define the boundaries of every major region in RAM: program text, variables, arrays, and string storage. Understanding them is essential for anyone who wants to know how much memory is free, where a program lives, or how to relocate data.

The pointers are arranged so that, in a healthy system:

$$\texttt{Smem} \leq \texttt{Svar} \leq \texttt{Sarry} \leq \texttt{Earry} \leq \texttt{Sstor} \leq \texttt{Emem}$$

Program text grows upward from Smem; string storage grows *downward* from Emem. The gap between Earry and Sstor is free memory.

$0079 **Smeml** R/W | Default: —

Start of BASIC program text, low byte.

$007A **Smemh** R/W | Default: —

Start of BASIC program text, high byte. This is the base address of the tokenised program. On the NovaVM, this is typically $0280 (the first byte above the vector table).

```
PRINT DEEK($79)
```

This prints the start-of-program address in decimal.

$007B **Svarl** R/W | Default: —

Start of variables, low byte.

$007C Svarh

R/W | Default: —

Start of variables, high byte. This pointer also marks the end of program text. The difference `DEEK($7B) - DEEK($79)` is the size of your program in bytes.

```
PRINT DEEK($7B)-DEEK($79);"BYTES PROGRAM"
```

$007D Sarryl

R/W | Default: —

Start of arrays, low byte.

$007E Sarryh

R/W | Default: —

Start of arrays, high byte. Simple variables (scalars) occupy the region from Svar to Sarry.

$007F Earryl

R/W | Default: —

End of arrays, low byte.

$0080 Earryh

R/W | Default: —

End of arrays, high byte. All dimensioned arrays live between Sarry and Earry.

$0081 Sstorl

R/W | Default: —

Bottom of string storage, low byte.

$0082 Sstorh

R/W | Default: —

Bottom of string storage, high byte. String data is allocated from the top of memory downward. Sstor marks the lowest string address currently in use. As more strings are created, this pointer moves down toward Earry. When the two meet, you get an `?OUT OF MEMORY` error.

$0083 Sutill

R/W | Default: —

String utility pointer, low byte.

$0084 Sutilh

R/W | Default: —

String utility pointer, high byte. A working pointer used during string allocation and garbage collection.

$0085 Ememl

R/W | Default: —

End of memory, low byte.

$0086 Ememh

R/W | Default: —

End of memory, high byte. This is the top of available RAM — the highest address BASIC is allowed to use. String storage begins just below this point and grows downward.

```
REM -- Free memory calculation --
FM = DEEK($81)-DEEK($7F)
PRINT FM;"BYTES FREE"
```

The FRE() function in EhBASIC returns the same value but also triggers a garbage collection pass, which can cause a brief pause. Reading the pointers directly with DEEK is instantaneous but does not account for reclaimable string space.

A quick memory map dump:

```
PRINT "PROGRAM  ";DEEK($79);" TO ";DEEK($7B)-1
PRINT "VARIABLES";DEEK($7B);" TO ";DEEK($7D)-1
PRINT "ARRAYS   ";DEEK($7D);" TO ";DEEK($7F)-1
PRINT "FREE     ";DEEK($7F);" TO ";DEEK($81)-1
PRINT "STRINGS  ";DEEK($81);" TO ";DEEK($85)-1
```

1.11 Line and Data Pointers ($87–$9F)

These pointers track the interpreter's current position in the program, the DATA read cursor, and the state of the most recent variable reference.

$0087 Clinel

R/W | Default: —

Current BASIC line number, low byte.

$0088 Clineh

R/W | Default: —

Current BASIC line number, high byte. While a program is running, this pair holds the line number currently being executed. In immediate mode, it contains $FFFF.

```
10 PRINT "LINE ";DEEK($87)
20 PRINT "LINE ";DEEK($87)
RUN
```

This prints `LINE 10` then `LINE 20`.

$0089 Blinel

R/W | Default: —

Break line number, low byte.

$008A Blineh

R/W | Default: —

Break line number, high byte. When a program is stopped (by the `STOP` statement, Ctrl+C, or an error), EhBASIC saves the line number here. The `CONT` command uses this to resume execution.

$008B Cpntrl

R/W | Default: —

Continue pointer, low byte.

$008C Cpntrh

R/W | Default: —

Continue pointer, high byte. The address within program text where execution should resume after `CONT`. If you edit any line, this pointer is invalidated and `CONT` will refuse to proceed.

$008D Dlinel

R/W | Default: —

Current DATA line number, low byte.

$008E **Dlineh** R/W | Default: —

Current DATA line number, high byte. Tracks which line the DATA cursor is on, so that an `?OUT OF DATA` error can report the correct line number.

$008F **Dptrl** R/W | Default: —

DATA pointer, low byte.

$0090 **Dptrh** R/W | Default: —

DATA pointer, high byte. Points to the next byte to be consumed by a `READ` statement. `RESTORE` resets this to the start of program text.

$0091 **Rdptrl** R/W | Default: —

Read pointer, low byte.

$0092 **Rdptrh** R/W | Default: —

Read pointer, high byte. Used during INPUT/READ to track the current position in the data being parsed.

$0093 **Varnm1** R/W | Default: —

Current variable name, first byte. When the interpreter encounters a variable reference (e.g. `A$`), the first character of the name is stored here.

$0094 **Varnm2** R/W | Default: —

Current variable name, second byte. For single-letter variables this is zero. For two-letter names, it holds the second character. Bit 7 of either byte may be set to indicate a string ($) or integer (%) type suffix.

$0095 **Cvaral** R/W | Default: —

Current variable address, low byte.

$0096 **Cvarah** R/W | Default: —

Current variable address, high byte. After a variable lookup, this pair points to the variable's storage in the variable table (between Svar and Sarry).

$0097 **Frnxtl** R/W | Default: —

FOR/NEXT variable pointer, low byte. Also aliased as `Tidx1` (temp line index) and `Lvarpl` (LET variable pointer).

$0098 **Frnxth** R/W | Default: —

FOR/NEXT variable pointer, high byte. Also `Lvarph`. During FOR/NEXT execution, this pair points to the loop control variable so that NEXT can find the matching FOR frame on the stack.

$0099 **prstk** R/W | Default: $00

Precedence-stacked flag. Set non-zero when the expression evaluator has pushed an operator onto the precedence stack. Cleared when the stack is unwound.

$009B **comp_f** R/W | Default: $00

Compare function flag. Uses bits 0, 1, and 2 to encode the comparison being evaluated:

7	6	5	4	3	2	1	0
--	--	--	--	--	>	=	<

This is the internal companion to $0063 (Cflag), used during the parsing phase of a comparison expression.

$009C **func_l / garb_l** R/W | Default: —

Function pointer low byte. During function evaluation, points to the body of a `DEF FN` definition. Also serves as `garb_l`, the garbage collection working pointer low byte.

$009D **func_h / garb_h** R/W | Default: —

Function pointer high byte / garbage collection working pointer high byte.

$009E **des_2l** R/W | Default: —

String descriptor 2 pointer, low byte.

$009F **des_2h** R/W | Default: —

String descriptor 2 pointer, high byte. Used during string operations that need to reference two descriptors simultaneously (e.g. concatenation or comparison).

1.12 Garbage Collection and Array Workspace ($A0–$AB)

$00A0 **g_step** R/W | Default: —

Garbage collection step size. Controls the search increment during string heap compaction.

$00A1 **Fnxjmp** R/W | Default: $4C

Jump vector for function dispatch — another three-byte `JMP` instruction, like the warm-start and USR() vectors. Byte $A1 holds the `JMP` opcode.

$00A2 **Fnxjpl / g_indx** R/W | Default: —

Low byte of the function jump target. Also aliased as `g_indx`, a temporary index used during garbage collection.

$00A3 **FAC2_r** R/W | Default: —

FAC2 rounding byte. The extra precision byte for the second floating-point accumulator, used during multi-step arithmetic to reduce rounding errors.

$00A4 **Adatal / Nbendl** R/W | Default: —

Array data pointer, low byte. Also serves as `Nbendl` — the new block end pointer during memory block moves (INSERT, DELETE).

$00A5 **Adatah / Nbendh** R/W | Default: —

Array data pointer, high byte / new block end pointer high byte.

$00A6 **Obendl** R/W | Default: —

Old block end pointer, low byte. Used alongside `Nbend` during block move operations.

$00A7 **Obendh** R/W | Default: —

Old block end pointer, high byte.

$00A8 **numexp / numbit** R/W | Default: —

Number exponent count during string-to-float conversion. Also `numbit`, a bit counter during array element address calculations.

$00A9 **expcnt** R/W | Default: —

Exponent count for string-to-float conversion — tracks the decimal exponent independently from the mantissa being assembled.

$00AA **numdpf / Astrtl** R/W | Default: —

Decimal point flag during number conversion. This byte is heavily aliased: also `Astrtl` (array start pointer low), `Histrl` (highest string low), `Baslnl` (BASIC search line pointer low), `Fvar_l` (find variable pointer low), `Ostrtl` (old block start pointer low), and `Vrschl` (variable search pointer low).
Only one use is active at any given time — the interpreter context determines the meaning.

$00AB **expneg / Astrth** R/W | Default: —

Exponent negate flag during number conversion. Similarly aliased: `Astrth`, `Histrh`, `Baslnh`, `Fvar_h`, `Ostrth`, `Vrschh`.

1.13 Floating-Point Accumulator 1 ($AC–$B2)

FAC1 is the primary floating-point accumulator. All arithmetic operations ultimately produce their result here. EhBASIC uses a five-byte format: one byte for the exponent and four bytes for the mantissa (with an implicit leading 1 bit) plus a separate sign byte.

$00AC **FAC1_e** R/W | Default: —

FAC1 exponent. Uses excess-128 notation: the stored value minus 128 gives the true binary exponent. A stored zero means the entire FAC is zero. Also aliased as `str_ln` (string length) during string operations.

$00AD **FAC1_1** R/W | Default: —

FAC1 mantissa byte 1 (most significant). Also `str_pl` (string pointer low) in string context.

$00AE **FAC1_2** R/W | Default: —

FAC1 mantissa byte 2. Also `str_ph` (string pointer high) and `des_pl` (descriptor pointer low).

$00AF **FAC1_3** R/W | Default: —

FAC1 mantissa byte 3 (least significant in the four-byte mantissa). Also `des_ph` (descriptor pointer high) and `mids_l` (MID$ temporary length).

$00B0 **FAC1_s** R/W | Default: —

FAC1 sign. Bit 7 indicates negative. The remaining bits are unused.

$00B1 **negnum / numcon** R/W | Default: —

String-to-float negate flag: set when parsing a negative numeric literal. Also `numcon`, the constant counter during polynomial series evaluation (used by SIN, COS, EXP, LOG, ATN).

$00B2 **FAC1_o** R/W | Default: —

FAC1 overflow byte. Holds the extra precision bit that falls off the bottom of the mantissa during normalisation. Used for rounding the final result.

1.14 Floating-Point Accumulator 2 ($B3–$B9)

FAC2 is the secondary accumulator. Binary operations (add, subtract, multiply, divide) load one operand into FAC2 and the other into FAC1; the result lands in FAC1.

$00B3 **FAC2_e** R/W | Default: —

FAC2 exponent. Same excess-128 encoding as FAC1.

$00B4 **FAC2_1** R/W | Default: —

FAC2 mantissa byte 1 (most significant).

$00B5 **FAC2_2** R/W | Default: —

FAC2 mantissa byte 2.

$00B6 **FAC2_3** R/W | Default: —

FAC2 mantissa byte 3 (least significant).

$00B7 **FAC2_s** R/W | Default: —

FAC2 sign byte. Bit 7 indicates negative.

$00B8 **FAC_sc** R/W | Default: —

FAC sign comparison result. After comparing FAC1 and FAC2, this byte records whether the signs agree or differ. Also aliased as `ssptr_l` (string start pointer low) and `sdescr` (string descriptor pointer).

$00B9 **FAC1_r** R/W | Default: —

FAC1 rounding byte. The ninth bit of precision, used during final rounding of arithmetic results. Also `ssptr_h` (string start pointer high).

1.15 BASIC Byte Fetch Subroutines ($BA–$D7)

This region contains one of EhBASIC's most distinctive features: executable machine code planted directly in the zero page. Two small subroutines live here, giving the interpreter fast,

one-byte `JSR` calls for its two most frequent operations — fetching the next byte from program text and re-fetching the current byte.

$00BA **csidx / Asptl** R/W | Default: —

Line crunch save index. Used during tokenisation to remember the current position. Also aliased as `Asptl` (array size pointer low), `Btmpl`, `Cptrl`, and `Sendl` — all temporary pointer low bytes used in different contexts.

$00BB **Aspth** R/W | Default: —

Array size pointer high byte. Also `Btmph`, `Cptrh`, and `Sendh`.

$00BC-$00C1 **LAB_IGBY (Increment and Get BYte)** R/W | Default: —

Six bytes of machine code implementing the "get next BASIC byte" subroutine. When the interpreter calls `JSR $BC`, this routine increments the BASIC execute pointer ($00C3–$00C4) and falls through into `LAB_GBYT` to fetch the byte at the new address.
This subroutine is placed in the zero page purely for speed. The 6502's `JSR zp` is not actually faster (there is no zero-page JSR), but having the code here keeps the execute pointer in the zero page where `LDA (zp),Y` can reach it in just five cycles.

$00C2-$00C4 **LAB_GBYT (Get BYTe)** R/W | Default: —

Three bytes of machine code that load the byte at the current BASIC execute pointer and set the processor flags accordingly. The interpreter calls `JSR $C2` whenever it needs to re-examine the current token without advancing.

$00C3 **Bpntrl** R/W | Default: —

BASIC execute pointer, low byte. This is the interpreter's program counter — it points to the byte in program text that will be (or has just been) processed.

$00C4 **Bpntrh** R/W | Default: —

BASIC execute pointer, high byte. Together with $00C3, this is the single most important pointer in the entire interpreter.

The region $C5–$D7 continues the LAB_GBYT subroutine code. These 19 bytes are part of the bulk-initialised machine code block and should never be modified. They do not correspond to individually meaningful variables.

1.16 Pseudo-Random Number Generator ($D8–$DB)

EhBASIC's `RND()` function uses a simple four-byte linear feedback shift register stored in the zero page.

$00D8 **Rbyte4** R/W | Default: —

PRNG byte 4 (most significant). The "extra" byte added by EhBASIC's extended PRNG algorithm.

$00D9 **Rbyte1** R/W | Default: —

PRNG byte 1.

$00DA **Rbyte2** R/W | Default: —

PRNG byte 2.

$00DB **Rbyte3** R/W | Default: —

PRNG byte 3 (least significant). Each call to `RND(1)` shifts and XORs all four bytes to produce a new pseudo-random value.

```
REM Seed the PRNG
X = RND(-42)
REM Now RND(1) gives a repeatable sequence
FOR I=1 TO 5 : PRINT RND(1) : NEXT
```

Calling `RND()` with a negative argument seeds the generator. `RND(0)` returns the most recent value. `RND(1)` advances to the next.

1.17 Interrupt Handler Vectors ($DC–$E1)

The NovaVM extends EhBASIC with support for NMI and IRQ interrupts that can be handled from BASIC. Each handler has a control byte and a two-byte address vector.

$00DC **NmiBase** R/W | Default: $00

NMI (Non-Maskable Interrupt) control register.

7	6	5	4	3	2	1	0
Enable	Setup	Fired	--	--	--	--	--

- **Bit 7** — Interrupt enabled. When set, the NMI handler is armed.
- **Bit 6** — Setup complete. Set by the `NMI` BASIC command after installing a handler.
- **Bit 5** — Interrupt occurred. Set by the hardware when an NMI fires; cleared after the handler runs.

$00DD **NMI Address Low** R/W | Default: —

Low byte of the NMI handler routine address.

$00DE **NMI Address High** R/W | Default: —

High byte of the NMI handler address. Together with `$00DD`, this points to the BASIC line (or machine-language routine) that should execute when an NMI arrives.

$00DF **IrqBase** R/W | Default: $00

IRQ (Interrupt Request) control register. Same bit layout as `$00DC`.

7	6	5	4	3	2	1	0
Enable	Setup	Fired	--	--	--	--	--

$00E0 **IRQ Address Low** R/W | Default: —

Low byte of the IRQ handler routine address.

$00E1 **IRQ Address High** R/W | Default: —

High byte of the IRQ handler address.

```
10 IRQ 1000
20 REM main loop here
30 GOTO 20
1000 REM interrupt handler
1010 PRINT "TICK"
1020 RETIRQ
```

The `IRQ` command sets up `$00DF`–`$00E1` automatically. You only need to POKE these directly if you are installing a machine-language handler.

1.18 HELP Scratch ($E2)

$00E2 **help_len** R/W | Default: $00

Scratch byte used by the `HELP` command to track the length of the keyword being searched. No user-visible purpose outside of that command.

1.19 Unused ($E3–$EE)

Addresses $E3 through $EE — twelve bytes — are completely unassigned. They are safe for user programs, machine-language scratch space, or small lookup tables. Combined with the seven bytes at $03–$09 and the large block at $13–$5A, this gives a total of 91 free zero-page bytes available for your own use.

1.20 Decimal Conversion Buffer ($EF–$FF)

$00EF-$00FF **Decss (Decimal String Buffer)** R/W | Default: —

Seventeen bytes reserved for number-to-decimal-string conversion. When BASIC needs to print a number or convert it with `STR$()`, the result is assembled here, character by character, starting from the least significant digit and working left. The buffer is large enough for the longest possible numeric string: a sign, up to ten digits, a decimal point, an "E", an exponent sign, and two exponent digits.

This buffer is transient — its contents are only valid during and immediately after a conversion. Do not store persistent data here.

Summary

The zero page is a tightly-packed workspace shared between EhBASIC and the 6502 hardware. The table below summarises the allocation at a glance:

Range	**Usage**	**Owner**
$00–$02	Warm-start JMP	EhBASIC
$03–$09	*Free*	User
$0A–$0C	USR() JMP vector	EhBASIC
$0D–$12	Terminal settings & temp int	EhBASIC
$13–$5A	*Free (72 bytes)*	User
$5B–$64	Interpreter flags	EhBASIC
$65–$70	String descriptor stack	EhBASIC
$71–$78	Utility pointers & FAC temp	EhBASIC
$79–$86	Memory management pointers	EhBASIC
$87–$9F	Line, data, & var pointers	EhBASIC
$A0–$AB	GC, array, & number work	EhBASIC
$AC–$B9	FAC1 & FAC2 accumulators	EhBASIC
$BA–$D7	Byte-fetch subroutines	EhBASIC
$D8–$DB	PRNG state	EhBASIC
$DC–$E1	NMI/IRQ handlers	EhBASIC
$E2	HELP scratch	EhBASIC
$E3–$EE	*Free (12 bytes)*	User
$EF–$FF	Decimal conversion buffer	EhBASIC

Chapter 2

The Stack

"Push it down, pull it back up. If only life were so orderly."

— *Anonymous assembly programmer, ca. 1979*

Introduction

On the MOS 6502, the hardware stack occupies exactly one page of memory: the 256 bytes from $0100 to $01FF. There is no option to move it, extend it, or protect it. The processor's 8-bit stack pointer (SP) is simply an offset into page 1 — push an item and SP decrements; pull an item and SP increments. When SP wraps from $00 back to $FF (or vice versa), the CPU does not complain. It just keeps going, silently overwriting whatever was there before. This simplicity is both a strength and a hazard. The stack is fast and predictable, but 256 bytes is not very much, especially when BASIC is using it for subroutine calls, loop variables, and expression evaluation all at once.

`$0100-$01FF` Hardware Stack R/W | Default: —

The entire 256-byte region is ordinary read/write RAM. The CPU treats it as special only because push and pull instructions always address page 1 using SP as the low byte. Nothing prevents you from reading or writing stack memory directly with PEEK and POKE — just be aware that the interpreter is using it too.

2.1 Stack Mechanics

The 6502 stack grows *downward* in memory. When the system starts, SP is initialised to $FF, meaning the first byte pushed will be written to $01FF. After the push, SP decrements to $FE. The next push writes to $01FE, and so on.

Pulling works in reverse: SP increments first, then the byte at the new address is read. This means the "current" top of stack is always at `$0100 + SP + 1` — the byte just above where SP points.

Operation	Effect on SP	Address Used
`PHA` / `PHP`	SP ← SP − 1	Write to $0100 + SP (before decrement)
`PLA` / `PLP`	SP ← SP + 1	Read from $0100 + SP (after increment)
`JSR`	SP ← SP − 2	Pushes PCH then PCL
`RTS`	SP ← SP + 2	Pulls PCL then PCH
`BRK` / IRQ / NMI	SP ← SP − 3	Pushes PCH, PCL, then P
`RTI`	SP ← SP + 3	Pulls P, then PCL, then PCH

Warning

The 6502 performs no overflow or underflow detection on the stack pointer. If SP wraps from $00 to $FF during a push, the CPU will happily write into the top of the stack page, overwriting data that was pushed earlier. Similarly, pulling past $FF wraps to $00. There is no trap, no error message — just silent data corruption.

2.2 Stack Frame Formats

The stack is used by both the 6502 hardware and EhBASIC for several distinct frame types. Understanding these formats helps when debugging programs that crash or behave unexpectedly.

2.2.1 JSR/RTS — Subroutine Call (2 bytes)

When the CPU executes a `JSR addr` instruction, it pushes the return address *minus one* onto the stack, high byte first, then low byte:

Stack offset	Content
SP + 2	PCH (high byte of return address − 1)
SP + 1	PCL (low byte of return address − 1)

The "minus one" quirk is because `RTS` increments the pulled address before jumping to it. This is a well-known 6502 subtlety and the reason that `JMP` and `JSR` are not interchangeable in dispatch tables.

2.2.2 BRK/IRQ/NMI — Interrupt Frame (3 bytes)

Hardware interrupts and the `BRK` instruction push three bytes: the full return address (high byte first, then low byte), followed by the processor status register (P):

Stack offset	Content
SP + 3	PCH
SP + 2	PCL
SP + 1	P (processor status)

`RTI` reverses the process: it pulls P, then PCL, then PCH, and resumes execution at the restored address. Unlike `RTS`, `RTI` does *not* add one to the return address — the pushed PC already points to the correct instruction.

2.2.3 EhBASIC FOR/NEXT Frame (~18 bytes)

Each active `FOR` loop pushes a substantial frame onto the stack so that `NEXT` can find the loop parameters:

Offset	Size	Content
+0	1 byte	FOR token ($81) — identifies this as a FOR frame
+1	4 bytes	Loop limit (floating-point)
+5	4 bytes	Step value (floating-point)
+9	1 byte	Step sign
+10	2 bytes	Current line number
+12	2 bytes	Text pointer (resume position in program text)
+14	2 bytes	FOR variable pointer (address of loop counter)

That is approximately 18 bytes per `FOR` loop. The stack can hold at most about seven nested `FOR`/`NEXT` loops before running into trouble.

2.2.4 EhBASIC GOSUB Frame (5 bytes)

A `GOSUB` call pushes a much smaller frame:

Offset	Size	Content
+0	1 byte	GOSUB token ($89)
+1	2 bytes	Current line number
+3	2 bytes	Text pointer (resume position after RETURN)

Five bytes per `GOSUB` means you can nest about 12 subroutine calls before the stack fills up.

2.3 Stack Pointer Initialisation

When the NovaVM resets, the CPU fetches the reset vector from $FFFC–$FFFD and jumps to the monitor code at $FF80. One of the first things the monitor does is initialise the stack pointer:

```
LDX #$FF
TXS
```

This sets SP to $FF, placing the stack at the very top of page 1. EhBASIC then takes over and begins using the stack for its own purposes. By the time you see the `Ready` prompt, SP is typically somewhere around $F8 — the interpreter keeps a few bytes on the stack at all times for its main loop.

2.4 Reading the Stack Pointer

The 6502's stack pointer is an internal CPU register and cannot be read directly with PEEK. However, the `TSX` instruction copies SP into X, and a small machine-language routine can make this value available to BASIC:

```
10 REM read current stack pointer
20 REM SP is not directly readable from BASIC,
30 REM but you can watch stack depth via USR()
40 REM or CALL to a small assembly routine
```

You can also get a rough sense of stack usage by examining memory near $01FF and looking for the boundary between pushed data and uninitialised space.

Practical Limits

In practice, the stack has room for about 12 nested GOSUB calls or 7 nested FOR/NEXT loops before overflow. A mix of both will reduce these limits further. If your program uses deep recursion via GOSUB, consider restructuring it with GOTOs or moving the inner logic into machine language.

Warning

Stack overflow in EhBASIC manifests as bizarre behaviour: variables change unexpectedly, the program jumps to wrong lines, or the interpreter locks up entirely. Unlike modern systems, there is no "stack overflow" error message. If your program behaves erratically after adding nested loops or subroutines, check your nesting depth first.

Chapter 3

The Vector Table

"Never hardcode what you can look up. Addresses change; indirection is forever."

— *Pragmatic advice from a systems programmer, ca. 1985*

Introduction

When the NovaVM boots, the monitor code at $FF80 writes a table of 16-bit addresses into the bottom of page 2. Each entry holds the base address of a major hardware device — the graphics controller, the SID chip, the file system, the network interface, and so on. Programs that read these vectors instead of hardcoding device addresses will continue to work even if the hardware map changes in a future NovaVM version.

The vector table is ordinary RAM. The system writes it once at boot, and programs are free to read it at any time. Writing to the table is technically possible but strongly discouraged — altering a vector will not move the hardware, it will just confuse any code that reads the table afterward.

The 128-byte region from $0200 to $027F is reserved for the system vector table. The first 24 bytes ($0200–$0217) contain twelve 16-bit vectors, stored in standard 6502 little-endian format (low byte at the even address, high byte at the odd address). The remaining 104 bytes ($0218–$027F) are currently reserved.

3.1 Vector Entries

Each vector is a two-byte pointer to the base register of a hardware controller. Use `DEEK` to read a vector as a 16-bit address:

```
PRINT "$";HEX$(DEEK($0200))
```

This prints `$A000` — the VGC base address.

$0200 **VgcBase (low)** R/W | Default: $00

$0201 **VgcBase (high)** R/W | Default: $A0

Points to $A000, the base of the Virtual Graphics Controller register block. The VGC core registers (mode, colours, cursor, scroll, collision) begin here. See Chapter 5.2 for full VGC documentation.

```
VB = DEEK($0200)
POKE VB+1, 6 : REM set background to blue
```

$0202 **RegCmdAddr (low)** R/W | Default: $10

$0203 **RegCmdAddr (high)** R/W | Default: $A0

Points to $A010, the VGC command register. Writing a command byte here triggers immediate execution of graphics primitives (plot, line, circle, fill) and sprite operations. Parameters are loaded into $A011–$A01E before the command write. See Chapter 5.2 for the complete command list.

$0204 **CharRamBase (low)** R/W | Default: $00

$0205 **CharRamBase (high)** R/W | Default: $AA

Points to $AA00, the start of the 2000-byte character RAM (80 columns × 25 rows). Each byte holds an ASCII character code. See Chapter 8.1 for screen memory details.

```
CB = DEEK($0204)
POKE CB, ASC("H") : REM put 'H' at top-left
```

$0206 **ColorRamBase (low)** R/W | Default: $D0

$0207 **ColorRamBase (high)** R/W | Default: $B1

Points to $B1D0, the start of the 2000-byte color RAM. Each byte holds the foreground colour for the corresponding character position. See Chapter 8.1.

$0208 **SidBase (low)** R/W | Default: $00

$0209 **SidBase (high)** R/W | Default: $D4

Points to $D400, the base of SID chip 1. The SID provides three analogue synthesis voices with ADSR envelopes, ring modulation, and programmable filters. See Chapter 12.1 for register details.

$020A **FileIoBase (low)** R/W | Default: $A0

$020B **FileIoBase (high)** R/W | Default: $B9

Points to $B9A0, the File I/O controller. This device handles loading and saving BASIC programs, directory listing, graphics save/load, SID file playback, and music sequencing. See Chapter 9.2.

$020C **XmcBase (low)** R/W | Default: $00

$020D **XmcBase (high)** R/W | Default: $BA

Points to $BA00, the Expansion Memory Controller. The XMC provides access to 512 KB of expansion RAM through a register-based command interface and four 256-byte memory windows at $BC00–$BFFF. See Chapter 10.1.2.

$020E **TimerBase (low)** R/W | Default: $40

$020F **TimerBase (high)** R/W | Default: $BA

Points to $BA40, the Timer controller. A simple programmable interval timer that generates IRQ interrupts at a configurable rate, useful for game loops and real-time tasks.

$0210 **Sid2Base (low)** R/W | Default: $20

$0211 **Sid2Base (high)** R/W | Default: $D4

Points to $D420, the base of SID chip 2. A second complete SID with three additional synthesis voices, giving six SID voices total. See Chapter 12.1.

$0212 **NicBase (low)** R/W | Default: $00

$0213 **NicBase (high)** R/W | Default: $A1

Points to $A100, the Network Interface Controller. The NIC provides four TCP connection slots for client and server networking with DMA message transfers and optional per-slot IRQ notification. See Chapter 6.1.

$0214 **DmaBase (low)** R/W | Default: $63

$0215 **DmaBase (high)** R/W | Default: $BA

Points to $BA63, the DMA controller. High-speed block transfers between CPU RAM, VGC memory spaces (character, colour, graphics, sprite), and expansion RAM — all without tying up the CPU.

$0216 **BlitterBase (low)** R/W | Default: $83

$0217 **BlitterBase (high)** R/W | Default: $BA

Points to $BA83, the Blitter controller. A 2D block-transfer engine that supports rectangular copy, fill, and colour-key transparency operations across all memory spaces.

3.2 Reserved Range ($0218–$027F)

The 104 bytes from $0218 to $027F are currently unused. They are ordinary RAM and could be used by your own programs for variables or small data tables. However, future NovaVM versions may add new hardware vectors here, so it is best to treat this range as reserved if you want your programs to remain forward-compatible.

3.3 Reading the Complete Vector Table

The following program dumps all twelve vectors:

```
10 REM dump the vector table
20 FOR I = $0200 TO $0216 STEP 2
30   PRINT "$"; HEX$(I); " -> $"; HEX$(DEEK(I))
40 NEXT I
```

On a standard NovaVM, this produces:

```
$0200 -> $A000
$0202 -> $A010
$0204 -> $AA00
$0206 -> $B1D0
$0208 -> $D400
$020A -> $B9A0
$020C -> $BA00
$020E -> $BA40
$0210 -> $D420
$0212 -> $A100
$0214 -> $BA63
$0216 -> $BA83
```

Programs should read the vector table rather than hardcoding device addresses. This ensures forward compatibility if hardware base addresses change in future NovaVM versions. For example, instead of writing directly to $A001, use:

```
VB = DEEK($0200)
POKE VB+1, 6
```

Summary

Address	Vector Name	Points To	Device
$0200	VgcBase	$A000	VGC core registers
$0202	RegCmdAddr	$A010	VGC command register
$0204	CharRamBase	$AA00	Character RAM
$0206	ColorRamBase	$B1D0	Color RAM
$0208	SidBase	$D400	SID chip 1
$020A	FileIoBase	$B9A0	File I/O controller
$020C	XmcBase	$BA00	Expansion memory controller
$020E	TimerBase	$BA40	Timer controller
$0210	Sid2Base	$D420	SID chip 2
$0212	NicBase	$A100	Network interface controller
$0214	DmaBase	$BA63	DMA controller
$0216	BlitterBase	$BA83	Blitter controller
$0218–$027F	*Reserved for future use*		

Chapter 4

BASIC Program Space

"Forty kilobytes ought to be enough for anybody's BASIC program."

— *Apocryphal, ca. 1982*

Introduction

The roughly 40 KB region from $0280 to $9FFF is where BASIC does all of its living. Every line you type, every variable you assign, every array you dimension, and every string you concatenate ends up somewhere in this space. Understanding how the interpreter organises its data here is essential for writing programs that are memory-efficient, for debugging mysterious `?OUT OF MEMORY` errors, and for writing machine-language routines that can inspect or manipulate BASIC's internal state.

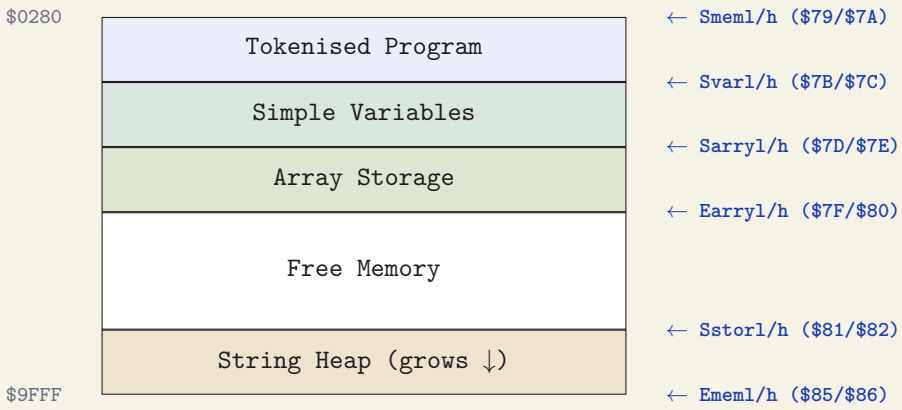
The region is not one flat pool. It is divided into four distinct areas — program text, simple variables, array storage, and the string heap — whose boundaries shift dynamically as your program runs. Six zero-page pointer pairs (documented in Chapter 1) track where each area begins and ends.

4.1 Memory Layout Overview

$0280–$9FFF **BASIC Program Space** R/W | Default: 0

The entire region is ordinary read/write RAM. At boot, EhBASIC sets the program-start pointer (`$0079`/`$007A`) to $0280 and the end-of-memory pointer (`$0085`/`$0086`) to $A000. Everything in between is available for your program and its data.

The layout from low addresses to high is:



The invariant is always:

$$\texttt{Smem} \leq \texttt{Svar} \leq \texttt{Sarry} \leq \texttt{Earry} \leq \texttt{Sstor} \leq \texttt{Emem}$$

Program text, variables, and arrays grow *upward* from $0280. The string heap grows *downward* from $9FFF. When `Earry` meets `Sstor`, memory is exhausted and EhBASIC raises `?OUT OF MEMORY`.

You can watch these pointers in real time:

```
10 PRINT "PROG  :$";HEX$(DEEK($79))
20 PRINT "VARS  :$";HEX$(DEEK($7B))
30 PRINT "ARRAYS:$";HEX$(DEEK($7D))
40 PRINT "EARRY :$";HEX$(DEEK($7F))
50 PRINT "SSTOR :$";HEX$(DEEK($81))
60 PRINT "EMEM  :$";HEX$(DEEK($85))
```

4.2 Tokenised Line Format

BASIC programs are not stored as ASCII text. When you enter a line, EhBASIC *tokenises* it — replacing keywords with single-byte (or two-byte) tokens, while leaving literal text, numbers, and punctuation untouched. This saves memory and speeds up the interpreter, which can dispatch on a single byte rather than parsing multi-character keywords.

Each line in memory has the following structure:

Offset	Size	Description
+0	2 bytes	Pointer to next line (little-endian). $0000 marks end of program.
+2	2 bytes	Line number (little-endian, 0–65535).
+4	n bytes	Tokenised content.
+4+n	1 byte	$00 terminator.

Lines form a singly linked list. The interpreter walks the list by reading the next-line pointer; when it finds $0000, the program is over. Line numbers need not be contiguous — `10`, `20`, `100` is perfectly valid.

4.2.1 Worked Example: `10 PRINT "HELLO"`

Suppose this one-line program is stored beginning at $0280. Here is the byte-by-byte hex dump:

Address	Hex	Meaning
$0280	`8D 02`	Next-line pointer = $028D
$0282	`0A 00`	Line number = $000A (10)
$0284	`9F`	PRINT token
$0285	`20`	Space (literal)
$0286	`22`	`"` (open quote)
$0287	`48`	`H`
$0288	`45`	`E`
$0289	`4C`	`L`
$028A	`4C`	`L`
$028B	`4F`	`O`
$028C	`00`	End-of-line terminator
$028D	`00 00`	Next-line pointer = $0000 (end of program)
$028F	`00 00`	(padding / end marker)

The closing quote character (`$22`) is not stored — EhBASIC strips trailing quotes at the end of `PRINT` arguments. The string literal ends at the $00 line terminator.

4.3 Token Table

NovaBASIC uses two token spaces. **Primary tokens** occupy the range $80–$FF (one byte each). **Extended tokens** are two bytes: a prefix byte $01 followed by a sub-token byte.

4.3.1 Primary Tokens ($80–$FF)

Command Tokens

Hex	Keyword	Hex	Keyword	Hex	Keyword
$80	END	$90	RETURN	$A0	CONT
$81	FOR	$91	REM	$A1	LIST
$82	NEXT	$92	STOP	$A2	CLEAR
$83	DATA	$93	ON	$A3	NEW
$84	INPUT	$94	NULL	$A4	WIDTH
$85	DIM	$95	INC	$A5	GET
$86	READ	$96	WAIT	$A6	SWAP
$87	LET	$97	LOAD	$A7	BITSET
$88	DEC	$98	SAVE	$A8	BITCLR
$89	GOTO	$99	DEF	$A9	IRQ
$8A	RUN	$9A	POKE	$AA	NMI
$8B	IF	$9B	DOKE	$AB	CLS
$8C	RESTORE	$9C	SYS	$AC	COLOR
$8D	GOSUB	$9D	DO	$AD	LOCATE
$8E	RETIRQ	$9E	LOOP	$AE	PLOT

Hex	Keyword	Hex	Keyword	Hex	Keyword
$8F	RETNMI	$9F	PRINT	$AF	UNPLOT

Hex	Keyword	Hex	Keyword	Hex	Keyword
$B0	LINE	$B8	SPRITE	$C0	VSYNC
$B1	CIRCLE	$B9	SPRITESHAPE	$C1	TAB(
$B2	RECT	$BA	SPRITESET	$C2	ELSE
$B3	FILL	$BB	SPRITEDATA	$C3	TO
$B4	PAINT	$BC	SOUND	$C4	FN
$B5	MODE	$BD	VOLUME	$C5	SPC(
$B6	GCLS	$BE	INSTRUMENT	$C6	THEN
$B7	GCOLOR	$BF	WAVE	$C7	NOT

Operator and Structure Tokens

Hex	Token	Hex	Token	Hex	Token
$C8	STEP	$CC	+	$D0	^
$C9	UNTIL	$CD	-	$D1	AND
$CA	WHILE	$CE	*	$D2	EOR
$CB	OFF	$CF	/	$D3	OR
		$D4	»	$D6	>
		$D5	«	$D7	=
				$D8	<

Function Tokens

Hex	Function	Hex	Function	Hex	Function
$D9	SGN(	$E3	BITTST(	$ED	SPRITEX(
$DA	INT(	$E4	MAX(	$EE	SPRITEY(
$DB	ABS(	$E5	MIN(	$EF	COLLISION(
$DC	USR(	$E6	PI	$F0	BUMPED(
$DD	FRE(	$E7	TWOPI		
$DE	POS(	$E8	VARPTR(		
$DF	SQR(	$E9	LEFT$(		
$E0	RND(	$EA	RIGHT$(		
$E1	LOG(	$EB	MID$(		
$E2	EXP(	$EC	— (reserved)		
Continued:					
$E3	COS(	$E7	PEEK(	$EB	STR$(
$E4	SIN(	$E8	DEEK(	$EC	VAL(
$E5	TAN(	$E9	SADD(	$ED	ASC(
$E6	ATN(	$EA	LEN(	$EE	UCASE$(

Warning

The function token table above is presented in the order matching the `tokens.json` source file. Here is the complete mapping from $D9 to $FF:

Hex	Function	Hex	Function
$D9	SGN(	$E9	SADD(
$DA	INT(	$EA	LEN(
$DB	ABS(	$EB	STR$(
$DC	USR(	$EC	VAL(
$DD	FRE(	$ED	ASC(
$DE	POS(	$EE	UCASE$(
$DF	SQR(	$EF	LCASE$(
$E0	RND(	$F0	CHR$(
$E1	LOG(	$F1	HEX$(
$E2	EXP(	$F2	BIN$(
$E3	COS(	$F3	BITTST(
$E4	SIN(	$F4	MAX(
$E5	TAN(	$F5	MIN(
$E6	ATN(	$F6	PI
$E7	PEEK(	$F7	TWOPI
$E8	DEEK(	$F8	VARPTR(
		$F9	LEFT$(
		$FA	RIGHT$(
		$FB	MID$(
		$FC	SPRITEX(
		$FD	SPRITEY(
		$FE	COLLISION(
		$FF	BUMPED(

4.3.2 Extended Tokens ($01 + sub-token)

When the tokeniser encounters a keyword that does not fit in the primary table, it emits the prefix byte $01 followed by a sub-token. These are the NovaVM extensions — commands and functions not present in the original EhBASIC.

Prefix	Sub	Keyword	Prefix	Sub	Keyword
$01	$01	DIR	$01	$19	NOPEN
$01	$02	DEL	$01	$1A	NCLOSE
$01	$03	XMEM	$01	$1B	NLISTEN
$01	$04	XBANK	$01	$1C	NACCEPT
$01	$05	XPOKE	$01	$1D	NSEND
$01	$06	XPEEK(	$01	$1E	NRECV$(
$01	$07	STASH	$01	$1F	NSTATUS(
$01	$08	FETCH	$01	$21	NREADY(

Prefix	Sub	Keyword	Prefix	Sub	Keyword
$01	$09	XFREE	$01	$22	NLEN
$01	$0A	XRESET	$01	$23	DMACOPY
$01	$0B	XALLOC	$01	$24	DMAFILL
$01	$0C	XDIR	$01	$25	DMASTATUS
$01	$0D	XDEL	$01	$27	DMAERR
$01	$0E	XMAP	$01	$28	DMACOUNT
$01	$0F	XUNMAP	$01	$29	MIDPLAY
$01	$10	GSAVE	$01	$2A	MIDSTOP
$01	$11	GLOAD	$01	$2B	DIROPEN
$01	$12	SIDPLAY	$01	$2C	DIRNEXT
$01	$13	SIDSTOP	$01	$2D	DIRNAM$(
$01	$14	MUSIC	$01	$2E	DIRSIZ
$01	$15	PLAYING	$01	$2F	DIRTYP
$01	$16	MNOTE(	$01	$30	META$(
$01	$17	COPPER			
$01	$18	RESET			
$01	$40	BLITCOPY	$01	$46	HELP
$01	$41	BLITFILL	$01	$47	NCC
$01	$42	BLITSTATUS	$01	$48	FONT
$01	$43	BLITERR	$01	$49	CD
$01	$44	BLITCOUNT	$01	$4A	MKDIR
$01	$45	BITTGL	$01	$4B	RMDIR
			$01	$4C	FORMAT
			$01	$4D	MOUNT
			$01	$4E	UNMOUNT
			$01	$4F	PWD

Inside quoted strings, no tokenisation occurs. The bytes between double quotes are stored as literal ASCII. This is why you can safely embed keyword substrings in strings: `"FORGET"` does not become `"$81GET"`.

4.4 Simple Variable Storage

Immediately after the last program line (at the address in `$007B`/`$007C`), EhBASIC stores simple (non-array) variables. Each variable occupies either six or seven bytes, depending on its type.

4.4.1 Variable Name Encoding

The first two bytes of each variable entry encode the name:

Byte	Contents
Byte 1	First significant character of the name (A–Z). Bit 7 set indicates a string variable (`$` suffix).
Byte 2	Second significant character, or $00 if the name is a single letter. Bit 7 set indicates an integer function variable.

EhBASIC only distinguishes variables by their first two characters. `SCORE` and `SCREEN` are the same variable.

4.4.2 Numeric Variables

A numeric variable entry is six bytes:

Offset	Contents
+0	Name byte 1 (bit 7 clear = numeric)
+1	Name byte 2
+2..+5	4-byte floating-point value (EhBASIC internal format)

EhBASIC uses a non-standard 32-bit floating-point format: 1 byte exponent (biased by 128), followed by a 24-bit mantissa with an implied leading 1 bit. The sign is stored in the high bit of the first mantissa byte. This gives roughly 6.5 decimal digits of precision.

4.4.3 String Variables

A string variable entry is also six bytes, but the value portion is different:

Offset	Contents
+0	Name byte 1 (bit 7 set = string)
+1	Name byte 2
+2	String length (0–255)
+3	(unused / padding)
+4..+5	Pointer to string data in the string heap (little-endian)

The string data itself lives in the string heap at the top of memory. The variable entry only holds a descriptor pointing to it.

4.5 Array Storage

Arrays are stored after simple variables, between `$007D`/`$007E` and `$007F`/`$0080`. Each array entry consists of a header followed by packed element values.

4.5.1 Array Header

Offset	Contents
+0	Name byte 1 (same encoding as simple variables)
+1	Name byte 2
+2..+3	Total size of this array entry in bytes (little-endian), including the header
+4	Number of dimensions (1–255)
+5..+n	2 bytes per dimension: element count (little-endian). `DIM A(9)` stores 10 (0 through 9).

4.5.2 Array Element Data

After the dimension sizes, elements are stored in row-major order:

- **Numeric arrays**: 4 bytes per element (same floating-point format as simple variables).

- **String arrays**: 3 bytes per element (1 byte length + 2 bytes pointer into the string heap).

Warning

Arrays consume memory rapidly. `DIM A(100)` allocates a header plus $101 \times 4 = 404$ bytes for numeric data. A two-dimensional array `DIM B(10,10)` needs $11 \times 11 \times 4 = 484$ bytes. Check `FRE(0)` before dimensioning large arrays.

4.6 String Heap

String data occupies the top of the BASIC program space, growing *downward* from `$0085`/`$0086` (Ememl/h). The current bottom of the heap is tracked by `$0081`/`$0082` (Sstorl/h).

When you assign `A$="HELLO"`, EhBASIC:

1. Decrements Sstor by the string length (5).
2. Copies the five bytes into the new location.
3. Updates the variable entry's pointer to the new address.

4.6.1 Garbage Collection

Over time, reassigning strings leaves orphaned data in the heap. When free memory runs low, EhBASIC performs *garbage collection*: it scans all string variables and array elements, compacts the live strings toward the top of memory, and reclaims the dead space.

This can cause a noticeable pause — the dreaded "BASIC hiccup" familiar to 8-bit programmers. Programs that do heavy string manipulation may experience periodic delays.

You can force garbage collection at a convenient time by calling `FRE("")`. This is useful before a timing-critical section:

```
1000 X=FRE("") : REM compact strings now
1010 REM -- timing-sensitive code follows --
```

4.7 Free Memory

The free memory region is the gap between the end of array storage (`$007F`/`$0080`) and the bottom of the string heap (`$0081`/`$0082`). The `FRE(0)` function returns the size of this gap in bytes.

```
10 PRINT "FREE MEMORY:"; FRE(0); "BYTES"
20 DIM A(100)
30 PRINT "AFTER DIM:"; FRE(0); "BYTES"
```

On a freshly booted NovaVM with no program loaded, free memory is approximately 40,192 bytes ($9D00 in hex) — the full span from $0280 to $9FFF minus the few bytes EhBASIC uses for the end-of-program marker.

`FRE(0)` returns a numeric result. `FRE("")` also triggers garbage collection and then returns the free count. Always use `FRE("")` if you want an accurate count after heavy string use.

4.8 Walking the Data Structures

The linked-list nature of tokenised program lines and the packed variable tables can be explored directly from BASIC using PEEK and DEEK.

4.8.1 Walking the Program Line List

This program follows the next-line pointers from the start of the program, printing each line number it finds:

```
1000 REM -- Walk program lines --
1010 P=DEEK($79) : REM start of program
1020 NP=DEEK(P) : REM next-line pointer
1030 IF NP=0 THEN PRINT "END OF PROGRAM" : END
1040 LN=DEEK(P+2) : REM line number
1050 SZ=NP-P : REM line size in bytes
1060 PRINT "LINE";LN;"AT $";HEX$(P);"SIZE";SZ
1070 P=NP : GOTO 1020
```

4.8.2 Dumping a Tokenised Line

This program dumps the raw bytes of a specific line, showing how tokens replace keywords:

```
2000 REM -- Hex dump of a BASIC line --
2010 INPUT "LINE NUMBER";TL
2020 P=DEEK($79)
2030 NP=DEEK(P) : IF NP=0 THEN PRINT "NOT FOUND" : END
2040 LN=DEEK(P+2) : IF LN=TL THEN 2070
2050 P=NP : GOTO 2030
2060 REM -- Found it --
2070 SZ=NP-P
2080 PRINT "LINE";TL;"AT $";HEX$(P);"-";
2090 PRINT "$";HEX$(P+SZ-1);" (";SZ;"BYTES)"
2100 FOR I=0 TO SZ-1
2110 PRINT HEX$(PEEK(P+I));" ";
2120 IF (I AND 15)=15 THEN PRINT
2130 NEXT I
2140 PRINT
```

4.8.3 Showing the Memory Map

A quick-reference program that displays all six pointer pairs and the computed region sizes:

```
3000 REM -- BASIC memory map --
3010 SM=DEEK($79):SV=DEEK($7B)
3020 SA=DEEK($7D):EA=DEEK($7F)
3030 SS=DEEK($81):EM=DEEK($85)
3040 PRINT "PROGRAM  : $";HEX$(SM);" - $";HEX$(SV-1)
3050 PRINT "          ";SV-SM;"BYTES"
3060 PRINT "VARIABLES: $";HEX$(SV);" - $";HEX$(SA-1)
3070 PRINT "          ";SA-SV;"BYTES"
3080 PRINT "ARRAYS   : $";HEX$(SA);" - $";HEX$(EA-1)
3090 PRINT "          ";EA-SA;"BYTES"
3100 PRINT "FREE     : $";HEX$(EA);" - $";HEX$(SS-1)
3110 PRINT "          ";SS-EA;"BYTES"
3120 PRINT "STRINGS  : $";HEX$(SS);" - $";HEX$(EM-1)
3130 PRINT "          ";EM-SS;"BYTES"
```

```
3140 PRINT
3150 PRINT "FRE(0) = ";FRE(0)
```

Chapter 5

The Virtual Graphics Controller

"Pixels are the atoms of the digital universe. If you can control them, you can build anything."

— *Overheard at a demo party, 1987*

Introduction

The Virtual Graphics Controller — VGC for short — is the heart of the NovaVM's display system. It manages three independent layers of visual output: an 80×25 text screen, a 320×200 pixel bitmap with 16 colours, and 16 multicolour 16×16 hardware sprites. All three layers are composited at 60 Hz into a single frame, with the rendering order determined by the active display mode.

From the programmer's perspective, the VGC is a collection of memory-mapped registers starting at $A000. The core registers at $A000–$A00F control mode, colour, cursor, and status. A command interface at $A010–$A01E provides graphics primitives, sprite manipulation, copper (raster) effects, and memory I/O. Finally, 128 bytes of sprite registers at $A040–$A0BF give direct access to each sprite's position, shape, and flags.

This chapter documents every register and command in the VGC. It is the largest chapter in the book for good reason: the VGC is where programs become visible.

5.1 Display Modes

The VGC supports four display modes, selected by writing to `$A000`:

Mode	Name	Description
0	Text only	80×25 text screen. No graphics bitmap is drawn. This is the default mode at power-on.
1	Gfx over text	The graphics bitmap is drawn on top of the text layer. Non-zero pixels in the bitmap obscure the text beneath.
2	Text over gfx	The text layer is drawn on top of the graphics bitmap. Characters with non-background colours obscure the bitmap.
3	Gfx + sprites	Graphics bitmap and sprites only, no text layer. Ideal for full-screen graphical games.

Rendering Order

Each frame at 60 Hz proceeds in the following order:

1. Background colour fill (the entire frame is cleared to $A001).
2. Priority-0 sprites are drawn (behind everything).
3. Text and/or graphics layers, in the order determined by the display mode.
4. Priority-1 sprites are drawn (between text and graphics layers).
5. Priority-2 sprites are drawn (in front of everything).
6. Collision detection is performed and results are latched into $A00B and $A00C.

```
10 MODE 2 : REM text over graphics
20 GCLS   : REM clear bitmap
30 CIRCLE 160,100,80,1
40 PRINT "HELLO FROM TEXT LAYER"
```

5.2 Core Registers ($A000–$A00F)

These sixteen registers control the fundamental display state: mode, colours, cursor position, scrolling, and I/O.

$A000 RegMode

R/W | Default: 0

Display mode selector. Write a value from 0 to 3 to select the active display mode (see Section 5.1).

```
POKE $A000,1 : REM graphics over text
```

The BASIC command `MODE n` writes to this register.

$A001 RegBgCol

R/W | Default: 0

Background colour (0–15). This colour fills the entire screen behind all layers. It is also the "paper" colour for the text layer — character cells that are not covered by text are drawn in this colour.

```
POKE $A001,6 : REM dark blue background
```

`COLOR fg,bg` sets both $A002 and $A001 simultaneously.

$A002 RegFgCol

R/W | Default: 1

Foreground/text colour (0–15). Characters printed after changing this register will appear in the new colour.

```
POKE $A002,7 : REM white text
PRINT "NOW IN WHITE"
```

$A003 RegCursorX

R/W | Default: 0

Cursor column position (0–79). Reading returns the current column; writing moves the cursor.

```
POKE $A003,40 : REM move cursor to column 40
PRINT "CENTERED-ISH"
```

The BASIC command `LOCATE x,y` sets both $A003 and $A004.

$A004 RegCursorY

R/W | Default: 0

Cursor row position (0–24). Row 0 is the top of the screen.

```
POKE $A004,12 : REM move cursor to row 12
```

$A005 RegScrollX

R/W | Default: 0

Horizontal scroll offset for the text layer. Writing a non-zero value shifts the text layer left by that many pixels.

```
POKE $A005,4 : REM shift text 4 pixels left
```

$A006 RegScrollY

R/W | Default: 0

Vertical scroll offset for the text layer. Shifts the text layer up by the specified number of pixels.

```
POKE $A006,2 : REM shift text 2 pixels up
```

$A007 RegFont

R/W | Default: 0

Active font slot (0–7, 3 bits). The VGC supports up to eight loaded font definitions. Slot 0 is the built-in system font.

```
POKE $A007,1 : REM switch to font slot 1
```

$A008 RegStatus

R | Default: 0

Frame counter. Increments by one at each vertical blank (60 Hz) and wraps from 255 back to 0. This is the simplest way to synchronise with the display refresh.

```
10 F = PEEK($A008)
20 IF PEEK($A008) = F THEN 20
30 REM -- new frame has started --
```

The BASIC command `VSYNC` is equivalent to the wait loop above: it blocks until the frame counter changes, then returns.

$A009 RegSpriteCount

R | Default: 0

Returns the number of sprites currently enabled (0–16).

```
PRINT PEEK($A009);" SPRITES ACTIVE"
```

$A00A RegCursorEnable

R/W | Default: 1

Cursor visibility. Any non-zero value displays the blinking cursor; zero hides it. Useful for games and full-screen applications where the cursor is a distraction.

```
POKE $A00A,0   : REM hide cursor
POKE $A00A,1   : REM show cursor
```

$A00B RegColSt

R | Default: 0

Sprite-to-sprite collision bitmask. After each frame, any sprite that overlaps with another sprite has its corresponding bit set. Bit N represents sprite N (bit 0 = sprite 0, bit 7 = sprite 7). For sprites 8–15 use the BASIC `COLLISION()` function.

Warning

Reading this register **clears** it. If you need to test multiple bits, read the value into a variable first.

```
10 C = PEEK($A00B)
20 IF C AND 1 THEN PRINT "SPRITE 0 HIT SOMETHING"
30 IF C AND 2 THEN PRINT "SPRITE 1 HIT SOMETHING"
```

$A00C RegColBg

R | Default: 0

Sprite-to-background collision bitmask. Bit N is set when sprite N overlaps a non-zero pixel in the graphics bitmap. Reading clears the register, just like `$A00B`.

```
10 B = PEEK($A00C)
20 IF B AND 1 THEN PRINT "SPRITE 0 HIT BACKGROUND"
```

$A00D RegBorder

R/W | Default: 0

Border (frame) colour (0–15). The border is the area outside the active display area, if the host window is larger than the 320×200 pixel viewport.

```
POKE $A00D,2 : REM red border
```

$A00E RegCharOut

W | Default: —

Character output port. Writing a byte here places it on the screen at the current cursor position and advances the cursor, exactly as if the character had been PRINTed. Control codes (carriage return, backspace, etc.) are interpreted.

This is how BASIC's `PRINT` statement works internally — every character ultimately arrives here.

```
10 POKE $A00E,72   : REM 'H'
20 POKE $A00E,73   : REM 'I'
30 POKE $A00E,13   : REM carriage return
```

$A00F RegCharIn

R | Default: —

Character input port. Returns the ASCII code of the next key in the keyboard queue, or 0 if no key is pending. This is how `INPUT` and `GET` work internally — the interpreter polls this register until a key arrives.

```
10 K = PEEK($A00F)
20 IF K = 0 THEN 10
30 PRINT "YOU PRESSED: ";CHR$(K)
```

5.3 Command Interface ($A010–$A01E)

The VGC command interface provides a rich set of graphics primitives, sprite manipulation, copper (raster) effects, and memory I/O — all triggered by writing a command byte to the command register.

$A010 **RegCmd** W | Default: —

Command register. Writing a command byte here executes the command immediately. Parameters must be loaded into P0–P13 ($A011–$A01E) *before* writing the command byte.

$A011-$A01E **RegP0–RegP13 (Parameter Registers)** R/W | Default: 0

Fourteen parameter registers, addressed as $A011 (P0) through $A01E (P13). Each command uses a different subset of these registers; unused parameters are ignored. Parameters are readable, so you can inspect the last-written values at any time.

Address	Name	Offset from $A011
$A011	P0	+0
$A012	P1	+1
$A013	P2	+2
$A014	P3	+3
$A015	P4	+4
$A016	P5	+5
$A017	P6	+6
$A018	P7	+7
$A019	P8	+8
$A01A	P9	+9
$A01B	P10	+10
$A01C	P11	+11
$A01D	P12	+12
$A01E	P13	+13

Many parameters represent 16-bit values split across two consecutive registers in little-endian order (low byte first, high byte second). For example, an X coordinate of 260 would be stored as P0=$04 (low), P1=$01 (high): $260 = 256 \times 1 + 4$.

5.3.1 Graphics Commands

$01 **CmdPlot** *Plot a single pixel*

Sets a pixel in the graphics bitmap.

Param	Meaning
P0, P1	X coordinate (16-bit LE, 0–319)
P2, P3	Y coordinate (16-bit LE, 0–199)
P8	Colour (0–15)

```
10 REM plot a red pixel at (100, 50)
20 POKE $A011,100 : POKE $A012,0 : REM X=100
30 POKE $A013,50  : POKE $A014,0 : REM Y=50
40 POKE $A019,2                  : REM colour=red
50 POKE $A010,$01                : REM execute PLOT
```

The BASIC command `PLOT x,y,c` does exactly this.

$02 CmdUnplot *Erase a pixel (plot colour 0)*

Identical to CmdPlot but always uses colour 0 (the background colour), effectively erasing the pixel.

Param	Meaning
P0, P1	X coordinate (16-bit LE)
P2, P3	Y coordinate (16-bit LE)

```
10 POKE $A011,100 : POKE $A012,0
20 POKE $A013,50  : POKE $A014,0
30 POKE $A010,$02  : REM erase pixel at (100,50)
```

BASIC equivalent: `UNPLOT x,y`.

$03 CmdLine *Draw a line*

Draws a straight line between two endpoints using Bresenham's algorithm.

Param	Meaning
P0, P1	X1 (16-bit LE)
P2, P3	Y1 (16-bit LE)
P4, P5	X2 (16-bit LE)
P6, P7	Y2 (16-bit LE)
P8	Colour (0–15)

```
10 REM line from (0,0) to (319,199) in white
20 POKE $A011,0 : POKE $A012,0 : REM X1=0
30 POKE $A013,0 : POKE $A014,0 : REM Y1=0
40 POKE $A015,$3F : POKE $A016,$01 : REM X2=319
50 POKE $A017,$C7 : POKE $A018,$00 : REM Y2=199
60 POKE $A019,1 : REM colour=white
70 POKE $A010,$03 : REM execute LINE
```

BASIC equivalent: `LINE x1,y1,x2,y2,c`.

$04 CmdCircle *Draw a circle outline*

Draws a circle using the midpoint circle algorithm.

Param	Meaning
P0, P1	Centre X (16-bit LE)
P2, P3	Centre Y (16-bit LE)
P4, P5	Radius (16-bit LE)
P8	Colour (0–15)

```
10 REM circle at centre (160,100) radius 50, green
20 POKE $A011,160 : POKE $A012,0
30 POKE $A013,100 : POKE $A014,0
40 POKE $A015,50  : POKE $A016,0
50 POKE $A019,5   : REM colour=green
60 POKE $A010,$04  : REM execute CIRCLE
```

BASIC equivalent: `CIRCLE x,y,r,c`.

$05 CmdRect *Draw a rectangle outline*

Draws a rectangular outline from the top-left corner to the bottom-right corner.

Param	Meaning
P0, P1	X1 top-left (16-bit LE)
P2, P3	Y1 top-left (16-bit LE)
P4, P5	X2 bottom-right (16-bit LE)
P6, P7	Y2 bottom-right (16-bit LE)
P8	Colour (0–15)

```
10 REM rectangle from (10,10) to (100,80) in cyan
20 POKE $A011,10 : POKE $A012,0
30 POKE $A013,10 : POKE $A014,0
40 POKE $A015,100 : POKE $A016,0
50 POKE $A017,80  : POKE $A018,0
60 POKE $A019,3   : REM colour=cyan
70 POKE $A010,$05  : REM execute RECT
```

BASIC equivalent: `RECT x1,y1,x2,y2,c`.

$06 CmdFill *Draw a filled rectangle*

Identical to CmdRect but fills the interior with the specified colour.

Param	Meaning
P0, P1	X1 top-left (16-bit LE)
P2, P3	Y1 top-left (16-bit LE)
P4, P5	X2 bottom-right (16-bit LE)
P6, P7	Y2 bottom-right (16-bit LE)
P8	Colour (0–15)

```
10 REM filled blue box
20 POKE $A011,0 : POKE $A012,0
30 POKE $A013,0 : POKE $A014,0
40 POKE $A015,$3F : POKE $A016,$01
50 POKE $A017,$C7 : POKE $A018,$00
60 POKE $A019,6 : REM colour=blue
70 POKE $A010,$06 : REM execute FILL
```

BASIC equivalent: `FILL x1,y1,x2,y2,c`.

$07 CmdGcls *Clear graphics bitmap*

Clears the entire 320×200 graphics bitmap to colour 0. No parameters are needed.

```
POKE $A010,$07 : REM clear bitmap
```

BASIC equivalent: `GCLS`.

$08 CmdGcolor *Set default drawing colour*

Sets the default draw colour used by subsequent graphics commands when no explicit colour is provided.

Param	Meaning
P0	Colour (0–15)

```
POKE $A011,5 : POKE $A010,$08 : REM default colour = green
```

BASIC equivalent: `GCOLOR c`.

$09 CmdPaint *Flood fill from a point*

Performs a flood fill starting from the specified point, filling all connected pixels of the same colour with the new colour.

Param	Meaning
P0, P1	X coordinate (16-bit LE)
P2, P3	Y coordinate (16-bit LE)
P8	Fill colour (0–15)

```
10 CIRCLE 160,100,50,1  : REM draw circle outline
20 REM flood fill the interior with yellow
30 POKE $A011,160 : POKE $A012,0
40 POKE $A013,100 : POKE $A014,0
50 POKE $A019,7   : REM colour=yellow
60 POKE $A010,$09  : REM execute PAINT
```

BASIC equivalent: `PAINT x,y,c`.

5.3.2 Sprite — Commands

Sprites are 16×16 pixel objects with 4-bit colour per pixel (16 colours including transparent). The VGC supports 16 sprites numbered 0–15. Sprite shapes are stored in a separate shape RAM with 256 slots; each sprite register block selects which shape slot to display.

$10 CmdSprDef *Define a single sprite pixel*

Sets one pixel in a sprite's shape definition.

Param	Meaning
P0	Sprite number (0–15)
P1	Pixel X within sprite (0–15)
P2	Pixel Y within sprite (0–15)
P3	Colour (0–15, 0 = transparent)

```
10 REM set pixel (7,7) of sprite 0 to red
20 POKE $A011,0    : REM sprite 0
30 POKE $A012,7    : REM x=7
40 POKE $A013,7    : REM y=7
50 POKE $A014,2    : REM colour=red
60 POKE $A010,$10   : REM execute SPRDEF
```

$11 CmdSprRow *Load a sprite row (8 pixels)*

Loads eight pixels into one half of a sprite row. Since sprites are 16 pixels wide and each parameter byte holds one 4-bit colour value, two CmdSprRow calls are needed per row — but the BASIC `SPRITEDATA` command handles this automatically.

Param	Meaning
P0	Sprite number (0–15)
P1	Row index (0–15)
P2–P9	Eight pixel colours (each 0–15)

```
10 REM load row 0 of sprite 0 with 8 red pixels
20 POKE $A011,0     : REM sprite 0
30 POKE $A012,0     : REM row 0
40 FOR I=0 TO 7
50   POKE $A013+I,2 : REM red
60 NEXT
70 POKE $A010,$11    : REM execute SPRROW
```

$12 CmdSprClr *Clear a sprite's shape data*

Resets all pixels in the specified sprite's shape to transparent (colour 0).

Param	Meaning
P0	Sprite number (0–15)

```
POKE $A011,3 : POKE $A010,$12 : REM clear sprite 3
```

$13 CmdSprCopy *Copy a sprite's shape to another*

Duplicates the shape data from one sprite to another. Useful for creating animation frames or mirrored variants.

Param	Meaning
P0	Source sprite number
P1	Destination sprite number

```
10 REM copy sprite 0's shape to sprite 1
20 POKE $A011,0 : POKE $A012,1
30 POKE $A010,$13
```

$14 CmdSprPos *Set sprite position*

Positions a sprite at the given screen coordinates. Coordinates can extend beyond the visible area for smooth entry/exit.

Param	Meaning
P0	Sprite number (0–15)
P1, P2	X position (16-bit LE)
P3, P4	Y position (16-bit LE)

```
10 REM position sprite 2 at (160, 100)
20 POKE $A011,2     : REM sprite 2
30 POKE $A012,160   : POKE $A013,0 : REM X=160
40 POKE $A014,100   : POKE $A015,0 : REM Y=100
50 POKE $A010,$14    : REM execute SPRPOS
```

The BASIC command `SPRITE n,x,y` sets position and enables the sprite in one step.

$15 CmdSprEna *Enable (show) a sprite*

Makes the sprite visible on screen. The sprite must have shape data and a position set, or it will appear as a transparent rectangle at (0,0).

Param	Meaning
P0	Sprite number (0–15)

```
POKE $A011,0 : POKE $A010,$15 : REM enable sprite 0
```

$16 CmdSprDis *Disable (hide) a sprite*

Removes the sprite from the display. Its shape and position data are preserved.

Param	Meaning
P0	Sprite number (0–15)

```
POKE $A011,0 : POKE $A010,$16 : REM disable sprite 0
```

BASIC equivalent: `SPRITE n,OFF`.

$17 CmdSprFlip *Set sprite flip flags*

Mirrors the sprite horizontally and/or vertically.

Param	Meaning
P0	Sprite number (0–15)
P1	Flip flags: bit 0 = X flip, bit 1 = Y flip

```
10 POKE $A011,0    : REM sprite 0
20 POKE $A012,1    : REM flip horizontally
30 POKE $A010,$17
```

Values for P1:

- 0 = no flip
- 1 = horizontal flip (mirror left-right)
- 2 = vertical flip (mirror top-bottom)
- 3 = both flips (180-degree rotation)

$18 CmdSprPri *Set sprite priority*

Controls the sprite's draw order relative to the text and graphics layers.

Param	Meaning
P0	Sprite number (0–15)
P1	Priority (0–2)

Priority values:

- 0 = behind all layers (drawn first)
- 1 = between text and graphics layers
- 2 = in front of all layers (drawn last)

```
10 POKE $A011,0    : REM sprite 0
20 POKE $A012,2    : REM in front of everything
30 POKE $A010,$18
```

5.3.3 Memory I/O Commands

These commands allow direct read/write access to the VGC's internal memory spaces, which are not mapped into the CPU's address space.

$19 CmdMemRead *Read from VGC memory*

Reads a byte from one of the VGC's internal memory spaces and returns it in P3.

Param	Meaning
P0	Memory space selector
P1, P2	Address within space (16-bit LE)
P3	*(output)* Data byte read
P4	Flags: bit 0 = auto-increment address

Memory space values:

- $00 = Screen RAM (2000 bytes, character codes)
- $01 = Colour RAM (2000 bytes, per-cell colour attributes)
- $02 = Graphics bitmap (64000 bytes, 320×200 pixels)
- $03 = Sprite shape RAM (256 slots × 128 bytes = 32768 bytes)

```
10 REM read the character at screen position 0
20 POKE $A011,0  : REM space = screen RAM
30 POKE $A012,0  : POKE $A013,0 : REM address = 0
40 POKE $A015,0  : REM no auto-increment
50 POKE $A010,$19 : REM execute MEMREAD
60 PRINT "CHAR CODE: ";PEEK($A014)
```

Set P4 bit 0 to enable auto-increment mode. After each read, the address in P1/P2 is automatically advanced by one, making it efficient to read consecutive bytes in a loop without rewriting the address registers.

$1A **CmdMemWrite** *Write to VGC memory*

Writes a byte to one of the VGC's internal memory spaces.

Param	Meaning
P0	Memory space selector (same as CmdMemRead)
P1, P2	Address within space (16-bit LE)
P3	Data byte to write
P4	Flags: bit 0 = auto-increment address

```
10 REM write 'A' (65) to screen position 0
20 POKE $A011,0   : REM space = screen RAM
30 POKE $A012,0   : POKE $A013,0 : REM address = 0
40 POKE $A014,65  : REM data = 'A'
50 POKE $A015,0   : REM no auto-increment
60 POKE $A010,$1A  : REM execute MEMWRITE
```

5.3.4 Copper (Raster) Commands

The copper is a raster-synchronised coprocessor inspired by the Amiga's Copper. It executes a list of register-write events keyed to specific scanline positions, allowing per-scanline colour changes, split-screen effects, and raster bars — all without CPU intervention.

The VGC supports 128 copper lists, each holding up to 256 events. One list is *active* (being executed each frame) and one is the *target* (being edited). This double-buffering allows glitch-free list updates.

$1B CmdCopperAdd *Add a copper event*

Adds an event to the target copper list.

Param	Meaning
P0, P1	Scanline (16-bit LE, 0–199)
P2	Column
P3, P4	Target register address (16-bit LE)
P5	Value to write to the register

```
10 REM change background colour at scanline 100
20 POKE $A011,100 : POKE $A012,0  : REM scanline 100
30 POKE $A013,0                   : REM column 0
40 POKE $A014,$01 : POKE $A015,$A0 : REM target = $A001 (BgCol)
50 POKE $A016,2                   : REM value = red
60 POKE $A010,$1B                 : REM add event
```

The BASIC command `COPPER` provides a higher-level interface to the copper system.

$1C CmdCopperClear *Clear the target copper list*

Removes all events from the currently selected target copper list. No parameters.

```
POKE $A010,$1C : REM clear target list
```

$1D CmdCopperEnable *Enable copper execution*

Starts executing the active copper list each frame. No parameters.

```
POKE $A010,$1D : REM enable copper
```

$1E CmdCopperDisable *Disable copper execution*

Stops copper execution. The list contents are preserved. No parameters.

```
POKE $A010,$1E : REM disable copper
```

$1F CmdSysReset *Full system reset*

Performs a complete visual and audio reset: clears the screen, silences all audio, disables all sprites, resets colours to defaults. No parameters.

```
POKE $A010,$1F : REM full reset
```

BASIC equivalent: `RESET`.

$20 CmdCopperList *Select copper list for editing*

Selects which copper list is the *target* for subsequent CmdCopperAdd and CmdCopperClear operations.

Param	Meaning
P0	List index (0–127)

```
POKE $A011,1 : POKE $A010,$20 : REM edit list 1
```

$21 **CmdCopperUse** *Activate a copper list*

Switches the active copper list to the specified index. The switch takes effect at the next vertical blank.

Param	Meaning
P0	List index (0–127)

```
POKE $A011,1 : POKE $A010,$21 : REM activate list 1
```

$22 **CmdCopperListEnd** *Set target to active list*

Sets the target copper list back to whichever list is currently active. Useful after finishing edits on a different list. No parameters.

```
POKE $A010,$22 : REM target = active list
```

5.4 IRQ Control Register ($A01F)

$A01F **RegIrqCtrl** R/W | Default: 0

Controls raster-triggered interrupts from the VGC.

7	6	5	4	3	2	1	0
--	--	--	--	--	--	--	RSTR

- **Bit 0 (RSTR)**: Raster IRQ enable. When set, the VGC fires an IRQ when the raster beam reaches the trigger scanline. This allows frame-synchronised processing and split-screen effects.

```
POKE $A01F,1 : REM enable raster IRQ
POKE $A01F,0 : REM disable raster IRQ
```

Combine the raster IRQ with the copper for advanced effects: use the IRQ handler to swap copper lists mid-frame, creating complex multi-zone colour schemes without burning CPU cycles in a polling loop.

5.5 Help System Registers ($A020–$A030)

The VGC includes a built-in help system accessible through a small register block.

$A020 **RegHelp** W | Default: —

Help command register. Write one of the following values:

- $01 — Display the help index (list of available topics).

- $02 — Search for a keyword. The search query must be written to the search buffer at $A021 first.

```
POKE $A020,1 : REM show help index
```

$A021–$A030 **HelpSearchBuffer** R/W | Default: 0

A 16-byte buffer for the help search query. Write a null-terminated ASCII string here, then write $02 to $A020 to execute the search.

```
10 REM search for "PLOT" in help
20 POKE $A021,ASC("P")
30 POKE $A022,ASC("L")
40 POKE $A023,ASC("O")
50 POKE $A024,ASC("T")
60 POKE $A025,0          : REM null terminator
70 POKE $A020,2          : REM search
```

5.6 Compiler Controller ($A031–$A03E)

The NovaVM includes an on-board NCC compiler. These registers provide access to compilation commands, status, and error reporting.

$A031 **CmpCmd** W | Default: —

Compiler command register. Write one of:

- $01 — Compile source code from XRAM at the address in $A03B–$A03D.
- $02 — Populate error fields for the error at index $A03A.
- $03 — Populate warning fields for the warning at index $A03A.

$A032 **CmpStatus** R | Default: 0

Compiler status:

- $00 = idle
- $01 = compiling
- $02 = compilation successful
- $03 = compilation failed with errors

$A033 **CmpErrCount** R | Default: 0

Number of errors from the last compilation.

$A034 **CmpWarnCount** R | Default: 0

Number of warnings from the last compilation.

$A035–$A036 **CmpErrLine** R | Default: 0

16-bit line number (little-endian) of the current error.

$A037–$A038 **CmpCodeSize** R | Default: 0

16-bit size (little-endian) of the compiled code in bytes.

$A039 **CmpErrMsg** R | Default: —

Read this register repeatedly to retrieve the error message one byte at a time. Returns 0 when the message is exhausted.

$A03A **CmpErrIdx** W | Default: —

Write an error or warning index here (0-based), then issue command $02 or $03 to `$A031` to populate the error fields.

$A03B–$A03D **CmpSrcAddr** W | Default: —

24-bit little-endian XRAM address of the source code to compile. Set this before issuing the compile command.

5.7 ROM Swap Register ($A03F)

$A03F **RegRomSwap** W | Default: —

Controls which ROM image is active in the $C000–$FFFF region.

- $01 — Switch to NCC ROM (native code compiler).
- $02 — Switch to EhBASIC ROM (the default).
- $03 — Activate the NCC editor.

Warning

Switching ROMs while BASIC is running will crash the system. This register is intended for use by the boot loader and compiler infrastructure, not by user programs.

5.8 Sprite Registers ($A040–$A0BF)

The 128 bytes from $A040 to $A0BF are divided into 16 blocks of 8 bytes each, one block per sprite. These registers provide direct read/write access to each sprite's position, shape slot, flags, priority, and transparent colour — bypassing the command interface entirely.

The base address of sprite N is:

$$\texttt{Base = \$A040 + } N \times 8$$

Offset	Name	R/W	Description
+0	SprRegXLo	R/W	X position low byte
+1	SprRegXHi	R/W	X position high byte
+2	SprRegYLo	R/W	Y position low byte
+3	SprRegYHi	R/W	Y position high byte
+4	SprRegShape	R/W	Shape slot index (0–255)
+5	SprRegFlags	R/W	Flags (see below)
+6	SprRegPriority	R/W	Draw priority (0–2)
+7	SprRegTransColor	R/W	Transparent colour (default 0)

SprRegFlags (offset +5)

7	6	5	4	3	2	1	0
ENA	--	--	--	--	--	YFLP	XFLP

- **Bit 7 (ENA)**: Enable. Set to show the sprite, clear to hide it.
- **Bit 1 (YFLP)**: Vertical flip.
- **Bit 0 (XFLP)**: Horizontal flip.

Bits 2–6 are unused and should be written as zero.

Sprite Register Addresses

For quick reference, here are the base addresses for all 16 sprites:

Sprite	Base	Sprite	Base
0	$A040	8	$A080
1	$A048	9	$A088
2	$A050	10	$A090
3	$A058	11	$A098
4	$A060	12	$A0A0
5	$A068	13	$A0A8
6	$A070	14	$A0B0
7	$A078	15	$A0B8

Direct Register Access Example

```
10 REM position sprite 3 at (100, 50) via direct register access
20 BASE = $A040 + 3*8
30 POKE BASE+0, 100 : POKE BASE+1, 0  : REM X=100
40 POKE BASE+2, 50  : POKE BASE+3, 0  : REM Y=50
50 POKE BASE+5, $80                   : REM enable
```

```
10 REM read sprite 5's current position
20 BASE = $A040 + 5*8
30 X = PEEK(BASE+0) + PEEK(BASE+1)*256
40 Y = PEEK(BASE+2) + PEEK(BASE+3)*256
50 PRINT "SPRITE 5 AT ";X;",";Y
```

Direct register access is faster than using the command interface for positioning sprites in a tight game loop. Updating the X and Y registers directly avoids the overhead of loading the parameter registers and issuing a command.

```
10 REM enable sprite 0 with horizontal flip
20 POKE $A045, $80 OR $01 : REM ENA + XFLP
```

```
10 REM set sprite 0 to draw behind everything
20 POKE $A046, 0 : REM priority = 0
```

The transparent colour register (offset +7) defaults to 0, meaning black pixels in the sprite shape are treated as transparent. Change this if your sprite needs to contain black pixels — for example, set it to 15 to make light grey transparent instead.

Summary

The VGC occupies addresses $A000–$A0BF and provides the complete visual output system for the NovaVM. The table below summarises the register map:

Range	Usage	Access
$A000–$A00F	Core registers (mode, colour, cursor, I/O)	R/W
$A010	Command register	W
$A011–$A01E	Parameter registers P0–P13	R/W
$A01F	IRQ control	R/W
$A020–$A030	Help system	R/W
$A031–$A03E	Compiler controller	R/W
$A03F	ROM swap	W
$A040–$A0BF	Sprite registers (16 × 8 bytes)	R/W

With 9 graphics commands, 9 sprite commands, 2 memory I/O commands, 8 copper commands, and 128 bytes of direct sprite registers, the VGC packs a remarkable amount of capability into a small address range. Master this chapter, and you have mastered the display.

Chapter 6

The Network Interface Controller

"The network is the computer. The packet is the thought."

— *Sun Microsystems proverb, paraphrased*

Introduction

The Network Interface Controller — NIC for short — provides message-oriented TCP networking through four independent connection slots. Each slot may operate as a *client* (connecting out to a remote host) or as a *server* (listening for incoming connections and accepting them).

Messages on the wire are length-prefixed: a single byte gives the payload length, where a length byte of zero denotes 256 bytes. This simple framing removes the need for delimiter parsing on the 6502 side.

Data moves between the NIC and CPU RAM via DMA. To send, the program writes the source address and length into the DMA registers and issues a Send command. To receive, the program issues a Recv command and the NIC copies the next queued message into RAM at the DMA address. Up to 16 messages may be queued per slot before older messages are dropped.

The NIC occupies 64 bytes of address space from $A100 to $A13F.

6.1 Command Register

$A100 NicCmd

W | Default: —

Writing a command byte to this register executes the command immediately on the slot selected by `$A102`. The available commands are:

$01 Connect *Connect to remote host*

Opens a TCP connection to the hostname in the name buffer (`$A120`) on the port specified by `$A108`–`$A109`. The connection is established asynchronously; poll `$A118` (slot status) for the Connected bit, or check the Error bit on failure. A 10-second timeout applies.

$02 Disconnect *Close connection*

Closes the active connection on the selected slot and drains any unread messages from the receive queue.

$03 Send *Send message via DMA*

Reads $A112 bytes from CPU RAM starting at the DMA address ($A110–$A111) and sends them as a single length-prefixed message on the selected slot's TCP connection. If the slot is not connected, the slot's Error flag is set.

$04 Recv *Receive message via DMA*

Dequeues the next message from the selected slot's receive queue and writes it into CPU RAM at the DMA address. The length of the received message is stored in $A113. If no message is available, $A113 reads as zero.

$05 Listen *Start listening for connections*

Begins listening for incoming TCP connections on the local port specified by $A10A–$A10B. The listener binds to the loopback interface. When a remote client connects, the slot's Data Ready bit is set in the slot status register, indicating that an Accept command should be issued.

$06 Accept *Accept pending connection*

Accepts a pending inbound connection on the selected slot. After acceptance, the slot transitions to Connected and incoming messages are queued automatically.

All commands operate on the slot selected by $A102. Set the slot *before* writing any command, port, hostname, or DMA parameters.

6.2 Status and Control Registers

$A101 NicStatus

R | Default: —

Global NIC status register. Provides a quick summary of the controller state.

7	6	5	4	3	2	1	0
Err	--	--	--	--	--	Any	Rdy

Bit 0 — Ready
Always set to 1 once the NIC is initialised.

Bit 1 — Any Data
Set if *any* slot has data waiting in its receive queue (or a pending accept). Useful for a single polling check across all slots.

Bit 7 — Error
Set if any slot has encountered an error.

```
10 REM wait for data on any slot
20 WAIT $A101, 2              : REM wait for bit 1 set
```

$A102 NicSlot

R/W | Default: 0

Active slot selector (0–3). All commands, port settings, hostname writes, and DMA operations act on the slot selected here.

```
POKE $A102, 2 : REM select slot 2
```

$A103 NicIrqCtrl

R/W | Default: 0

IRQ enable bitmask. Bit N (0–3) enables an IRQ when data arrives on slot N. When a message is received on an enabled slot, the NIC asserts the CPU's IRQ line.

7	6	5	4	3	2	1	0
--	--	--	--	S13	S12	S11	S10

```
POKE $A103, 1    : REM enable IRQ on slot 0 only
POKE $A103, $0F  : REM enable IRQ on all 4 slots
```

$A104 NicIrqStatus

R | Default: 0

IRQ pending flags. Bit N is set when data has arrived on slot N (and IRQ was enabled for that slot). **Reading this register clears all pending flags** and de-asserts the IRQ line.

7	6	5	4	3	2	1	0
--	--	--	--	S13	S12	S11	S10

Warning

Because reading clears the flags, read this register only once per interrupt handler and save the value for inspection.

6.3 Port Registers

$A108-$A109 NicRemotePort

R/W | Default: —

Remote port for the Connect command (16-bit, little-endian). Write the low byte to `$A108` and the high byte to `$A109`.

```
DOKE $A108, 8080 : REM port 8080
```

$A10A-$A10B NicLocalPort

R/W | Default: —

Local port for the Listen command (16-bit, little-endian). Write the low byte to `$A10A` and the high byte to `$A10B`.

```
DOKE $A10A, 9000 : REM listen on port 9000
```

6.4 DMA Registers

$A110-$A111 NicDmaAddr

R/W | Default: —

CPU RAM address for DMA transfers (16-bit, little-endian). For Send, this is the source address. For Recv, this is the destination address.

```
DOKE $A110, $4000 : REM DMA to/from $4000
```

$A112 NicDmaLen

R/W | Default: 0

DMA transfer length for the Send command. Valid values are 1–255. A value of 0 is treated as 256 (the maximum message size).

$A113 **NicMsgLen** R | Default: 0

Length of the last received message, set after a Recv command. If no message was available, this reads as 0. A received message of exactly 256 bytes is reported as 0 (matching the wire protocol convention).

6.5 Per-Slot Status Registers

$A118–$A11B **NicSlotStatus0–3** R | Default: —

Four consecutive bytes, one per slot. Each provides the current state of that connection. Slot N's status is at $A118 + N.

7	6	5	4	3	2	1	0
--	--	--	RmCl	Err	SndR	DatR	Conn

Bit 0 — Connected
Set when the slot has an active TCP connection.

Bit 1 — Data Ready
Set when one or more messages are waiting in the receive queue, or when a pending accept is available (server mode).

Bit 2 — Send Ready
Always set; indicates the slot can accept a Send command.

Bit 3 — Error
Set on connection failure, send error, or receive queue overflow.

Bit 4 — Remote Closed
Set when the remote peer has closed the connection.

```
10 REM check if slot 0 is connected
20 IF PEEK($A118) AND 1 THEN PRINT "CONNECTED"
```

Use `WAIT $A118, 1` to block until slot 0 becomes connected. The `WAIT` instruction polls the address until the specified bits are set.

6.6 Hostname Buffer

$A120–$A13F **NicNameBuf** R/W | Default: —

A 32-byte buffer for the hostname used by the Connect command. The name must be null-terminated ASCII. Write the hostname before issuing a Connect command.

```
10 REM write "localhost" into the name buffer
20 A$ = "localhost" + CHR$(0)
30 FOR I=1 TO LEN(A$)
```

```
40   POKE $A120+I-1, ASC(MID$(A$,I,1))
50 NEXT I
```

6.7 Complete Examples

6.7.1 Client: Connect, Send, and Receive

This example connects to a server on localhost port 8080, sends a "HELLO" message, and waits for a response.

```
100 REM === TCP client example ===
110 POKE $A102, 0                : REM select slot 0
120 DOKE $A108, 8080             : REM remote port 8080
130 REM write hostname
140 A$ = "localhost" + CHR$(0)
150 FOR I=1 TO LEN(A$)
160   POKE $A120+I-1, ASC(MID$(A$,I,1))
170 NEXT I
180 POKE $A100, 1                : REM connect
190 WAIT $A118, 1                : REM wait for connected
200 PRINT "CONNECTED!"
210 REM
220 REM send "HELLO"
230 A$ = "HELLO"
240 FOR I=1 TO LEN(A$)
250   POKE $3FFF+I, ASC(MID$(A$,I,1))
260 NEXT I
270 DOKE $A110, $4000            : REM DMA source address
280 POKE $A112, LEN(A$)          : REM DMA length
290 POKE $A100, 3                : REM send
300 PRINT "SENT: "; A$
310 REM
320 REM receive response
330 DOKE $A110, $4100            : REM DMA dest address
340 WAIT $A118, 2                : REM wait for data ready
350 POKE $A100, 4                : REM recv
360 L = PEEK($A113)              : REM message length
370 R$ = ""
380 FOR I=0 TO L-1
390   R$ = R$ + CHR$(PEEK($4100+I))
400 NEXT I
410 PRINT "RECEIVED: "; R$
420 POKE $A100, 2                : REM disconnect
```

6.7.2 Server: Listen, Accept, and Echo

This example listens on port 9000, accepts one connection, and echoes back every message it receives.

```
100 REM === TCP echo server ===
110 POKE $A102, 0                : REM select slot 0
120 DOKE $A10A, 9000             : REM local port 9000
130 POKE $A100, 5                : REM listen
140 PRINT "LISTENING ON PORT 9000..."
150 WAIT $A118, 2                : REM wait for pending accept
160 POKE $A100, 6                : REM accept
170 PRINT "CLIENT CONNECTED!"
```

```
180 REM
190 REM echo loop
200 S = PEEK($A118)
210 IF S AND 16 THEN GOTO 310  : REM remote closed?
220 IF NOT(S AND 2) THEN 200   : REM no data? loop
230 DOKE $A110, $4000          : REM DMA address
240 POKE $A100, 4              : REM recv
250 L = PEEK($A113)
260 IF L=0 THEN 200            : REM empty message
270 POKE $A112, L              : REM set send length
280 POKE $A100, 3              : REM echo it back
290 PRINT "ECHOED "; L; " BYTES"
300 GOTO 200
310 PRINT "CLIENT DISCONNECTED"
320 POKE $A100, 2              : REM disconnect
```

The BASIC commands `NOPEN`, `NCLOSE`, `NLISTEN`, `NACCEPT`, and `NSEND` provide higher-level access to the NIC without direct register manipulation. See the NovaVM BASIC User Guide for details.

Chapter 7

The Wavetable Synthesizer

"Any sufficiently advanced synthesizer is indistinguishable from an orchestra."

— *With apologies to Arthur C. Clarke*

Introduction

The Wavetable Synthesizer — WTS — is an 8-voice sample-based sound engine that complements the SID chips' analog-style synthesis with realistic instrument timbres. It loads standard SF2 soundfont files from the host filesystem, keeping sample data in host memory, and renders stereo PCM16 audio at 44 100 Hz through OpenAL.

Each voice provides independent control over note pitch, velocity, instrument selection, volume, stereo panning, and pitch bend. Two global effects — reverb (Freeverb algorithm) and chorus (modulated delay) — process the mixed output before the master volume stage.

The WTS occupies the address range $A140–$A1DF, divided into three regions: 8 voice register blocks ($A140–$A17F), global control registers ($A180–$A187), and the instrument enumeration interface ($A1A0–$A1DF).

7.1 Voice Registers ($A140–$A17F)

Eight voices, eight bytes each. The base address for voice N is $A140 + $N \times 8$.

Voice	Base	End	Range
0	`$A140`	`$A147`	Voice 0
1	`$A148`	`$A14F`	Voice 1
2	`$A150`	`$A157`	Voice 2
3	`$A158`	`$A15F`	Voice 3
4	`$A160`	`$A167`	Voice 4
5	`$A168`	`$A16F`	Voice 5
6	`$A170`	`$A177`	Voice 6
7	`$A178`	`$A17F`	Voice 7

The register offsets within each voice block are identical. In the entries below, the address shown is for voice 0; add $N \times 8$ for voice N.

`$A140` WtsVoiceNote W | Default: —

MIDI note number. Writing a value from 1 to 127 triggers a note-on at that pitch (60 = middle C). Writing 0 triggers a note-off, releasing the voice into its envelope's release phase.

Set the velocity and instrument *before* writing the note number, as the write to this register is what triggers playback.

```
POKE $A140, 60 : REM note on: middle C
POKE $A140, 0  : REM note off
```

$A141 WtsVoiceVelocity

W | Default: —

Note velocity (0–127). Affects the volume and, depending on the soundfont, the timbre of the note. Set this before writing the note number.

```
POKE $A141, 100 : REM velocity 100 (moderately loud)
```

$A142 WtsVoiceInstrument

W | Default: —

Instrument index (0–255). Selects from the currently loaded soundfont. The number of available instruments can be read from $A184. Set this before playing notes.

```
POKE $A142, 0   : REM instrument 0 (typically piano)
POKE $A142, 40  : REM instrument 40 (typically violin)
```

$A143 WtsVoiceVolume

R/W | Default: 255

Per-voice volume (0–255). Independent of velocity — this acts as a channel-level volume control. The final output level is the product of velocity, voice volume, and master volume.

```
POKE $A143, 128 : REM half volume on voice 0
```

$A144 WtsVoicePanning

R/W | Default: 128

Stereo position. 0 = hard left, 128 = centre, 255 = hard right.

```
POKE $A144, 0   : REM voice 0 hard left
POKE $A14C, 255 : REM voice 1 hard right
```

$A145 WtsVoicePitchBendLo

W | Default: 0

Pitch bend low byte. Combined with the high byte ($A146) to form a 16-bit pitch bend value.

$A146 WtsVoicePitchBendHi

W | Default: $80

Pitch bend high byte. The centre (no bend) value is $80. Values above $80 bend the pitch upward; values below bend downward. The bend range depends on the soundfont preset but is typically ±2 semitones.

```
10 REM bend voice 0 up one step
20 POKE $A145, 0    : REM low byte
30 POKE $A146, $C0  : REM high byte (above centre)
```

$A147 WtsVoiceStatus

R | Default: 0

Voice status flags.

7	6	5	4	3	2	1	0
--	--	--	--	--	--	Rel	Act

Bit 0 — Active
Set while the voice is producing sound (note on or sustaining).

Bit 1 — Releasing
Set when the voice is in the release phase of its envelope (after note off). Cleared when the release completes and the voice becomes silent.

```
10 REM wait for voice 0 to finish releasing
20 WAIT $A147, 3, 3  : REM wait until bits 0+1 both clear
```

7.2 Global Registers ($A180–$A187)

$A180 WtsReverbLevel

R/W | Default: 80

Reverb wet level. 0 = completely dry (no reverb), 255 = maximum reverb. The reverb uses a Freeverb algorithm with multiple comb and allpass filters.

```
POKE $A180, 120 : REM generous reverb
POKE $A180, 0   : REM reverb off
```

$A181 WtsChorusLevel

R/W | Default: 40

Chorus depth. 0 = no chorus effect, 255 = maximum depth. The chorus uses a modulated delay line to create a thickening, ensemble-like effect.

```
POKE $A181, 80 : REM moderate chorus
```

$A182 WtsMasterVolume

R/W | Default: 255

Master output volume for all WTS voices. Scales the final mixed output before it is sent to the audio device. 0 = silent, 255 = full volume.

```
POKE $A182, 200 : REM slightly reduced master volume
```

$A183 WtsSoundfontStatus

R | Default: 0

Soundfont loading status:

Value	Meaning
$00	No soundfont loaded
$01	Ready — soundfont loaded and playable
$02	Loading — soundfont is being loaded (wait before playing)
$FF	Error — soundfont failed to load

```
10 REM wait for soundfont to finish loading
20 WAIT $A183, 1   : REM wait for bit 0 (ready)
30 PRINT "SOUNDFONT READY"
```

$A184 WtsInstrumentCount

R | Default: 0

Number of instruments available in the currently loaded soundfont. Returns 0 if no soundfont is loaded.

```
PRINT "INSTRUMENTS:"; PEEK($A184)
```

$A185 **WtsCommand** W | Default: —

Command register. Writing a command byte executes it immediately.

$01 **AllNotesOff** *Silence all voices*

Immediately stops all playing notes on all eight voices.

$02 **ResetEffects** *Reset reverb and chorus*

Resets reverb and chorus levels to their defaults (80 and 40 respectively) and clears any residual audio in the effect buffers.

$A186 **WtsCommandArg** W | Default: —

Command argument byte. Currently unused; reserved for future commands.

$A187 **WtsActiveVoices** R | Default: 0

Bitmask of currently active voices. Bit N is set if voice N is producing sound (either playing or releasing).

7	6	5	4	3	2	1	0
V7	V6	V5	V4	V3	V2	V1	V0

```
10 REM check which voices are active
20 A = PEEK($A187)
30 FOR I=0 TO 7
40   IF A AND (2^I) THEN PRINT "VOICE"; I; "ACTIVE"
50 NEXT I
```

7.3 Instrument Enumeration ($A1A0–$A1DF)

The enumeration interface allows programs to discover which instruments are available in the loaded soundfont. Write an instrument index to `$A1A0` and the name, bank, and program number are populated for reading.

$A1A0 **WtsEnumIndex** W | Default: —

Write an instrument index (0 to `PEEK($A184)-1`) to populate the enumeration buffer.

$A1A1 **WtsEnumBank** R | Default: —

MIDI bank number of the selected instrument.

$A1A2 **WtsEnumProgram** R | Default: —

MIDI program number of the selected instrument.

$A1A3-$A1DF **WtsEnumName** R | Default: —

Null-terminated instrument name string, up to 28 characters.

```
10 REM list all instruments
20 N = PEEK($A184)
30 IF N=0 THEN PRINT "NO SOUNDFONT" : END
```

```
40 FOR I=0 TO N-1
50   POKE $A1A0, I          : REM select instrument
60   B = PEEK($A1A1)        : REM bank
70   P = PEEK($A1A2)        : REM program
80   N$ = ""
90   FOR J=0 TO 27
100    C = PEEK($A1A3+J)
110    IF C=0 THEN J=27 : GOTO 130
120    N$ = N$ + CHR$(C)
130  NEXT J
140  PRINT I; ": ["; B; ":"; P; "] "; N$
150 NEXT I
```

7.4 Complete Examples

7.4.1 Loading a Soundfont

Soundfonts are loaded via the FileIoController at `$B9A0`. Command $15 loads an SF2 file by name.

```
10 REM load a soundfont via FileIoController
20 F$ = "GeneralUser.sf2" + CHR$(0)
30 FOR I=1 TO LEN(F$)
40   POKE $B9BF+I, ASC(MID$(F$,I,1))
50 NEXT I
60 POKE $B9A0, $15            : REM load soundfont command
70 WAIT $A183, 1              : REM wait for ready
80 PRINT PEEK($A184); " INSTRUMENTS LOADED"
```

7.4.2 Playing a Note

```
10 REM play middle C on voice 0
20 BASE = $A140
30 POKE BASE+2, 0             : REM instrument 0 (piano)
40 POKE BASE+1, 100           : REM velocity 100
50 POKE BASE+0, 60            : REM note on: middle C
60 FOR I=1 TO 5000 : NEXT I   : REM wait
70 POKE BASE+0, 0             : REM note off
```

7.4.3 Three-Note Chord with Panning

```
100 REM play a C major chord across the stereo field
110 FOR V=0 TO 2
120   B = $A140 + V*8
130   POKE B+2, 0             : REM piano
140   POKE B+3, 200           : REM volume
150   POKE B+1, 90            : REM velocity
160 NEXT V
170 POKE $A144, 64            : REM voice 0: left of centre
180 POKE $A14C, 128           : REM voice 1: centre
190 POKE $A154, 192           : REM voice 2: right of centre
200 REM play C-E-G
210 POKE $A140, 60            : REM C
220 POKE $A148, 64            : REM E
230 POKE $A150, 67            : REM G
```

```
240 FOR I=1 TO 10000 : NEXT I
250 REM release all
260 POKE $A140, 0
270 POKE $A148, 0
280 POKE $A150, 0
```

7.4.4 Setting Up Reverb and Chorus

```
10 REM cathedral-style reverb with gentle chorus
20 POKE $A180, 180            : REM heavy reverb
30 POKE $A181, 60             : REM moderate chorus
40 POKE $A182, 220            : REM master volume
50 REM now any notes played will have these effects
```

The MML `MUSIC` command supports WTS voices with the `@W` prefix. For example, `MUSIC 6, "@W0 L4 CDEFG"` plays a C major scale on WTS voice 0 using instrument 0. This is often easier than direct register manipulation for musical sequences.

Chapter 8

Screen Memory

"The screen is a window into the machine's mind. What you write there, the world will see."

— *An anonymous systems programmer*

Introduction

The NovaVM's 80×25 text display is backed by two contiguous 2000-byte RAM blocks: **Character RAM** holds the ASCII code for each cell, and **Color RAM** holds the foreground colour. Together they occupy $AA00–$B99F — 4000 bytes that map one-to-one to the 2000 visible character positions on screen.

These addresses are ordinary read/write memory. Any store to Character or Color RAM takes effect immediately; there is no need to wait for VSYNC or issue a VGC command. This makes direct screen writes the fastest way to update small regions of the display, ideal for scoreboards, status bars, and text animations.

8.1 Character RAM ($AA00–$B1CF)

$AA00-$B1CF Character RAM

R/W | Default: $20

2000 bytes arranged as 25 rows of 80 columns. Each byte is a standard ASCII character code; the VGC renders it using the built-in 8×16 font. The default fill value is $20 (space), giving a blank screen after reset or `CLS`.

The address of any cell is calculated as:

```
addr = $AA00 + row * 80 + column
```

where `row` ranges from 0 (top) to 24 (bottom) and `column` from 0 (left) to 79 (right).

Writing a byte to any address in this range instantly updates the corresponding character on screen. Reading returns the character code currently displayed at that position.

```
10 REM place an asterisk at column 40, row 12 (centre of screen)
20 POKE $AA00 + 12*80 + 40, ASC("*")
```

8.2 Color RAM ($B1D0–$B99F)

$B1D0–$B99F **Color RAM** R/W | Default: 1

2000 bytes, one-to-one with Character RAM. Each byte holds the foreground colour (0–15) for the corresponding character cell. The default value is 1 (white on the standard palette). The address formula mirrors Character RAM:

```
addr = $B1D0 + row * 80 + column
```

Writing a colour value here changes the foreground colour of the corresponding character immediately. The background colour for all cells is set globally via the VGC's `$A001` register (RegBgCol).

```
10 REM set the character at (0,0) to red (colour 2)
20 POKE $B1D0, 2
```

The VGC's 16-colour palette is documented in Appendix D. Colour index 0 is black and index 1 is white by default, but the palette can be customised via the VGC's colour registers.

8.3 Screen Coordinate Math

Each row occupies exactly 80 bytes ($50 in hex). The following table lists the starting address of every row in both Character and Color RAM.

Row	Char Addr	Color Addr
0	$AA00	$B1D0
1	$AA50	$B220
2	$AAA0	$B270
3	$AAF0	$B2C0
4	$AB40	$B310
5	$AB90	$B360
6	$ABE0	$B3B0
7	$AC30	$B400
8	$AC80	$B450
9	$ACD0	$B4A0
10	$AD20	$B4F0
11	$AD70	$B540
12	$ADC0	$B590
13	$AE10	$B5E0
14	$AE60	$B630
15	$AEB0	$B680
16	$AF00	$B6D0
17	$AF50	$B720
18	$AFA0	$B770
19	$AFF0	$B7C0
20	$B040	$B810
21	$B090	$B860
22	$B0E0	$B8B0
23	$B130	$B900

Row	Char Addr	Color Addr
24	$B180	$B950

8.3.1 Worked Example

To place the letter `A` (ASCII $41) in colour 14 (light blue) at column 35, row 10:

1. Character address: $AA00 + 10 × 80 + 35 = $AA00 + 835 = $AA00 + $0343 = `$AD43`
2. Color address: $B1D0 + 10 × 80 + 35 = $B1D0 + $0343 = `$B513`

```
POKE $AD43, $41 : POKE $B513, 14
```

For speed-critical routines, pre-compute row base addresses in an array rather than multiplying by 80 on every access. See the "Fast Text Rendering" example below.

8.4 BASIC Examples

8.4.1 Direct Screen Write

Write a string with a chosen colour without using `PRINT`:

```
10 REM write "HELLO" at row 0, col 0 in red
20 H$ = "HELLO"
30 FOR I=0 TO LEN(H$)-1
40   POKE $AA00+I, ASC(MID$(H$,I+1,1))
50   POKE $B1D0+I, 2 : REM colour 2 = red
60 NEXT I
```

8.4.2 Screen Dump

Read the current contents of the screen and print them to a string:

```
10 REM read row 0 of the screen
20 R$ = ""
30 FOR I=0 TO 79
40   C = PEEK($AA00+I)
50   IF C<32 THEN C=32
60   R$ = R$ + CHR$(C)
70 NEXT I
80 PRINT R$
```

8.4.3 Fast Text Rendering

For games and animations, direct POKEs outperform `PRINT` because they avoid cursor movement, scrolling, and control-character interpretation. This example pre-computes row addresses for speed:

```
100 REM pre-compute row base addresses
110 DIM CA(24), CO(24)
120 FOR R=0 TO 24
130   CA(R) = $AA00 + R*80
140   CO(R) = $B1D0 + R*80
150 NEXT R
200 REM draw a status bar on row 24
210 S$ = " SCORE: 00000   LIVES: 3   LEVEL: 01 "
220 FOR I=0 TO LEN(S$)-1
230   POKE CA(24)+I, ASC(MID$(S$,I+1,1))
240   POKE CO(24)+I, 14 : REM light blue
250 NEXT I
```

8.4.4 Colour Cycling Effect

Cycle all characters on screen through the 16 colours:

```
10 REM colour cycling effect
20 C = 0
30 FOR I=0 TO 1999
40   POKE $B1D0+I, C
50 NEXT I
60 C = (C+1) AND 15
70 GOTO 30
```

The VGC also provides indirect access to screen and color RAM via the MemRead ($19) and MemWrite ($1A) commands (see $A010). Direct POKE is faster for small updates; DMA transfers are faster for bulk operations.

Warning

Writing outside the 2000-byte range of each block will corrupt adjacent hardware registers. Character RAM ends at $B1CF — bytes written to $B1D0 and above will overwrite Color RAM. Color RAM ends at $B99F — bytes written to $B9A0 and above will hit the File I/O Controller registers. Always bounds-check your row and column values.

Chapter 9

The File I/O Controller

"A computer without persistent storage is just a very expensive calculator."

— *Unknown, circa 1975*

Introduction

The File I/O Controller — FIO — is a coprocessor that bridges the 6502's address space to the host filesystem. It handles file save/load, directory enumeration, SID file playback, music sequencing via MML, MIDI file playback, soundfont loading, and virtual disk operations.

Commands are issued by writing to the command register at $B9A0. Parameters are set in surrounding registers *before* the command write. After execution, the status register at $B9A1 reports success or failure, and the error code register at $B9A2 provides detail.

Files are stored in the save directory (`~/e6502-programs` by default). The FIO automatically appends the appropriate file extension (`.bas`, `.sid`, `.gfx`, etc.) based on the command.

9.1 Register Map

Address	Name	R/W	Default	Description
$B9A0	FioCmd	W	—	Command register (write triggers execution)
$B9A1	FioStatus	R	$00	Status code
$B9A2	FioErrCode	R	$00	Error code
$B9A3	FioNameLen	R/W	0	Filename length (1–63)
$B9A4	FioSrcL	R/W	0	Source/destination RAM address (low byte)
$B9A5	FioSrcH	R/W	0	Source/destination RAM address (high byte)
$B9A6	FioEndL	R/W	0	End address for SAVE (low byte)
$B9A7	FioEndH	R/W	0	End address for SAVE (high byte)
$B9A8	FioSizeL	R	0	Loaded data size (low byte)
$B9A9	FioSizeH	R	0	Loaded data size (high byte)
$B9AA	FioGSpace	R/W	0	Graphics space selector
$B9AB	FioGAddrL	R/W	0	Graphics offset (low byte)
$B9AC	FioGAddrH	R/W	0	Graphics offset (high byte)
$B9AD	FioGLenL	R/W	0	Graphics length (low byte)
$B9AE	FioGLenH	R/W	0	Graphics length (high byte)
$B9AF	FioDirType	R	0	Directory entry file type
$B9B0–$B9EF	FioName	R/W	0	Filename buffer (64 bytes ASCII)

9.2 Core Registers

$B9A0 FioCmd

W | Default: —

Command register. Writing a command byte here immediately executes the corresponding operation. All parameters (filename, addresses, lengths) must be set in the other registers *before* writing to FioCmd.

$B9A1 FioStatus

R | Default: $00

Status of the last command:

Value	Meaning
$00	Idle — no command has been issued
$02	OK — last command completed successfully
$03	Error — last command failed (see FioErrCode)

$B9A2 FioErrCode

R | Default: $00

Detailed error code when FioStatus is $03:

Value	Meaning
0	No error
1	File not found
2	I/O error
3	End of directory
4	Disk full
5	Device not mounted

$B9A3 FioNameLen

R/W | Default: 0

Length of the filename stored in the FioName buffer. Valid range: 1–63. Some commands use this value; others read the buffer directly until they encounter a null byte or reach 64 characters.

$B9A4 FioSrcL

R/W | Default: 0

Low byte of the 16-bit source or destination address in CPU RAM. For **Save**, this is the start of the region to save. For **Load**, this is where loaded data will be written. For music commands, the low byte may carry a parameter value (e.g., instrument index, volume level).

$B9A5 FioSrcH

R/W | Default: 0

High byte of the source/destination address.

$B9A6 FioEndL

R/W | Default: 0

Low byte of the end address. For **Save**, the region FioSrc–FioEnd (inclusive) is written to disk. For **MusicSeq**, FioEnd minus FioSrc gives the length of the MML string in RAM.

$B9A7 FioEndH

R/W | Default: 0

High byte of the end address.

$B9A8 **FioSizeL** R | Default: 0

Low byte of the loaded data size. Set by the host after a successful **Load** command.

$B9A9 **FioSizeH** R | Default: 0

High byte of the loaded data size.

$B9AA **FioGSpace** R/W | Default: 0

Graphics space selector for **GSave**/**GLoad** commands:

Value	Space
0	Screen (Character RAM)
1	Color (Color RAM)
2	Graphics (bitmap pixel data)
3	Sprite (sprite definition data)

$B9AB **FioGAddrL** R/W | Default: 0

Low byte of the offset within the selected graphics space.

$B9AC **FioGAddrH** R/W | Default: 0

High byte of the graphics offset.

$B9AD **FioGLenL** R/W | Default: 0

Low byte of the graphics data length.

$B9AE **FioGLenH** R/W | Default: 0

High byte of the graphics data length.

$B9AF **FioDirType** R | Default: 0

File type of the current directory entry, populated by **DirRead**:

Value	Type
0	`.bas` — BASIC program
1	`.sid` — SID music file
2	`.bin` — Binary executable
3	`.mid` — MIDI file
5	Directory

$B9B0-$B9EF **FioName** R/W | Default: 0

64-byte filename buffer. Store the filename as ASCII characters starting at $B9B0 before issuing commands that require a filename. Null-terminate the string or set FioNameLen to the correct length.

```
10 REM store filename "MYFILE" into FioName buffer
20 F$ = "MYFILE" + CHR$(0)
30 FOR I=1 TO LEN(F$)
40   POKE $B9AF+I, ASC(MID$(F$,I,1))
50 NEXT I
```

9.3 Command Reference

Commands are executed by writing the command byte to `$B9A0`. Set all required parameters *before* writing the command byte.

9.3.1 File Operations

$01 Save *Save BASIC program to disk*

Saves the region of CPU RAM from FioSrc to FioEnd (inclusive) as a `.bas` file. The filename is read from the FioName buffer.

Parameters:

- FioSrc ($B9A4–$B9A5): start address of data in RAM
- FioEnd ($B9A6–$B9A7): end address of data in RAM
- FioName ($B9B0+): filename (without extension)

Result: FioStatus = $02 on success, $03 on error.

$02 Load *Load BASIC program from disk*

Loads a `.bas` file into CPU RAM starting at FioSrc. After a successful load, FioSize ($B9A8–$B9A9) contains the number of bytes loaded.

Parameters:

- FioSrc ($B9A4–$B9A5): destination address in RAM
- FioName ($B9B0+): filename (without extension)

Result: FioStatus = $02, FioSize updated; or $03 with ErrCode = 1 (not found) or 2 (I/O error).

$03 DirOpen *Open directory for enumeration*

Opens the current save directory for sequential reading with DirRead. Resets the internal directory index to the first entry.

Parameters: None.

Result: FioStatus = $02.

$04 DirRead *Read next directory entry*

Reads the next file in the directory. The filename is placed in the FioName buffer and the file type in FioDirType ($B9AF). When all entries have been read, FioStatus is set to $03 with ErrCode = 3 (end of directory).

Parameters: None (directory must be opened with DirOpen first).

Result: FioName and FioDirType populated, or ErrCode = 3 at end.

$05 Delete *Delete a file*

Deletes the file named in the FioName buffer.

Parameters:

- FioName ($B9B0+): filename (without extension)

Result: FioStatus = $02 on success, $03 with ErrCode = 1 if not found.

$06 GSave *Save VGC memory to disk*

Saves a region of VGC memory to a `.gfx` file. The space selector determines which VGC buffer is read.

Parameters:

- FioGSpace ($B9AA): graphics space (0–3)
- FioGAddr ($B9AB–$B9AC): offset within the space
- FioGLen ($B9AD–$B9AE): number of bytes to save
- FioName ($B9B0+): filename

Result: FioStatus = $02 on success.

$07 GLoad *Load VGC memory from disk*

Loads a `.gfx` file into the selected VGC memory space.

Parameters:

- FioGSpace ($B9AA): target graphics space (0–3)
- FioGAddr ($B9AB–$B9AC): offset within the space
- FioGLen ($B9AD–$B9AE): number of bytes to load
- FioName ($B9B0+): filename

Result: FioStatus = $02 on success.

9.3.2 SID Playback

$08 SidPlay *Play a .sid file*

Loads and plays a `.sid` file through the SID chip emulation. The SID player injects an IRQ trampoline into CPU RAM that calls the file's init and play routines at 60 Hz.

Parameters:

- FioName ($B9B0+): filename (without `.sid` extension)

Result: FioStatus = $02 if playback started.

$09 SidStop *Stop SID playback*

Stops the currently playing SID file and removes the IRQ trampoline.

Parameters: None.

Result: FioStatus = $02.

9.3.3 Music Engine

The Music Engine provides a 14-voice sequencer (6 SID + 8 WTS) driven by MML (Music Macro Language) strings. These commands control instrument selection, sound effects, volume, and MML sequence playback.

$0A Instrument *Select SID instrument*

Selects a SID instrument preset for subsequent sound and music commands.

Parameters:

- FioSrcL ($B9A4): instrument index (0–15)

Result: FioStatus = $02.

`$0B` Sound *Trigger a sound effect*

Triggers a one-shot sound effect on the SID, using the currently selected instrument. The effect uses voice stealing if all voices are busy.

Parameters:

- FioSrcL ($B9A4): MIDI note number (0–127)

Result: FioStatus = $02.

`$0C` Volume *Set music volume*

Sets the master volume for the music engine.

Parameters:

- FioSrcL ($B9A4): volume level (0–255)

Result: FioStatus = $02.

`$0D` MusicSeq *Load MML music sequence*

Loads an MML string from CPU RAM into the music engine for playback. The MML string is read from the address range FioSrc to FioEnd.

Parameters:

- FioSrc ($B9A4–$B9A5): start address of MML string in RAM
- FioEnd ($B9A6–$B9A7): end address of MML string in RAM

Result: FioStatus = $02 if the MML was parsed successfully.

> In practice, the BASIC `MUSIC` statement is the easiest way to load and play MML sequences. Direct FIO access is useful when MML data is generated programmatically or stored in expansion memory.

`$0E` MusicPlay *Start music playback*

Begins playing the MML sequence loaded by MusicSeq.

Parameters: None.

Result: FioStatus = $02.

`$0F` MusicStop *Stop music playback*

Stops the currently playing music sequence and silences all music voices.

Parameters: None.

Result: FioStatus = $02.

`$10` MusicTempo *Set music tempo*

Sets the tempo in beats per minute.

Parameters:

- FioSrc ($B9A4–$B9A5): tempo in BPM (16-bit value)

Result: FioStatus = $02.

`$11` **MusicLoop** *Set loop mode*

Controls whether the music sequence loops or plays once.

Parameters:

- FioSrcL ($B9A4): 0 = play once, 1 = loop

Result: FioStatus = $02.

`$12` **MusicPri** *Set music priority*

Sets the priority level for the music engine, affecting voice-stealing behaviour when sound effects compete with background music.

Parameters:

- FioSrcL ($B9A4): priority level

Result: FioStatus = $02.

9.3.4 MIDI — Playback

`$13` **MidPlay** *Play a MIDI file*

Loads and plays a standard `.mid` file through the Music Engine's 14-voice system. Three routing modes are available: Auto (WTS preferred with SID overflow), Manual (per-channel table), and SID-only (legacy 6-voice). GM program numbers are automatically mapped to WTS instrument indices or SID instrument buckets.

Parameters:

- FioName ($B9B0+): filename (without `.mid` extension)

Result: FioStatus = $02 if playback started.

`$14` **MidStop** *Stop MIDI playback*

Stops the currently playing MIDI file and releases all voices.

Parameters: None.

Result: FioStatus = $02.

9.3.5 Soundfont

`$15` **SfLoad** *Load SF2 soundfont for WTS*

Loads an SF2 soundfont file from the host filesystem into the Wavetable Synthesizer. Sample data is kept in host memory. The WTS status register at `$A183` can be polled to detect when loading is complete.

Parameters:

- FioName ($B9B0+): soundfont filename (with `.sf2` extension)

Result: FioStatus = $02 if loading was initiated. Poll `PEEK($A183)` for ready status.

9.3.6 Disk Operations

These commands operate on the virtual storage device system. They require a mounted storage device; issuing them without one produces ErrCode = 5 (not mounted).

$20 Cd *Change directory*

Changes the current working directory on the active storage device.

Parameters:

- FioName ($B9B0+): directory name or path

Result: FioStatus = $02 on success.

$21 Mkdir *Create directory*

Creates a new directory.

Parameters:

- FioName ($B9B0+): directory name

Result: FioStatus = $02 on success.

$22 Rmdir *Remove directory*

Removes an empty directory.

Parameters:

- FioName ($B9B0+): directory name

Result: FioStatus = $02 on success.

$23 Format *Format disk*

Formats the active storage device, erasing all data.

Parameters: None.

Result: FioStatus = $02 on success.

> **Warning**
>
> Format erases all data on the device. There is no confirmation prompt and no undo.

$24 Mount *Mount disk*

Mounts a storage device, making it available for file operations.

Parameters:

- FioName ($B9B0+): device identifier

Result: FioStatus = $02 on success.

$25 Unmount *Unmount disk*

Unmounts the active storage device.

Parameters: None.

Result: FioStatus = $02 on success.

$26 Pwd *Print working directory*

Writes the current working directory path into the FioName buffer.

Parameters: None.

Result: FioName populated with the current path.

9.4 File Metadata Buffer ($BAB0–$BB1F)

The 112-byte metadata buffer at $BAB0–$BB1F is populated by filtered directory reads. It provides rich metadata for SID files, MIDI files, and binary executables — information that is not available from the filename alone.

$BAB0-$BB1F **File Metadata Buffer** R | Default: 0

Address	Name	Size	Description
$BAB0	MetaType	1	File type (same encoding as FioDirType)
$BAB1–$BAB2	MetaSize	2	File size in bytes (16-bit LE)
$BAB3–$BAD2	MetaTitle	32	Title (null-padded ASCII)
$BAD3–$BAF2	MetaAuthor	32	Author name (null-padded ASCII)
$BAF3–$BB12	MetaCopyright	32	Copyright notice (null-padded ASCII)
$BB13–$BB14	MetaLoadAddr	2	Load address (BIN/SID, 16-bit LE)
$BB15–$BB16	MetaInitAddr	2	Init address (SID only, 16-bit LE)
$BB17–$BB18	MetaPlayAddr	2	Play address (SID only, 16-bit LE)
$BB19	MetaSongs	1	Song count (SID subtunes / MIDI tracks)
$BB1A–$BB1B	MetaDuration	2	Duration in seconds (MIDI only, 16-bit LE)
$BB1C	MetaGfxSpace	1	GFX space type

Read the metadata buffer immediately after a DirRead command. The buffer contents are overwritten by each subsequent DirRead.

9.5 Status and Error Code Summary

9.5.1 Status Codes (FioStatus, $B9A1)

Value	Name	Description
$00	Idle	No command has been issued, or the controller has been reset
$02	OK	The last command completed successfully
$03	Error	The last command failed; check FioErrCode for the reason

9.5.2 Error Codes (FioErrCode, $B9A2)

Value	Name	Description
0	None	No error
1	Not Found	The specified file or directory does not exist
2	I/O Error	A read or write operation failed at the host level
3	End of Dir	All directory entries have been enumerated
4	Disk Full	Insufficient space on the storage device
5	Not Mounted	The target storage device is not mounted

9.6 BASIC Examples

9.6.1 Saving and Loading a Program

```
10 REM === SAVE a BASIC program ===
20 REM set filename "MYPROG"
30 F$ = "MYPROG" + CHR$(0)
40 FOR I=1 TO LEN(F$)
50   POKE $B9AF+I, ASC(MID$(F$,I,1))
60 NEXT I
70 REM set source range ($0300-$2000)
80 DOKE $B9A4, $0300 : REM FioSrc
90 DOKE $B9A6, $2000 : REM FioEnd
100 POKE $B9A0, $01  : REM execute Save
110 IF PEEK($B9A1)=$02 THEN PRINT "SAVED OK"
120 IF PEEK($B9A1)=$03 THEN PRINT "ERROR:"; PEEK($B9A2)
```

```
10 REM === LOAD a BASIC program ===
20 F$ = "MYPROG" + CHR$(0)
30 FOR I=1 TO LEN(F$)
40   POKE $B9AF+I, ASC(MID$(F$,I,1))
50 NEXT I
60 DOKE $B9A4, $0300 : REM FioSrc (load destination)
70 POKE $B9A0, $02   : REM execute Load
80 IF PEEK($B9A1)=$02 THEN PRINT "LOADED"; DEEK($B9A8); "BYTES"
90 IF PEEK($B9A1)=$03 THEN PRINT "ERROR:"; PEEK($B9A2)
```

9.6.2 Directory Listing

```
10 REM list all files in the save directory
20 POKE $B9A0, $03 : REM DirOpen
30 REM loop through entries
40 POKE $B9A0, $04 : REM DirRead
50 IF PEEK($B9A1)=$03 THEN PRINT "---END---" : END
60 REM read filename from buffer
70 N$ = ""
80 FOR I=0 TO 63
90   C = PEEK($B9B0+I)
100  IF C=0 THEN I=63 : GOTO 120
110  N$ = N$ + CHR$(C)
120 NEXT I
130 T = PEEK($B9AF) : REM file type
140 T$ = "???"
150 IF T=0 THEN T$ = "BAS"
160 IF T=1 THEN T$ = "SID"
170 IF T=2 THEN T$ = "BIN"
180 IF T=3 THEN T$ = "MID"
190 IF T=5 THEN T$ = "DIR"
200 PRINT T$; " "; N$
210 GOTO 40
```

9.6.3 Playing a SID File

```
10 REM play a SID file named "COMMANDO"
20 F$ = "COMMANDO" + CHR$(0)
30 FOR I=1 TO LEN(F$)
40   POKE $B9AF+I, ASC(MID$(F$,I,1))
50 NEXT I
60 POKE $B9A0, $08 : REM SidPlay
```

```
70 IF PEEK($B9A1)=$02 THEN PRINT "PLAYING..."
80 REM press any key to stop
90 GET K$ : IF K$="" THEN 90
100 POKE $B9A0, $09 : REM SidStop
110 PRINT "STOPPED"
```

9.6.4 Loading and Playing a MIDI File

```
10 REM load soundfont first
20 F$ = "GeneralUser.sf2" + CHR$(0)
30 FOR I=1 TO LEN(F$)
40   POKE $B9AF+I, ASC(MID$(F$,I,1))
50 NEXT I
60 POKE $B9A0, $15   : REM SfLoad
70 WAIT $A183, 1     : REM wait for WTS ready
80 PRINT PEEK($A184); " INSTRUMENTS LOADED"
90 REM now play a MIDI file
100 F$ = "BACH_FUGUE" + CHR$(0)
110 FOR I=1 TO LEN(F$)
120   POKE $B9AF+I, ASC(MID$(F$,I,1))
130 NEXT I
140 POKE $B9A0, $13  : REM MidPlay
150 IF PEEK($B9A1)=$02 THEN PRINT "MIDI PLAYING"
160 REM press any key to stop
170 GET K$ : IF K$="" THEN 170
180 POKE $B9A0, $14  : REM MidStop
190 PRINT "STOPPED"
```

The BASIC interpreter provides higher-level commands for many of these operations: `SAVE`, `LOAD`, `DIR`, `DEL`, `SIDPLAY`, `SIDSTOP`, `MUSIC`, and others. Direct FIO register access is most useful for operations not exposed by BASIC, for programs that need fine-grained error handling, or for accessing the metadata buffer.

Chapter 10

The Expansion Memory System

"640K ought to be enough for anybody — until you actually try to do something with it."

— *Apocryphal, circa 1981*

Introduction

The 96-byte address range from $BA00 to $BFFF packs five distinct subsystems into a dense block of hardware registers, followed by a 1 KB memory-mapped window into expansion RAM:

1. **Expansion Memory Controller** ($BA00–$BA3F) — manages 512 KB of banked XRAM with DMA transfers and a named-block filesystem.
2. **Timer Controller** ($BA40–$BA4F) — a programmable interval timer that can generate IRQs at configurable rates.
3. **Music Status Registers** ($BA50–$BA62) — read-only registers exposing the state of the 14-voice music engine.
4. **DMA Controller** ($BA63–$BA82) — a high-speed block transfer engine that moves data between any combination of CPU RAM, VGC memory spaces, and XRAM.
5. **Blitter Controller** ($BA83–$BAA2) — a 2D block transfer engine with stride control, fill mode, and color-key transparency.

The final 1 KB at $BC00–$BFFF provides four 256-byte windows that map directly into XRAM, allowing the 6502 to read and write expansion memory as though it were ordinary RAM.

10.1 Expansion Memory Controller ($BA00–$BA3F)

The Expansion Memory Controller (XMC) provides access to 512 KB of expansion RAM (XRAM) organized as 8 banks of 64 KB each. Data moves between CPU RAM and XRAM via Stash/Fetch commands, or byte-at-a-time through the data port. A named-block subsystem provides a simple filesystem within XRAM, and page-level allocation tracking prevents accidental overwrites.

10.1.1 Register Map

Address	Name	R/W	Default	Description
$BA00	XmcCmd	W	—	Command register
$BA01	XmcStatus	R	$00	Status code
$BA02	XmcErrCode	R	0	Error code
$BA03	XmcCfg	R/W	0	Reserved
$BA04	XmcAddrL	R/W	0	XRAM address low byte
$BA05	XmcAddrM	R/W	0	XRAM address middle byte
$BA06	XmcAddrH	R/W	0	XRAM address high byte
$BA07	XmcRamL	R/W	0	CPU RAM address low byte
$BA08	XmcRamH	R/W	0	CPU RAM address high byte
$BA09	XmcLenL	R/W	0	Transfer length low byte
$BA0A	XmcLenH	R/W	0	Transfer length high byte
$BA0B	XmcData	R/W	0	Single-byte data port
$BA0C	XmcBank	R/W	0	Active 64 KB bank selector
$BA0D	XmcBanks	R	8	Total bank count
$BA0E	XmcPagesUsedL	R	0	Used 256-byte pages (low)
$BA0F	XmcPagesUsedH	R	0	Used 256-byte pages (high)
$BA10	XmcPagesFreeL	R	0	Free pages (low)
$BA11	XmcPagesFreeH	R	0	Free pages (high)
$BA12	XmcNameLen	R/W	0	Named block name length (0–27)
$BA13	XmcHandle	R/W	0	Named block handle
$BA14	XmcDirCountL	R	0	Named block count (low)
$BA15	XmcDirCountH	R	0	Named block count (high)
$BA16	XmcWinCtl	R/W	0	Window enable bitmask (bits 0–3)
$BA18–$BA1A	XmcWin0A	R/W	0	Window 0 XRAM base (24-bit LE)
$BA1B–$BA1D	XmcWin1A	R/W	0	Window 1 XRAM base (24-bit LE)
$BA1E–$BA20	XmcWin2A	R/W	0	Window 2 XRAM base (24-bit LE)
$BA21–$BA23	XmcWin3A	R/W	0	Window 3 XRAM base (24-bit LE)
$BA24–$BA3F	XmcName	R/W	0	ASCII name buffer (28 bytes)

10.1.2 Detailed Register Descriptions

$BA00 **XmcCmd** W | Default: —

Command register. Writing a command byte here immediately executes the corresponding XMC operation. All parameters (addresses, lengths, names) must be set in the surrounding registers *before* writing to XmcCmd. See Section 10.1.3 for the full command reference.

$BA01 **XmcStatus** R | Default: $00

Status of the last command:

Value	Meaning
$00	Idle — no command has been issued
$02	OK — last command completed successfully
$03	Error — last command failed (see XmcErrCode)

$BA02 **XmcErrCode** R | Default: 0

Detailed error code when XmcStatus is $03:

Value	Meaning
0	No error
1	Range error — address or length out of bounds
2	Bad arguments — invalid parameter combination
3	Not found — named block does not exist
4	No space — insufficient free XRAM for allocation
5	Bad name — invalid or empty name string
6	End of directory — no more named blocks to enumerate

$BA03 **XmcCfg** R/W | Default: 0

Reserved for future use. Reads return 0; writes are accepted but have no effect.

$BA04 **XmcAddrL** R/W | Default: 0

Low byte of the 24-bit XRAM address. Together with XmcAddrM ($BA05) and XmcAddrH ($BA06), this forms the little-endian target address within expansion memory. The full range is $000000–$07FFFF (512 KB).

$BA05 **XmcAddrM** R/W | Default: 0

Middle byte of the 24-bit XRAM address.

$BA06 **XmcAddrH** R/W | Default: 0

High byte of the 24-bit XRAM address.

To set the XRAM address to $012345, write $45 to $BA04, $23 to $BA05, and $01 to $BA06. Little-endian byte order throughout.

$BA07 **XmcRamL** R/W | Default: 0

Low byte of the 16-bit CPU RAM address used by Stash and Fetch commands.

$BA08 **XmcRamH** R/W | Default: 0

High byte of the CPU RAM address.

$BA09 **XmcLenL** R/W | Default: 0

Low byte of the 16-bit transfer length in bytes.

$BA0A **XmcLenH** R/W | Default: 0

High byte of the transfer length.

$BA0B **XmcData** R/W | Default: 0

Single-byte data port. Used by the GetByte and PutByte commands to transfer one byte at a time between the 6502 and XRAM at the address specified by XmcAddr. Also used by the Fill command as the fill value.

$BA0C **XmcBank** R/W | Default: 0

Active 64 KB bank selector. Valid range: 0–7 (for the standard 512 KB configuration). This register provides a convenient shorthand for setting the high byte of XmcAddr: writing bank n is equivalent to setting XmcAddrH to n and clearing XmcAddrM and XmcAddrL.

The bank selector is a convenience for BASIC programs that think in terms of 64 KB banks. Assembly programs typically address XRAM directly via the 24-bit XmcAddr registers.

$BA0D **XmcBanks** R | Default: 8

Total number of 64 KB banks available. Typically 8, giving 512 KB of expansion memory. This is a read-only hardware configuration value.

$BA0E **XmcPagesUsedL** R | Default: 0

Low byte of the number of 256-byte pages currently marked as used.

$BA0F **XmcPagesUsedH** R | Default: 0

High byte of the used page count. Issue the Stats command ($07) to refresh these values before reading.

$BA10 **XmcPagesFreeL** R | Default: 0

Low byte of the number of 256-byte pages currently free.

$BA11 **XmcPagesFreeH** R | Default: 0

High byte of the free page count.

$BA12 **XmcNameLen** R/W | Default: 0

Length of the name string in the XmcName buffer. Valid range: 0–27. Set this before issuing named-block commands (NStash, NFetch, NDelete, NDirOpen, NDirRead).

$BA13 **XmcHandle** R/W | Default: 0

Handle for the current named block. After a successful NStash or NFetch, this register holds the handle assigned to (or looked up for) the block. The handle can be used for subsequent operations without re-specifying the name.

$BA14 **XmcDirCountL** R | Default: 0

Low byte of the total number of named blocks currently stored in XRAM.

$BA15 **XmcDirCountH** R | Default: 0

High byte of the named block count.

$BA16 **XmcWinCtl** R/W | Default: 0

Window enable bitmask. Each of the four bits enables one of the 256-byte memory-mapped windows at $BC00–$BFFF:

7	6	5	4	3	2	1	0
--	--	--	--	Win3	Win2	Win1	Win0

Bit	Window
0	Window 0 ($BC00–$BCFF)
1	Window 1 ($BD00–$BDFF)
2	Window 2 ($BE00–$BEFF)
3	Window 3 ($BF00–$BFFF)

When a window is disabled, reads return $00 and writes are ignored.

$BA18-$BA1A **XmcWin0A** R/W | Default: 0

24-bit little-endian XRAM base address for Window 0. Reads and writes to $BC00–$BCFF are redirected to XRAM starting at this address.

$BA1B-$BA1D **XmcWin1A** R/W | Default: 0

24-bit XRAM base address for Window 1 ($BD00–$BDFF).

$BA1E-$BA20 **XmcWin2A** R/W | Default: 0

24-bit XRAM base address for Window 2 ($BE00–$BEFF).

$BA21-$BA23 **XmcWin3A** R/W | Default: 0

24-bit XRAM base address for Window 3 ($BF00–$BFFF).

$BA24-$BA3F **XmcName** R/W | Default: 0

28-byte ASCII name buffer for named-block operations. Store the block name here before issuing NStash, NFetch, NDelete, or NDirOpen. After NDirRead, this buffer contains the name of the current directory entry. Names are null-padded; set XmcNameLen (`$BA12`) to the actual string length.

10.1.3 Command Reference

Commands are executed by writing the command byte to `$BA00`. Set all required parameters *before* writing the command byte.

Byte-Level **Access**

$01 **GetByte** *Read one byte from XRAM*

Reads the byte at the XRAM address specified by XmcAddr (`$BA04`–`$BA06`) and places it in XmcData (`$BA0B`).

Parameters:

- XmcAddr ($BA04–$BA06): 24-bit XRAM source address

Result: XmcData contains the byte read. XmcStatus = $02.

$02 **PutByte** *Write one byte to XRAM*

Writes the value in XmcData (`$BA0B`) to the XRAM address specified by XmcAddr (`$BA04`–`$BA06`).

Parameters:

- XmcAddr ($BA04–$BA06): 24-bit XRAM destination address
- XmcData ($BA0B): byte to write

Result: XmcStatus = $02.

Block Transfers

$03 Stash *Copy CPU RAM to XRAM*

Copies XmcLen bytes from CPU RAM starting at XmcRam to XRAM starting at XmcAddr.

Parameters:

- XmcRam ($BA07–$BA08): 16-bit source address in CPU RAM
- XmcAddr ($BA04–$BA06): 24-bit destination address in XRAM
- XmcLen ($BA09–$BA0A): number of bytes to copy

Result: XmcStatus = $02 on success, $03 on range error.

$04 Fetch *Copy XRAM to CPU RAM*

Copies XmcLen bytes from XRAM starting at XmcAddr to CPU RAM starting at XmcRam.

Parameters:

- XmcAddr ($BA04–$BA06): 24-bit source address in XRAM
- XmcRam ($BA07–$BA08): 16-bit destination address in CPU RAM
- XmcLen ($BA09–$BA0A): number of bytes to copy

Result: XmcStatus = $02 on success, $03 on range error.

$05 Fill *Fill XRAM with a byte value*

Fills XmcLen bytes of XRAM starting at XmcAddr with the value in XmcData.

Parameters:

- XmcAddr ($BA04–$BA06): 24-bit start address in XRAM
- XmcLen ($BA09–$BA0A): number of bytes to fill
- XmcData ($BA0B): fill value

Result: XmcStatus = $02 on success.

```
10 REM fill the first 256 bytes of XRAM with $AA
20 POKE $BA04, 0 : POKE $BA05, 0 : POKE $BA06, 0
30 DOKE $BA09, 256     : REM length = 256
40 POKE $BA0B, $AA     : REM fill value
50 POKE $BA00, $05     : REM execute Fill
60 IF PEEK($BA01)=$02 THEN PRINT "FILLED OK"
```

Page Management

$07 Stats *Refresh page usage statistics*

Recalculates the used and free page counts and updates XmcPagesUsed (`$BA0E`–`$BA0F`) and XmcPagesFree (`$BA10`–`$BA11`).

Parameters: None.

Result: Page counts updated. XmcStatus = $02.

$08 ResetUsage *Clear page tracking*

Clears all page allocation tracking, marking all XRAM pages as free. Does not erase the actual data — only the bookkeeping is reset.

Parameters: None.
Result: XmcStatus = $02.

Warning

ResetUsage does not zero out XRAM contents. Named blocks stored prior to the reset will still be physically present, but the allocator will treat their pages as free and may overwrite them.

$09 Release *Mark XRAM range as free*

Marks the pages covered by the range starting at XmcAddr with length XmcLen as free.

Parameters:

- XmcAddr ($BA04–$BA06): start of range to release
- XmcLen ($BA09–$BA0A): number of bytes to release

Result: XmcStatus = $02.

$0A Alloc *Allocate XRAM pages*

Allocates XmcLen bytes of contiguous XRAM. On success, the starting address of the allocated block is returned in XmcAddr ($BA04–$BA06).

Parameters:

- XmcLen ($BA09–$BA0A): number of bytes to allocate

Result: XmcAddr updated with the allocated address. XmcStatus = $02 on success, $03 with ErrCode = 4 (no space) if allocation fails.

```
10 REM allocate 1024 bytes of XRAM
20 DOKE $BA09, 1024    : REM length
30 POKE $BA00, $0A     : REM Alloc
40 IF PEEK($BA01)=$03 THEN PRINT "OUT OF MEMORY" : END
50 A = PEEK($BA04) + PEEK($BA05)*256
60 PRINT "ALLOCATED AT: $"; HEX$(A)
```

Named Blocks

Named blocks provide a simple key-value filesystem within XRAM. Each block is identified by a string name (up to 27 characters) and stores an arbitrary-length chunk of data. This allows programs to save and retrieve data structures by name without managing raw XRAM addresses.

$0B NStash *Store CPU RAM as named block*

Copies XmcLen bytes from CPU RAM at XmcRam into XRAM as a named block. The name is read from XmcName ($BA24–$BA3F) with the length in XmcNameLen ($BA12). If a block with the same name already exists, it is replaced.

Parameters:

- XmcRam ($BA07–$BA08): source address in CPU RAM
- XmcLen ($BA09–$BA0A): number of bytes to store
- XmcName ($BA24+): block name (ASCII)
- XmcNameLen ($BA12): name length

Result: XmcHandle ($BA13) set. XmcStatus = $02.

$0C NFetch *Retrieve named block to CPU RAM*

Looks up a named block by name and copies its contents to CPU RAM at XmcRam.

Parameters:

- XmcRam ($BA07–$BA08): destination address in CPU RAM
- XmcName ($BA24+): block name (ASCII)
- XmcNameLen ($BA12): name length

Result: XmcLen updated with the block size. XmcStatus = $02 on success, $03 with ErrCode = 3 (not found) if the name does not exist.

$0D NDelete *Delete named block*

Deletes the named block identified by XmcName and frees its XRAM pages.

Parameters:

- XmcName ($BA24+): block name (ASCII)
- XmcNameLen ($BA12): name length

Result: XmcStatus = $02 on success, $03 with ErrCode = 3 if the name does not exist.

$0E NDirOpen *Open named block directory*

Opens the named-block directory for sequential enumeration. Resets the internal directory index to the first entry.

Parameters: None.

Result: XmcDirCount ($BA14–$BA15) updated with the total number of named blocks. XmcStatus = $02.

$0F NDirRead *Read next directory entry*

Reads the next named block from the directory. The block name is placed in XmcName, the name length in XmcNameLen, and the block size in XmcLen. When all entries have been read, XmcStatus is set to $03 with ErrCode = 6 (end of directory).

Parameters: None (directory must be opened with NDirOpen first).

Result: XmcName, XmcNameLen, and XmcLen populated, or ErrCode = 6 at end.

```
10 REM list all named blocks in XRAM
20 POKE $BA00, $0E     : REM NDirOpen
30 N = DEEK($BA14) : PRINT N; " BLOCKS FOUND"
40 FOR I=1 TO N
50   POKE $BA00, $0F   : REM NDirRead
60   IF PEEK($BA01)=$03 THEN I=N : GOTO 120
70   L = PEEK($BA12) : N$ = ""
80   FOR J=0 TO L-1
90     N$ = N$ + CHR$(PEEK($BA24+J))
100  NEXT J
110  PRINT N$; " ("; DEEK($BA09); " BYTES)"
120 NEXT I
```

10.2 Timer Controller ($BA40–$BA4F)

The Timer Controller provides a single programmable interval timer that can generate IRQ interrupts at a configurable rate. The timer is clocked at the CPU clock rate divided into 100-cycle quanta. When the internal counter reaches the programmed divisor, it sets the IRQ pending flag and resets.

$BA40 TimerCtrl

R/W | Default: 0

Timer control register:

7	6	5	4	3	2	1	0
--	--	--	--	--	--	--	En

Bit	Function
0	Enable: 1 = timer running, 0 = timer stopped
1–7	Reserved (write 0)

$BA41 TimerStatus

R | Default: 0

Timer status register:

7	6	5	4	3	2	1	0
--	--	--	--	--	--	--	IRQ

Bit	Function
0	IRQ pending: 1 = timer has fired. Reading this register clears the flag.
1–7	Reserved

Reading TimerStatus clears the IRQ pending bit. In an interrupt handler, always read this register to acknowledge the timer interrupt and prevent it from firing again immediately.

$BA42 TimerDivL

R/W | Default: 0

Low byte of the 16-bit timer divisor.

$BA43 TimerDivH

R/W | Default: 0

High byte of the timer divisor. The timer fires every `divisor` × `100` CPU cycles. For example, a divisor of 100 fires every 10,000 cycles.

```
10 REM set up a timer IRQ at ~60 Hz
20 REM (assumes 1 MHz clock: 1000000/60 = 16667 cycles)
30 REM divisor = 16667/100 = 167
40 DOKE $BA42, 167     : REM set divisor
50 POKE $BA40, 1       : REM enable timer
60 REM install IRQ handler
70 IRQ 1000
80 PRINT "TIMER RUNNING - PRESS ANY KEY"
90 GET K$ : IF K$="" THEN 90
100 POKE $BA40, 0      : REM disable timer
110 IRQ OFF
```

```
120 END
1000 REM === IRQ handler ===
1010 T = PEEK($BA41) : REM read status (clears IRQ)
1020 PRINT ".";        : REM do something each tick
1030 RETIRQ
```

10.3 Music Status Registers ($BA50–$BA62)

The music status registers provide read-only visibility into the state of the 14-voice music engine. These are useful for visualizations, reactive graphics, and checking whether a sequence has finished playing.

$BA50 MusicStatus

R | Default: 0

Playback status flags:

7	6	5	4	3	2	1	0
--	--	--	--	--	--	Mus	SFX

Bit	Function
0	SFX playing: 1 = a sound effect is currently active
1	Music/MIDI playing: 1 = a music sequence or MIDI file is playing
2–7	Reserved

```
10 REM wait for music to finish
20 IF PEEK($BA50) AND 2 THEN 20
30 PRINT "PLAYBACK COMPLETE"
```

$BA51-$BA5E MusicNote1–14

R | Default: 0

Current MIDI note number for each of the 14 voices. A value of 0 means the voice is silent; values 1–127 correspond to standard MIDI note numbers (60 = Middle C).

Address	Voice	Engine
$BA51	Voice 0	SID chip 1, voice 1
$BA52	Voice 1	SID chip 1, voice 2
$BA53	Voice 2	SID chip 1, voice 3
$BA54	Voice 3	SID chip 2, voice 1
$BA55	Voice 4	SID chip 2, voice 2
$BA56	Voice 5	SID chip 2, voice 3
$BA57	Voice 6	WTS voice 0
$BA58	Voice 7	WTS voice 1
$BA59	Voice 8	WTS voice 2
$BA5A	Voice 9	WTS voice 3
$BA5B	Voice 10	WTS voice 4
$BA5C	Voice 11	WTS voice 5
$BA5D	Voice 12	WTS voice 6
$BA5E	Voice 13	WTS voice 7

```
10 REM display active notes in real-time
20 FOR V=0 TO 13
30   N = PEEK($BA51+V)
40   IF N>0 THEN PRINT "V"; V; "="; N; " ";
```

```
50 NEXT V
60 PRINT
70 VSYNC : GOTO 20
```

$BA5F-$BA60 **MusicElapsed** R | Default: 0

Elapsed frames since playback started, as a 16-bit little-endian value. At 60 Hz, divide by 60 to get elapsed seconds.

$BA61-$BA62 **MusicTotal** R | Default: 0

Total frames in the current music sequence, as a 16-bit little-endian value. Together with MusicElapsed, this allows programs to display a progress indicator.

```
10 REM display music progress
20 E = DEEK($BA5F) : T = DEEK($BA61)
30 IF T=0 THEN PRINT "NO MUSIC LOADED" : END
40 PCT = INT(E*100/T)
50 PRINT "PROGRESS:"; PCT; "%  ("; INT(E/60); "/"; INT(T/60); " SEC)
   "
60 VSYNC : IF PEEK($BA50) AND 2 THEN 20
70 PRINT "DONE"
```

10.4 DMA Controller ($BA63–$BA82)

The DMA Controller provides high-speed block transfers between any combination of six memory spaces: CPU RAM, four VGC memory regions (character, color, graphics, sprite), and XRAM. It operates at 4 bytes per CPU cycle, making it vastly faster than any software copy loop.

10.4.1 Register Map

Address	Name	R/W	Default	Description
$BA63	DmaCmd	W	—	Command register
$BA64	DmaStatus	R	$00	Status code
$BA65	DmaErrCode	R	0	Error code
$BA66	DmaSrcSpace	R/W	0	Source memory space
$BA67	DmaDstSpace	R/W	0	Destination memory space
$BA68	DmaSrcL	R/W	0	Source address low byte
$BA69	DmaSrcM	R/W	0	Source address middle byte
$BA6A	DmaSrcH	R/W	0	Source address high byte
$BA6B	DmaDstL	R/W	0	Destination address low byte
$BA6C	DmaDstM	R/W	0	Destination address middle byte
$BA6D	DmaDstH	R/W	0	Destination address high byte
$BA6E	DmaLenL	R/W	0	Transfer length low byte
$BA6F	DmaLenM	R/W	0	Transfer length middle byte
$BA70	DmaLenH	R/W	0	Transfer length high byte
$BA71	DmaMode	R/W	0	Mode flags
$BA72	DmaFillValue	R/W	0	Fill byte
$BA73	DmaCountL	R	0	Bytes transferred (low)
$BA74	DmaCountM	R	0	Bytes transferred (mid)
$BA75	DmaCountH	R	0	Bytes transferred (high)

10.4.2 Detailed Register Descriptions

$BA63 **DmaCmd** W | Default: —

Command register. Writing $01 starts a DMA transfer using the parameters set in the surrounding registers.

$01 **Start Transfer** *Begin DMA operation*

Initiates a block copy (or fill) according to the current register state. The transfer completes before the next CPU instruction executes.
Result: DmaStatus = $02 on success, $03 on error.

$BA64 **DmaStatus** R | Default: $00

Status of the last DMA operation:

Value	Meaning
$00	Idle
$01	Busy (transfer in progress)
$02	OK — transfer completed
$03	Error — see DmaErrCode

$BA65 **DmaErrCode** R | Default: 0

Detailed error code when DmaStatus is $03:

Value	Meaning
0	No error
1	Bad command — invalid command byte
2	Bad space — invalid source or destination space ID
3	Range error — address or length out of bounds
4	Bad arguments — invalid parameter combination
5	Write protected — destination is read-only

$BA66 **DmaSrcSpace** R/W | Default: 0

Source memory space selector:

Value	Space
0	CPU RAM (64 KB)
1	VGC Character RAM (2000 bytes)
2	VGC Color RAM (2000 bytes)
3	VGC Graphics bitmap (32 KB)
4	VGC Sprite data
5	XRAM (512 KB, 24-bit addressing)

$BA67 **DmaDstSpace** R/W | Default: 0

Destination memory space selector. Uses the same space IDs as DmaSrcSpace.

$BA68 **DmaSrcL** R/W | Default: 0

Low byte of the 24-bit source address.

$BA69 **DmaSrcM** R/W | Default: 0

Middle byte of the source address.

$BA6A **DmaSrcH** R/W | Default: 0

High byte of the source address. For CPU RAM (space 0), only the low two bytes are significant. For XRAM (space 5), all three bytes are used.

$BA6B **DmaDstL** R/W | Default: 0

Low byte of the 24-bit destination address.

$BA6C **DmaDstM** R/W | Default: 0

Middle byte of the destination address.

$BA6D **DmaDstH** R/W | Default: 0

High byte of the destination address.

$BA6E **DmaLenL** R/W | Default: 0

Low byte of the 24-bit transfer length.

$BA6F **DmaLenM** R/W | Default: 0

Middle byte of the transfer length.

$BA70 **DmaLenH** R/W | Default: 0

High byte of the transfer length.

$BA71 **DmaMode** R/W | Default: 0

DMA mode flags:

7	6	5	4	3	2	1	0
--	--	--	--	--	--	--	Fill

Bit	Function
0	Fill mode: when set, the destination is filled with DmaFillValue instead of copying from the source
1–7	Reserved (write 0)

$BA72 **DmaFillValue** R/W | Default: 0

The byte value used to fill the destination when DmaMode bit 0 is set.

$BA73 **DmaCountL** R | Default: 0

Low byte of the 24-bit count of bytes transferred so far.

$BA74 **DmaCountM** R | Default: 0

Middle byte of the bytes transferred count.

$BA75 **DmaCountH** R | Default: 0

High byte of the bytes transferred count.

10.4.3 BASIC Examples

Copy Screen RAM to XRAM

```
10 REM save a copy of screen RAM to XRAM bank 0
20 POKE $BA66, 1       : REM source = VGC char RAM
30 POKE $BA67, 5       : REM destination = XRAM
40 REM source offset = 0
50 POKE $BA68, 0 : POKE $BA69, 0 : POKE $BA6A, 0
60 REM destination = XRAM $000000
70 POKE $BA6B, 0 : POKE $BA6C, 0 : POKE $BA6D, 0
80 REM length = 2000 bytes (80x25 screen)
90 POKE $BA6E, $D0 : POKE $BA6F, $07 : POKE $BA70, 0
100 POKE $BA71, 0      : REM copy mode (not fill)
110 POKE $BA63, $01    : REM start transfer
120 IF PEEK($BA64)=$02 THEN PRINT "SCREEN SAVED TO XRAM"
130 IF PEEK($BA64)=$03 THEN PRINT "ERROR:"; PEEK($BA65)
```

Fill Graphics Bitmap with a Color

```
10 REM fill the entire graphics bitmap with color $05
20 POKE $BA67, 3       : REM destination = VGC graphics
30 REM destination offset = 0
40 POKE $BA6B, 0 : POKE $BA6C, 0 : POKE $BA6D, 0
50 REM length = 32000 bytes (320x200 / 2 nybbles)
60 POKE $BA6E, 0 : POKE $BA6F, $7D : POKE $BA70, 0
70 POKE $BA71, 1       : REM fill mode
80 POKE $BA72, $55     : REM fill value ($5 in both nybbles)
90 POKE $BA63, $01     : REM start transfer
100 IF PEEK($BA64)=$02 THEN PRINT "GRAPHICS CLEARED"
```

The BASIC statements `DMACOPY` and `DMAFILL` provide convenient wrappers around the DMA controller. Direct register access is useful when you need 24-bit XRAM addressing or fine-grained error checking.

10.5 Blitter Controller ($BA83–$BAA2)

The Blitter Controller performs 2D rectangular block transfers with stride control. Unlike the DMA controller which operates on flat linear regions, the blitter understands rows and columns, making it ideal for copying rectangular regions of screen or bitmap data. It supports fill mode and color-key transparency for sprite-like compositing.

The blitter operates at 2 operations per CPU cycle.

10.5.1 Register Map

Address	Name	R/W	Default	Description
$BA83	BltCmd	W	—	Command register

Address	Name	R/W	Default	Description
$BA84	BltStatus	R	$00	Status code
$BA85	BltErrCode	R	0	Error code
$BA86	BltSrcSpace	R/W	0	Source memory space
$BA87	BltDstSpace	R/W	0	Destination memory space
$BA88	BltSrcL	R/W	0	Source base address low
$BA89	BltSrcM	R/W	0	Source base address middle
$BA8A	BltSrcH	R/W	0	Source base address high
$BA8B	BltDstL	R/W	0	Destination base address low
$BA8C	BltDstM	R/W	0	Destination base address middle
$BA8D	BltDstH	R/W	0	Destination base address high
$BA8E	BltWidthL	R/W	0	Rectangle width low
$BA8F	BltWidthH	R/W	0	Rectangle width high
$BA90	BltHeightL	R/W	0	Rectangle height low
$BA91	BltHeightH	R/W	0	Rectangle height high
$BA92	BltSrcStrideL	R/W	0	Source row stride low
$BA93	BltSrcStrideH	R/W	0	Source row stride high
$BA94	BltDstStrideL	R/W	0	Destination row stride low
$BA95	BltDstStrideH	R/W	0	Destination row stride high
$BA96	BltMode	R/W	0	Mode flags
$BA97	BltFillValue	R/W	0	Fill byte
$BA98	BltColorKey	R/W	0	Transparent color
$BA99	BltCountL	R	0	Bytes written (low)
$BA9A	BltCountM	R	0	Bytes written (middle)
$BA9B	BltCountH	R	0	Bytes written (high)

10.5.2 Detailed Register Descriptions

$BA83 **BltCmd** W | Default: —

Command register. Writing $01 starts a blit operation.

$01 **Start** *Begin blit operation*

Initiates a 2D block transfer (or fill) using the current register state. For each row, the blitter copies (or fills) BltWidth bytes from the source to the destination, then advances both pointers by their respective strides. This repeats for BltHeight rows.

Result: BltStatus = $02 on success, $03 on error.

$BA84 **BltStatus** R | Default: $00

Status of the last blit operation. Uses the same codes as the DMA controller:

Value	Meaning
$00	Idle
$01	Busy
$02	OK
$03	Error

$BA85 **BltErrCode** R | Default: 0

Error code. Uses the same codes as the DMA controller (see Section 10.4).

$BA86 **BltSrcSpace** R/W | Default: 0

Source memory space selector. Uses the same space IDs as the DMA controller (0 = CPU RAM, 1 = VGC char, 2 = VGC color, 3 = VGC gfx, 4 = VGC sprite, 5 = XRAM).

$BA87 **BltDstSpace** R/W | Default: 0

Destination memory space selector.

$BA88 **BltSrcL** R/W | Default: 0

Low byte of the 24-bit source base address.

$BA89 **BltSrcM** R/W | Default: 0

Middle byte of the source base address.

$BA8A **BltSrcH** R/W | Default: 0

High byte of the source base address.

$BA8B **BltDstL** R/W | Default: 0

Low byte of the 24-bit destination base address.

$BA8C **BltDstM** R/W | Default: 0

Middle byte of the destination base address.

$BA8D **BltDstH** R/W | Default: 0

High byte of the destination base address.

$BA8E **BltWidthL** R/W | Default: 0

Low byte of the rectangle width in bytes.

$BA8F **BltWidthH** R/W | Default: 0

High byte of the rectangle width.

$BA90 **BltHeightL** R/W | Default: 0

Low byte of the rectangle height in rows.

$BA91 **BltHeightH** R/W | Default: 0

High byte of the rectangle height.

$BA92 **BltSrcStrideL** R/W | Default: 0

Low byte of the source row stride. The stride is the number of bytes between the start of one row and the start of the next in the source memory space. For a full-width screen copy, this equals the screen width.

$BA93 **BltSrcStrideH** R/W | Default: 0

High byte of the source row stride.

$BA94 **BltDstStrideL** R/W | Default: 0

Low byte of the destination row stride.

$BA95 **BltDstStrideH** R/W | Default: 0

High byte of the destination row stride.

$BA96 **BltMode** R/W | Default: 0

Blitter mode flags:

7	6	5	4	3	2	1	0
--	--	--	--	--	--	Key	Fill

Bit	Function
0	Fill mode: when set, the destination is filled with BltFillValue instead of copying from the source. Width and height still control the rectangular region.
1	Color-key transparency: when set, source bytes matching BltColorKey are skipped (the destination byte is not overwritten). This enables transparent blitting for sprite-like effects.
2–7	Reserved (write 0)

$BA97 **BltFillValue** R/W | Default: 0

Fill byte used when BltMode bit 0 is set.

$BA98 **BltColorKey** R/W | Default: 0

Transparent color. When BltMode bit 1 is set, source bytes equal to this value are skipped during the copy, leaving the destination unchanged.

$BA99 **BltCountL** R | Default: 0

Low byte of the 24-bit count of bytes actually written.

$BA9A **BltCountM** R | Default: 0

Middle byte.

$BA9B **BltCountH** R | Default: 0

High byte. When color-key transparency is active, BltCount may be less than Width × Height because skipped pixels are not counted.

10.5.3 BASIC Examples

Blit a Rectangular Region with Transparency

```
10 REM blit a 20x10 region from XRAM to graphics
20 REM with color 0 as transparent
30 POKE $BA86, 5       : REM source = XRAM
40 POKE $BA87, 3       : REM destination = VGC graphics
50 REM source address = XRAM $000000
60 POKE $BA88, 0 : POKE $BA89, 0 : POKE $BA8A, 0
70 REM destination = graphics offset 1000
80 POKE $BA8B, $E8 : POKE $BA8C, $03 : POKE $BA8D, 0
90 REM width=20, height=10
```

```
100 DOKE $BA8E, 20 : DOKE $BA90, 10
110 REM source stride = 20 (tiles packed tightly)
120 DOKE $BA92, 20
130 REM destination stride = 160 (320/2 nybbles per row)
140 DOKE $BA94, 160
150 REM mode: color-key transparency on (bit 1)
160 POKE $BA96, 2
170 POKE $BA98, 0      : REM color key = 0 (transparent)
180 POKE $BA83, $01    : REM start blit
190 IF PEEK($BA84)=$02 THEN PRINT "BLIT DONE"
200 IF PEEK($BA84)=$03 THEN PRINT "ERROR:"; PEEK($BA85)
```

Fill a Rectangle on Screen

```
10 REM fill a 40x5 rectangle of character RAM with 'X'
20 POKE $BA87, 1       : REM destination = VGC char RAM
30 REM destination = row 10, column 20 (offset = 10*80+20 = 820)
40 POKE $BA8B, $34 : POKE $BA8C, $03 : POKE $BA8D, 0
50 DOKE $BA8E, 40      : REM width = 40 columns
60 DOKE $BA90, 5       : REM height = 5 rows
70 DOKE $BA94, 80      : REM destination stride = 80 (screen width)
80 POKE $BA96, 1       : REM fill mode
90 POKE $BA97, ASC("X") : REM fill with 'X'
100 POKE $BA83, $01    : REM start blit
```

The BASIC statements `BLITCOPY` and `BLITFILL` provide convenient wrappers around the blitter. Use direct register access when you need color-key transparency or cross-space blits that the BASIC statements do not expose.

10.6 XMC Windows ($BC00–$BFFF)

$BC00-$BFFF XMC Window

R/W | Default: 0

The final 1 KB of this address range is divided into four 256-byte windows that provide direct memory-mapped access to XRAM. Each window maps a contiguous 256-byte page of XRAM into the 6502 address space, allowing reads and writes to pass through transparently — no commands, no register setup, just ordinary `PEEK` and `POKE`.

CPU Address	Window	Enable Bit	Base Register
$BC00–$BCFF	Window 0	Bit 0 of $BA16	XmcWin0A ($BA18–$BA1A)
$BD00–$BDFF	Window 1	Bit 1 of $BA16	XmcWin1A ($BA1B–$BA1D)
$BE00–$BEFF	Window 2	Bit 2 of $BA16	XmcWin2A ($BA1E–$BA20)
$BF00–$BFFF	Window 3	Bit 3 of $BA16	XmcWin3A ($BA21–$BA23)

When a window is **enabled**, reads from its 256-byte range return the corresponding byte from XRAM, and writes store the byte into XRAM. When a window is **disabled**, reads return $00 and writes are silently discarded.

The base address register specifies the XRAM address that the window's first byte ($xx00) maps to. The remaining 255 bytes map to consecutive XRAM addresses.

```
10 REM map window 0 to XRAM page $000100
20 POKE $BA18, 0 : POKE $BA19, 1 : POKE $BA1A, 0
30 POKE $BA16, 1  : REM enable window 0
40 POKE $BC00, 42 : REM writes to XRAM $000100
50 PRINT PEEK($BC00) : REM reads from XRAM $000100
```

```
10 REM use all four windows simultaneously
20 REM map to four consecutive XRAM pages
30 FOR W=0 TO 3
40   POKE $BA18+W*3, 0 : POKE $BA19+W*3, W : POKE $BA1A+W*3, 0
50 NEXT W
60 POKE $BA16, $0F : REM enable all four windows
70 REM now $BC00-$BFFF maps to XRAM $000000-$0003FF
80 FOR I=0 TO 1023
90   POKE $BC00+I, I AND 255
100 NEXT I
110 PRINT "WROTE 1KB TO XRAM VIA WINDOWS"
```

Windows provide the fastest way to access small, frequently-used regions of XRAM. For bulk transfers, the Stash/Fetch commands or DMA controller are more efficient.

Chapter 11

The ROM

"In ROM we trust — for everything else, there's RAM."

— *Anonymous systems programmer*

Introduction

The upper 16 KB of the 6502 address space — from $C000 to $FFFF — is occupied by read-only memory containing the EhBASIC interpreter and monitor code. Writes to this range are silently dropped, with two important exceptions:

- **SID registers** at $D400–$D43F are intercepted on write and routed to the SID sound chip emulation. Reads still return ROM data.
- **Hardware vectors** at $FFFA–$FFFF are writable so that programs can install custom interrupt handlers and SID players can redirect execution.

The ROM image is loaded from `ehbasic.bin` at boot time. It provides the complete NovaBASIC environment: tokenizer, expression evaluator, math library, string handling, I/O dispatch, and the extended command set that distinguishes NovaBASIC from stock EhBASIC.

11.1 ROM Layout Overview

$C000–$FFF9 **EhBASIC ROM** R | Default: —

The 16 KB ROM contains the following regions (approximate boundaries — exact addresses depend on the build):

Range	Region	Contents
$C000–$C002	Cold start entry	JMP to EhBASIC initialization
$C003	Warm start entry	JMP to EhBASIC warm start
$C000–$DFFF	Interpreter core	Tokenizer, executor, math routines, string handling, memory management
$E000–$EFFF	Functions & I/O	Built-in functions, commands, I/O handlers
$F000–$FF7F	Extended commands	NovaBASIC additions (graphics, sound, networking, expansion memory, DMA, blitter, etc.)
$FF80–$FFF9	Monitor code	Power-on initialization and vector setup

Range	Region	Contents
$FFFA–$FFFF	Hardware vectors	NMI, RESET, IRQ/BRK vectors (writable)

The boundaries above are approximate. The interpreter, functions, and extended commands are compiled from assembly source, and their exact layout shifts with each build. The entry points at $C000, $C003, and $FF80 are fixed by design.

11.2 Key Entry Points

$C000 LAB_COLD

R | Default: —

EhBASIC cold start entry point. Jumping here initializes all interpreter state: clears variables, resets the program pointer, sets up the default I/O vectors, prints the startup banner, and enters the command loop (the `Ready` prompt).

This is the normal entry path after the monitor code completes its setup. A cold start erases any BASIC program in memory.

$C003 LAB_WARM

R | Default: —

EhBASIC warm start entry point. Jumping here returns to the command prompt *without* erasing the current BASIC program in memory. Variables are cleared, but program text is preserved.

This entry point is used by the `STOP` statement, the `Break` key handler, and by user code that needs to abort execution without losing the loaded program.

11.3 Monitor Code ($FF80–$FFF9)

$FF80-$FFF9 Monitor Code

R | Default: —

The reset vector at $FFFC points to $FF80. This is the first code executed at power-on or after a hardware reset. The monitor performs the following initialization sequence:

1. **CLD** — Clear decimal mode flag to ensure binary arithmetic.
2. **LDX #$FF; TXS** — Initialize the stack pointer to the top of the stack page ($01FF).
3. **Copy I/O vectors** — Writes the default I/O vector table to page 2 ($0200). This table contains the addresses of the character input, character output, and load/save routines that EhBASIC calls through indirect jumps.
4. **Check NOAUTO** — Reads the `NOAUTO` environment variable. If set, the monitor skips auto-run and proceeds directly to EhBASIC.
5. **JMP LAB_COLD** — Transfers control to the EhBASIC cold start at $C000.

The monitor code is deliberately minimal. Its sole purpose is to bring the machine to a known good state and hand off to the BASIC interpreter. There is no built-in machine language monitor, hex editor, or disassembler — those functions are provided by the host

debugger or the MCP server.

11.4 SID Register Window

The SID sound chips are mapped at $D400–$D41F (SID 1) and $D420–$D43F (SID 2) within this ROM range. Although reads return ROM data, writes are intercepted and routed to the SID emulation hardware. See Chapter 12 for complete SID register documentation.

This dual-nature arrangement preserves compatibility with C64 SID player files, which expect to write SID registers at $D400, while allowing the ROM image to occupy a contiguous 16 KB block without gaps.

When reading back SID register values for diagnostic purposes, use the SID shadow registers or the MCP server's `sid_read` command. Reading $D400–$D43F directly returns ROM bytes, not the values last written to the SID chips.

11.5 Hardware Vectors ($FFFA–$FFFF)

$FFFA-$FFFF Hardware Vectors

R/W | Default: Set at boot

The six bytes at the very top of the address space contain the three hardware interrupt vectors. These are the *only* writable bytes in the entire ROM range. They are writable so that SID players, custom interrupt handlers, and timer-driven routines can redirect execution.

Address	Vector	Description
$FFFA–$FFFB	NMI	Non-Maskable Interrupt handler address. Triggered by an external NMI signal; cannot be disabled by the I flag.
$FFFC–$FFFD	RESET	Reset vector. Points to $FF80 (monitor code) at boot. The CPU reads this vector on power-on and hardware reset.
$FFFE–$FFFF	IRQ/BRK	Maskable Interrupt / Break handler address. Triggered by the IRQ line (when the I flag is clear) or by a BRK instruction.

Each vector is a 16-bit little-endian address. The CPU reads these vectors in hardware — there is no software dispatch layer.

Warning

Overwriting the RESET vector will prevent the system from booting correctly on next reset. Only modify the NMI and IRQ vectors unless you know what you are doing.

```
10 REM read the hardware vectors
20 PRINT "NMI:   $"; HEX$(DEEK($FFFA))
30 PRINT "RESET: $"; HEX$(DEEK($FFFC))
40 PRINT "IRQ:   $"; HEX$(DEEK($FFFE))
```

```
10 REM install a custom IRQ handler at address $8000
20 REM (assumes handler code is already loaded there)
30 DOKE $FFFE, $8000
40 PRINT "IRQ VECTOR NOW POINTS TO $"; HEX$(DEEK($FFFE))
```

The BASIC statements `IRQ` and `NMI` provide a safer, higher-level way to install interrupt handlers. They handle the vector setup, register saving, and proper return sequence automatically. Direct vector manipulation is only needed for assembly-language interrupt handlers.

Chapter 12

The SID Sound Chips

"Without music, life would be a mistake."

— *Friedrich Nietzsche*

Introduction

The NovaVM includes a software emulation of the legendary MOS 6581 Sound Interface Device — the **SID** chip that gave the Commodore 64 its distinctive sonic character. Two independent SID chips are provided:

Chip	Address Range	Voices	Music Engine Voices
SID 1	$D400–$D41F	1–3	0–2
SID 2	$D420–$D43F	1–3	3–5

Each chip provides three independently programmable voices, giving six SID voices in total. Every voice has: 16-bit frequency control, 12-bit pulse width modulation, waveform selection (triangle, sawtooth, pulse, noise), a four-stage ADSR envelope generator, ring modulation, and hard synchronisation. Each chip also provides a multi-mode resonant filter (low-pass, band-pass, high-pass) and a 4-bit master volume control.
These addresses sit inside the ROM range ($C000–$FFFF). Reads return ROM data — you cannot read back SID register values from these addresses. Writes, however, are intercepted by the composite bus and routed to the SID emulation engine.

The SID emulation renders audio in a background thread via OpenAL at 44 100 Hz, fully decoupled from the CPU clock. Audio output is mono per chip; stereo panning is available through the Wavetable Synthesizer or by using both SID chips simultaneously.

12.1 Voice Registers

Each SID chip has three voices occupying seven bytes each. The voice base addresses for SID 1 are:

Voice	Base	End	Offset from SID base
1	`$D400`	`$D406`	+$00
2	`$D407`	`$D40D`	+$07
3	`$D40E`	`$D414`	+$0E

For SID 2, add $20 to all addresses (voice 1 starts at $D420, etc.).
The register offsets within each voice are identical. In the entries below, the address shown is for SID 1 voice 1; add $07 for voice 2, $0E for voice 3, or $20 for the corresponding SID 2 voice.

$D400 FreqLo — Frequency Low

W | Default: 0

Low 8 bits of the 16-bit oscillator frequency.

$D401 FreqHi — Frequency High

W | Default: 0

High 8 bits of the 16-bit oscillator frequency. The output frequency is calculated as:

$$f_{\text{out}} = \frac{(\mathit{FreqHi} \times 256 + \mathit{FreqLo}) \times f_{\text{clock}}}{16\,777\,216}$$

where f_{clock} is the CPU clock rate (985 248 Hz PAL).

Note	Freq Value (hex)	Freq Value (dec)
C-4 (middle C)	$1CD6	7382
A-4 (concert A)	$23C5	9157

$D402 PwLo — Pulse Width Low

W | Default: 0

Low 8 bits of the 12-bit pulse width value.

$D403 PwHi — Pulse Width High

W | Default: 0

High 4 bits of the 12-bit pulse width (bits 0–3 only; bits 4–7 are unused). The duty cycle is:

$$\text{Duty} = \frac{PW}{4095}$$

A value of $800 (2048) produces a 50% square wave. Lower values narrow the pulse; higher values widen it.

$D404 WfCtrl — Waveform Control

W | Default: 0

7	6	5	4	3	2	1	0
Noise	Pulse	Saw	Tri	Test	Ring	Sync	Gate

Bit 7 — Noise Select noise waveform.

Bit 6 — Pulse Select pulse waveform. Width set by PwLo/PwHi.

Bit 5 — Saw Select sawtooth waveform.

Bit 4 — Tri Select triangle waveform.

Bit 3 — Test Silence the voice and reset the noise LFSR.

Bit 2 — Ring Enable ring modulation. The triangle output of this voice is XORed with bit 23 of the *previous* voice's oscillator, producing bell-like or metallic timbres. Voice 1's ring mod source is voice 3.

Bit 1 — Sync Enable hard sync. This voice's oscillator is reset whenever the previous voice's oscillator completes a cycle, creating harmonically rich timbres at the frequency ratio of the two voices.

Bit 0 — Gate Setting this bit starts the attack phase of the ADSR envelope. Clearing it begins the release phase.

Selecting multiple waveforms simultaneously produces their logical AND, which is generally not useful except for specific C64 sound effects. Stick to one waveform bit at a time for predictable results.

$D405 AD — Attack / Decay

W | Default: 0

7	6	5	4	3	2	1	0
A3	A2	A1	A0	D3	D2	D1	D0

Bits 7–4 — Attack Attack rate (0–15). Controls how quickly the envelope rises from zero to peak after the gate opens.

Bits 3–0 — Decay Decay rate (0–15). Controls how quickly the envelope falls from peak to the sustain level.

ADSR Timing Table The following table lists the approximate durations for each rate value. Attack times represent the time from zero to full amplitude; decay and release times represent the time from full amplitude to zero (the actual decay stops at the sustain level).

Value	Attack Time	Decay / Release Time
0	2 ms	6 ms
1	8 ms	24 ms
2	16 ms	48 ms
3	24 ms	72 ms
4	38 ms	114 ms
5	56 ms	168 ms
6	68 ms	204 ms
7	80 ms	240 ms
8	100 ms	300 ms
9	250 ms	750 ms
10	500 ms	1.5 s
11	800 ms	2.4 s
12	1 s	3 s
13	3 s	9 s
14	5 s	15 s
15	8 s	24 s

$D406 SR — Sustain / Release

W | Default: 0

7	6	5	4	3	2	1	0
S3	S2	S1	S0	R3	R2	R1	R0

Bits 7–4 — Sustain Sustain level (0–15). The envelope holds at this level after the decay phase completes, for as long as the gate remains set. 0 = silent, 15 = full volume.

Bits 3–0 — Release Release rate (0–15). Controls how quickly the envelope falls to zero after the gate is cleared. Uses the same timing values as the decay rate (see table above).

12.2 Filter Registers ($D415–$D418)

The SID filter is a state-variable design capable of low-pass, band-pass, and high-pass modes (and combinations thereof). Each voice can be independently routed through the filter or mixed directly to the output.

$D415 FiltLoFreq — Filter Cutoff Low

W | Default: 0

Low 3 bits of the 11-bit filter cutoff frequency (bits 0–2 only; bits 3–7 are unused).

$D416 FiltHiFreq — Filter Cutoff High

W | Default: 0

High 8 bits of the 11-bit filter cutoff frequency.

$D417 FiltCtrl — Filter Resonance & Voice Select

W | Default: 0

7	6	5	4	3	2	1	0
Res3	Res2	Res1	Res0	FiltEx	Filt3	Filt2	Filt1

Bits 7–4 — Resonance Filter resonance (0–15). Higher values produce a sharper peak at the cutoff frequency.

Bit 3 — FiltEx Route external audio input through the filter. Not used in the NovaVM emulation.

Bit 2 — Filt3 Route voice 3 through the filter.

Bit 1 — Filt2 Route voice 2 through the filter.

Bit 0 — Filt1 Route voice 1 through the filter.

$D418 VolMode — Filter Mode & Master Volume

W | Default: 0

7	6	5	4	3	2	1	0
V3Off	HP	BP	LP	Vol3	Vol2	Vol1	Vol0

Bit 7 — V3Off Mute voice 3 output. The voice's oscillator continues to run (useful as a modulation source for ring mod or sync without producing audible output).

Bit 6 — HP Enable high-pass filter mode.

Bit 5 — BP Enable band-pass filter mode.

Bit 4 — LP Enable low-pass filter mode.

Bits 3–0 — Volume Master volume (0–15). This is the overall output level for all three voices on this chip.

Multiple filter modes can be enabled simultaneously. Combining low-pass and high-pass produces a notch (band-reject) filter. Enabling all three gives a roughly flat response with a resonant peak at the cutoff frequency.

12.3 Extended Per-Voice Volume ($D41D–$D41F)

$D41D **Voice 1 Volume** W | Default: $0F

Per-voice volume for voice 1 (0–15, 4-bit). Only the low nibble is used.

$D41E **Voice 2 Volume** W | Default: $0F

Per-voice volume for voice 2 (0–15, 4-bit).

$D41F **Voice 3 Volume** W | Default: $0F

Per-voice volume for voice 3 (0–15, 4-bit).

These registers are a **NovaBASIC extension** not present on the original MOS 6581 or 8580. They provide independent volume control for each voice, supplementing the 4-bit master volume in register $D418. The default value of $0F (full) ensures that existing SID code behaves identically to a standard SID chip.

12.4 SID 2 ($D420–$D43F)

$D420-$D43F **SID Chip 2** W | Default: 0

The second SID chip has an identical register layout, offset by $20 from SID 1. All voice, filter, and extended volume registers described above apply — simply add $20 to each address.

SID 1	SID 2	Function
$D400–$D406	$D420–$D426	Voice 1
$D407–$D40D	$D427–$D42D	Voice 2
$D40E–$D414	$D42E–$D434	Voice 3
$D415–$D418	$D435–$D438	Filter + master volume
$D41D–$D41F	$D43D–$D43F	Per-voice volume (ext.)

12.5 BASIC Examples

Playing a Simple Note

```
10 REM play middle C on SID voice 1
20 POKE $D418, 15        : REM max volume
30 POKE $D405, $22       : REM attack=2, decay=2
40 POKE $D406, $A8       : REM sustain=10, release=8
50 POKE $D400, $D6       : REM freq low
60 POKE $D401, $1C       : REM freq high (middle C)
70 POKE $D404, $21       : REM sawtooth + gate on
80 FOR I=1 TO 5000:NEXT  : REM wait
90 POKE $D404, $20       : REM gate off (release)
```

Filter Sweep

```
10 REM low-pass filter sweep on a pulse wave
20 POKE $D418, 15        : REM max volume
30 POKE $D402, 0         : REM pulse width low
40 POKE $D403, 8         : REM pulse width high (50%)
50 POKE $D405, $09       : REM attack=0, decay=9
60 POKE $D406, $A0       : REM sustain=10, release=0
70 POKE $D400, $D6       : REM freq low  (middle C)
80 POKE $D401, $1C       : REM freq high
90 POKE $D417, $01       : REM route voice 1 through filter
100 POKE $D418, $1F      : REM low-pass + max volume
110 POKE $D404, $41      : REM pulse + gate on
120 FOR F=0 TO 255
130   POKE $D416, F       : REM sweep cutoff high byte
140   FOR W=1 TO 20:NEXT : REM small delay
150 NEXT F
160 POKE $D404, $40      : REM gate off
```

Two-Voice Ring Modulation

```
10 REM ring modulation: voice 1 modulated by voice 3
20 POKE $D418, 15        : REM max volume
30 REM set voice 3 as modulation source (no audible output)
40 POKE $D40E, $00       : REM voice 3 freq low
50 POKE $D40F, $03       : REM voice 3 freq high (low freq)
60 POKE $D412, $11       : REM triangle + gate on
70 POKE $D413, $00       : REM attack/decay = 0
80 POKE $D414, $F0       : REM sustain=15, release=0
90 REM voice 1 with ring mod enabled
100 POKE $D405, $09      : REM attack=0, decay=9
110 POKE $D406, $A8      : REM sustain=10, release=8
120 POKE $D400, $D6      : REM freq low  (middle C)
130 POKE $D401, $1C      : REM freq high
140 POKE $D404, $15      : REM triangle + ring mod + gate on
150 FOR I=1 TO 8000:NEXT
160 POKE $D404, $14      : REM gate off
170 POKE $D412, $10      : REM voice 3 gate off
```

12.6 Register Summary

The complete register map for SID 1 is shown below. SID 2 is identical with a base address of $D420.

Address	Register	R/W	Description
Voice 1			
$D400	FreqLo	W	Frequency low byte
$D401	FreqHi	W	Frequency high byte
$D402	PwLo	W	Pulse width low byte
$D403	PwHi	W	Pulse width high nibble (bits 0–3)
$D404	WfCtrl	W	Waveform / gate / ring / sync / test
$D405	AD	W	Attack (bits 7–4), decay (bits 3–0)

continued on next page...

Address	Register	R/W	Description
$D406	SR	W	Sustain (bits 7–4), release (bits 3–0)
Voice 2			
$D407	FreqLo	W	Frequency low byte
$D408	FreqHi	W	Frequency high byte
$D409	PwLo	W	Pulse width low byte
$D40A	PwHi	W	Pulse width high nibble (bits 0–3)
$D40B	WfCtrl	W	Waveform / gate / ring / sync / test
$D40C	AD	W	Attack (bits 7–4), decay (bits 3–0)
$D40D	SR	W	Sustain (bits 7–4), release (bits 3–0)
Voice 3			
$D40E	FreqLo	W	Frequency low byte
$D40F	FreqHi	W	Frequency high byte
$D410	PwLo	W	Pulse width low byte
$D411	PwHi	W	Pulse width high nibble (bits 0–3)
$D412	WfCtrl	W	Waveform / gate / ring / sync / test
$D413	AD	W	Attack (bits 7–4), decay (bits 3–0)
$D414	SR	W	Sustain (bits 7–4), release (bits 3–0)
Filter			
$D415	FiltLoFreq	W	Filter cutoff low (bits 0–2)
$D416	FiltHiFreq	W	Filter cutoff high byte
$D417	FiltCtrl	W	Resonance (bits 7–4), voice routing (bits 0–2)
$D418	VolMode	W	Filter mode (bits 4–7), master volume (bits 0–3)
Extended (NovaBASIC)			
$D41D	Voice 1 Vol	W	Per-voice volume (0–15, default $0F)
$D41E	Voice 2 Vol	W	Per-voice volume (0–15, default $0F)
$D41F	Voice 3 Vol	W	Per-voice volume (0–15, default $0F)

Chapter A

VGC Bitmap & Sprite Shape RAM

"The screen is a window into the machine's imagination."

— *Anonymous demo coder, 1989*

Introduction

The VGC manages two large memory spaces that live entirely outside the 6502's 64 KB address space. The graphics bitmap holds 64,000 bytes of pixel data for the 320×200 display, and the sprite shape RAM holds up to 256 shape definitions of 256 bytes each (64 KB total). Neither space can be addressed directly by the CPU; instead, the VGC provides two access mechanisms:

- **MemRead/MemWrite commands** — write a space selector and offset to the VGC command registers, then issue command $19 (MemRead) or $1A (MemWrite) at `$A010`.
- **DMA transfers** — the DMA controller can copy blocks to and from VGC memory using space identifiers 3 (graphics bitmap) and 4 (sprite shapes).

A.1 Graphics Bitmap

The graphics bitmap is a linear framebuffer of 320×200 pixels stored in row-major order. Each pixel occupies one byte, with the colour index in the low nibble (bits 3–0). The upper nibble is ignored on write and reads as zero.

Property	Value
Resolution	320×200 pixels
Bits per pixel	4 (stored in low nibble of each byte)
Total size	64,000 bytes
VGC MemRead/MemWrite space	$02 (`MemSpaceGfx`)
DMA space	3 (`DmaSpaceVgcGfx`)

A.1.1 Address Calculation

The offset of any pixel within the bitmap is:

$$offset = y \times 320 + x$$

where $x \in [0, 319]$ and $y \in [0, 199]$.

A.1.2 Row Start Offsets

The following table shows the starting offset (in decimal and hex) for every 10th row:

Row	Offset (dec)	Offset (hex)	Row	Offset (dec)	Offset (hex)
0	0	$0000	100	32000	$7D00
10	3200	$0C80	110	35200	$8980
20	6400	$1900	120	38400	$9600
30	9600	$2580	130	41600	$A280
40	12800	$3200	140	44800	$AF00
50	16000	$3E80	150	48000	$BB80
60	19200	$4B00	160	51200	$C800
70	22400	$5780	170	54400	$D480
80	25600	$6400	180	57600	$E100
90	28800	$7080	190	60800	$ED80

A.1.3 Drawing Commands vs. Direct Access

Most programs use the VGC's built-in drawing commands (PLOT, LINE, CIRCLE, RECT, FILL, PAINT) which operate on the bitmap internally. The GCLS command clears the entire bitmap to colour 0. Direct MemRead/MemWrite access is useful for reading individual pixel values or implementing custom rendering algorithms.

A.1.4 BASIC Example: Read/Write Pixels via MemRead/MemWrite

```
10 REM read pixel at (100, 50)
20 POKE $A011, 0  : POKE $A012, 2  : REM space=gfx
30 OFFSET = 50*320+100
40 POKE $A013, OFFSET AND 255
50 POKE $A014, INT(OFFSET/256)
60 POKE $A010, $19 : REM MemRead
70 PRINT "COLOR:"; PEEK($A015)
80 REM
90 REM write pixel at (100, 50) to color 1 (white)
100 POKE $A015, 1
110 POKE $A010, $1A : REM MemWrite
```

The VGC command registers at `$A011`–`$A014` retain their values after a command executes. When reading or writing consecutive pixels, you only need to update the offset registers between commands.

A.2 Sprite Shape RAM

Sprite shape RAM holds up to 256 shape definitions. Each shape is a 16×16 pixel image stored as 256 bytes (one byte per pixel, colour index in the low nibble). A pixel value of 0 is transparent by default.

Property	Value
Shape slots	256 (numbered 0–255)
Shape dimensions	16×16 pixels
Bytes per shape	256
Bits per pixel	4 (stored in low nibble of each byte)
Total size	65,536 bytes (64 KB)
VGC MemRead/MemWrite space	$03 (`MemSpaceSprite`)
DMA space	4 (`DmaSpaceVgcSprite`)

A.2.1 Address Calculation

The offset of any pixel within sprite shape RAM is:

$$offset = slot \times 256 + row \times 16 + column$$

where $slot \in [0, 255]$, $row \in [0, 15]$, and $column \in [0, 15]$.

A.2.2 Shape Slot Assignments

Shape slots 0–15 correspond to the default shapes for sprites 0–15 at power-on, but any sprite can reference any shape slot by writing to the `SprRegShape` register at offset +4 within the sprite's register block (base `$A040`). This means multiple sprites can share a single shape definition, and a sprite's appearance can be changed at any time by pointing it to a different slot.

A.2.3 BASIC Example: Define a Shape via MemWrite

```
10 REM define a simple arrow shape in slot 0
20 POKE $A011, 0 : POKE $A012, 3 : REM space=sprite
30 FOR ROW = 0 TO 15
40   FOR COL = 0 TO 15
50     OFFSET = 0*256 + ROW*16 + COL
60     POKE $A013, OFFSET AND 255
70     POKE $A014, INT(OFFSET/256)
80     C = 0 : REM transparent
90     IF COL = 7 AND ROW < 12 THEN C = 1
100    IF ROW = 0 AND COL >= 5 AND COL <= 9 THEN C = 1
110    POKE $A015, C
120    POKE $A010, $1A : REM MemWrite
130  NEXT COL
140 NEXT ROW
```

A.2.4 Using DMA for Bulk Shape Loading

For loading many shapes at once (e.g., from XRAM or CPU RAM), the DMA controller is far more efficient than individual MemWrite commands. Set the DMA source space to CPU RAM (space 0) or XRAM (space 5), the destination space to `DmaSpaceVgcSprite` (space 4), and issue a DMA copy command. See Chapter 10.1 for DMA register details.

```
10 REM DMA copy 2560 bytes (10 shapes) from CPU RAM
20 REM at $2000 into sprite shape slot 0
30 DMACOPY $2000, 0, 2560, 0, 4
```

Chapter B

Expansion RAM (XRAM)

"More memory is always the answer. The question is irrelevant."

— *Every programmer, eventually*

Introduction

The NovaVM provides 512 KB of expansion RAM (XRAM) — eight times the 6502's native 64 KB address space. XRAM is not directly addressable by the CPU; instead, it is accessed through the Expansion Memory Controller (XMC) registers at $BA00–$BA3F, four memory-mapped windows at $BC00–$BFFF, or via DMA transfers (space 5).

This appendix provides a consolidated reference for XRAM's organization, access methods, and named-block allocation system. For the XMC register-by-register documentation, see Chapter 10.

B.1 Bank Structure

XRAM uses 24-bit addressing: an 8-bit bank number (high byte) combined with a 16-bit offset within the bank.

Bank	Start Address	End Address	Size
0	$000000	$00FFFF	64 KB
1	$010000	$01FFFF	64 KB
2	$020000	$02FFFF	64 KB
3	$030000	$03FFFF	64 KB
4	$040000	$04FFFF	64 KB
5	$050000	$05FFFF	64 KB
6	$060000	$06FFFF	64 KB
7	$070000	$07FFFF	64 KB
	Total		**512 KB**

The active bank is selected by writing to `XmcBank` at `$BA0C`. The total number of available banks can be read from `XmcBanks` at `$BA0D`.

B.2 Access Methods

There are four ways to read and write XRAM, each suited to different use cases.

B.2.1 Byte Commands (GetByte / PutByte)

The simplest access method: set the bank, address, and data registers, then issue a GetByte or PutByte command. Good for reading or writing individual values, but too slow for large transfers.

```
10 REM write byte $42 to XRAM bank 0, offset $1000
20 XBANK 0
30 XPOKE $1000, $42
40 PRINT "READ BACK:"; XPEEK($1000)
```

B.2.2 Bulk Commands (Stash / Fetch)

Copy a block of bytes between CPU RAM and XRAM in a single operation. The XMC handles the transfer internally at full speed.

```
10 REM copy 256 bytes from CPU RAM at $2000
20 REM into XRAM bank 0 at offset $0000
30 STASH $2000, 256, 0, 0
40 REM
50 REM copy them back to CPU RAM at $3000
60 FETCH $3000, 256, 0, 0
```

B.2.3 XMC Windows ($BC00–$BFFF)

Four 256-byte windows are mapped directly into the 6502 address space. Each window can be independently pointed at any 256-byte page in XRAM using the XMAP command. Once mapped, ordinary PEEK/POKE operations read and write XRAM through the window.

Window	CPU Address Range	Size
0	$BC00–$BCFF	256 bytes
1	$BD00–$BDFF	256 bytes
2	$BE00–$BEFF	256 bytes
3	$BF00–$BFFF	256 bytes

```
10 REM map window 0 to XRAM bank 0, page 16 ($1000)
20 XMAP 0, 0, 16
30 REM now PEEK/POKE at $BC00-$BCFF accesses XRAM
40 POKE $BC00, $42
50 PRINT PEEK($BC00)
60 XUNMAP 0 : REM release window
```

XMC windows are ideal for random access to small data structures stored in XRAM, such as lookup tables or tile maps. For sequential bulk transfers, Stash/Fetch or DMA are faster.

B.2.4 DMA Transfers (Space 5)

The DMA controller treats XRAM as space 5. It can copy data between XRAM and any other DMA space (CPU RAM, VGC character/color/graphics/sprite memory, or even another region of XRAM). This is the fastest method for large transfers.

```
10 REM DMA copy 1000 bytes from XRAM offset $0000
20 REM to CPU RAM at $2000
30 DMACOPY 0, $2000, 1000, 5, 0
```

B.3 Named Block Allocation

The XMC provides a simple named-block filesystem within XRAM. Each block has a name (up to 27 characters) and can be stored and retrieved without needing to track offsets manually.

Command	Description
NStash	Store a block of CPU RAM into XRAM under a given name.
NFetch	Retrieve a named block from XRAM into CPU RAM.
NDelete	Delete a named block and free its XRAM pages.
NDirOpen / NDirRead	Enumerate all named blocks.

```
10 REM store level data under the name "LEVEL1"
20 XMEM "LEVEL1", $2000, 4096, 1
30 REM
40 REM retrieve it later
50 XMEM "LEVEL1", $2000, 0, 0
60 REM
70 REM list all named blocks
80 XDIR
90 REM
100 REM delete a named block
110 XDEL "LEVEL1"
```

Named blocks are allocated from the XRAM page pool. The NStash command automatically allocates the required number of 256-byte pages. Use XFREE to check how many pages remain available.

B.4 Page Usage Tracking

XRAM is divided into 256-byte pages. With 512 KB of XRAM, there are 2,048 pages in total. The XMC tracks which pages are in use and which are free.

Register	Address	Description
XmcPagesUsed	$BA0E (16-bit)	Number of pages currently allocated
XmcPagesFree	$BA10 (16-bit)	Number of pages available

The XALLOC command reserves a contiguous block of pages and returns the starting offset. XRESET releases all allocations and clears XRAM.

```
10 PRINT "PAGES FREE:"; XFREE
20 XALLOC 16 : REM allocate 16 pages (4KB)
30 PRINT "PAGES FREE:"; XFREE
40 XRESET : REM free everything
50 PRINT "PAGES FREE:"; XFREE
```

Warning

XRESET clears all XRAM contents and allocations, including named blocks. There is no undo. Use it only when you want to start fresh.

Chapter C

WTS Sample Memory

"A good sample library is worth a thousand oscillators."

— *Studio wisdom*

Introduction

The Wavetable Synthesizer's sample data lives entirely in host memory, outside both the 6502's 64 KB address space and the 512 KB XRAM. Samples are loaded from industry-standard SF2 (SoundFont 2) files on the host filesystem and decoded into PCM audio buffers that the WTS voices reference during playback.
This appendix describes how sample memory is organized, loaded, and used.

C.1 Loading Soundfonts

Soundfont loading is initiated through the FileIO controller using command $15 (`SfLoad`). The process is:

1. Write the soundfont filename (without path) into the FioName buffer at `$B9B0`.
2. Write command $15 to the FileIO command register at `$B9A0`.
3. Poll `WtsSoundfontStatus` at `$A183` until loading completes.

The file must be located in the `~/e6502-programs` directory (the same directory used for BASIC program files).

```
10 REM load a General MIDI soundfont
20 F$ = "GeneralUser_GS.sf2"
30 FOR I = 1 TO LEN(F$)
40   POKE $B9B0+I-1, ASC(MID$(F$,I,1))
50 NEXT I
60 POKE $B9B0+LEN(F$), 0 : REM null terminator
70 POKE $B9A0, $15 : REM SfLoad command
80 REM wait for load to complete
90 IF PEEK($A183) <> 0 THEN 90
```

The MML parser can also trigger automatic soundfont loading when a `@W<n>` voice command is encountered and no soundfont is currently loaded.

C.2 SF2 Format Overview

SF2 (SoundFont 2) is an industry-standard format originally developed by Creative Labs. Each file contains:

- **PCM audio samples** — raw waveform data, typically 16-bit mono or stereo at 44 100 Hz.
- **Instrument definitions (presets)** — each preset maps key ranges and velocity layers to one or more sample zones.
- **Articulation data** — volume envelopes, filter settings, loop points, and tuning parameters for each zone.

The NovaVM's WTS engine reads and interprets all of these at load time, building internal data structures for real-time playback.

C.3 Instrument Mapping

Each preset in the loaded soundfont becomes a WTS instrument, numbered sequentially starting from 0. For General MIDI–compatible soundfonts, the instrument numbers follow the GM program mapping:

Index	GM Instrument	Index	GM Instrument
0	Acoustic Grand Piano	24	Acoustic Guitar (nylon)
4	Electric Piano 1	32	Acoustic Bass
8	Celesta	40	Violin
11	Vibraphone	48	String Ensemble 1
16	Drawbar Organ	56	Trumpet
19	Church Organ	73	Flute

The total number of available instruments can be queried through the WTS instrument enumeration interface at `$A1A0`.
WTS voices select an instrument by writing the instrument index to the voice's instrument register (offset +2 from the voice base address).

C.4 Memory Usage

Sample data is decoded from the SF2 file at load time and stored as uncompressed PCM buffers in host memory. Typical memory usage:

Soundfont Type	Approximate Host RAM
Small (single instrument)	1–5 MB
Medium (partial GM set)	10–30 MB
Full GM soundfont	30–100 MB

Warning

Sample memory resides entirely in the host process and does not affect the 6502's 64 KB address space or the 512 KB XRAM. However, very large soundfonts may consume significant host RAM.

C.5 Limitations

- Only one soundfont can be loaded at a time. Loading a new soundfont replaces the previous one entirely, including all instrument definitions and sample data.
- Soundfont files must be in SF2 format; compressed variants (SF3/SF4) are not supported.
- Sample data is read-only from the 6502's perspective — there is no mechanism to upload custom PCM data from CPU RAM or XRAM.
- The maximum number of simultaneous WTS voices is 8, regardless of how many instruments are defined in the soundfont.

Chapter D

Color Table

"Sixteen colours ought to be enough for anybody."

— *Every 8-bit computer designer, 1977–1985*

Introduction

The NovaVM uses a fixed 16-colour palette inspired by the Commodore 64. Every colour index in the system — text foreground and background, graphics pixels, and sprite pixels — references this palette via a 4-bit value (0–15).

D.1 Palette

Reference

Index	Swatch	Name	R	G	B	Hex
0		Black	0	0	0	`#000000`
1		White	255	255	255	`#FFFFFF`
2		Red	136	0	0	`#880000`
3		Cyan	170	255	238	`#AAFFEE`
4		Purple	204	68	204	`#CC44CC`
5		Green	0	204	85	`#00CC55`
6		Blue	0	0	170	`#0000AA`
7		Yellow	238	238	119	`#EEEE77`
8		Orange	221	136	85	`#DD8855`
9		Brown	102	68	0	`#664400`
10		Light Red	255	119	119	`#FF7777`
11		Dark Grey	51	51	51	`#333333`
12		Medium Grey	119	119	119	`#777777`
13		Light Green	170	255	102	`#AAFF66`
14		Light Blue	0	136	255	`#0088FF`
15		Light Grey	187	187	187	`#BBBBBB`

D.2 Usage Notes

- **Transparency:** Colour index 0 (black) serves as the transparent colour for sprites. Sprite pixels with value 0 allow the background to show through.
- **Default colours:** At power-on, the text foreground is 1 (white) and the background is 0 (black).
- **Boot screen:** The EhBASIC cold-start routine sets the background to 6 (blue) and the foreground to 1 (white), mimicking the classic Commodore 64 appearance.
- **Colour RAM:** In text mode, each character cell has an associated byte in colour RAM. The low nibble sets the foreground colour and the high nibble sets the background colour for that cell.
- **Graphics bitmap:** Each pixel byte uses only the low nibble (bits 3–0) for the colour index. The upper nibble is ignored.

D.3 BASIC Example

```
10 REM cycle through all 16 colors
20 FOR C=0 TO 15
30   COLOR 1, C : CLS
40   PRINT "COLOR"; C
50   FOR D=1 TO 2000 : NEXT D
60 NEXT C
70 COLOR 1, 6 : CLS
```

Chapter E

ASCII & Screen Code Table

"In the beginning there was ASCII, and it was 7 bits, and it was good enough."

— *The Book of Standards, 1:1*

Introduction

The NovaVM uses standard ASCII character encoding (0–127), unlike the Commodore 64 which uses its own PETSCII encoding. This means ordinary ASCII text files and string constants work as expected, with no translation required.
Characters 128–255 map to the active font's extended character set. Font 0 (the default) includes box-drawing characters and additional symbols in this range.

E.1 ASCII Table (0–127)

The table below shows all 128 standard ASCII characters in a compact four-column layout. Control characters (0–31 and 127) are shown with their standard abbreviations.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	32	20	␣	64	40	@	96	60	‘
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	’	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

E.2 Control Characters Used by the Screen Editor

The NovaVM's screen editor recognises the following control characters:

Dec	Hex	Name	Action
8	`$08`	BS (Backspace)	Move cursor left one position and erase the character
10	`$0A`	LF (Line Feed)	Move cursor down one row
13	`$0D`	CR (Carriage Return)	Move cursor to the start of the next line
27	`$1B`	ESC (Escape)	Escape (context-dependent)
127	`$7F`	DEL (Delete)	Delete the character at the cursor position

E.3 Extended Characters (128–255)

Character codes 128–255 map to the active font's extended character set. The default font (font 0) includes box-drawing characters, commonly used for creating borders and UI elements in text mode:

Range	Contents (Font 0)	Notes
128–159	Box-drawing corners and junctions	Single and double lines
160–191	Horizontal and vertical lines	Single and double weight
192–223	Block elements and shading	Quarter, half, full blocks
224–255	Miscellaneous symbols	Arrows, bullets, etc.

The extended character set is font-dependent. If a different font is loaded, characters 128–255 may display differently. Characters 0–127 always follow standard ASCII regardless of the active font.

E.4 BASIC String Functions

Two BASIC functions convert between characters and their ASCII codes:

- `ASC(`*`string$`*`)` — returns the ASCII code of the first character in the string. For example, `ASC("A")` returns 65.

- `CHR$(code)` — returns a one-character string with the given ASCII code. For example, `CHR$(65)` returns `"A"`.

```
10 REM print the full printable ASCII set
20 FOR I = 32 TO 126
30   PRINT CHR$(I);
40 NEXT I
50 PRINT
60 REM
70 REM show the code for a character
80 PRINT "CODE FOR 'A':"; ASC("A")
```

Address Index

Symbols

$0000 — LAB_WARM 1
$0001 — Wrmjpl 1
$0002 — Wrmjph 1
$000A — Usrjmp 2
$000B — Usrjpl 2
$000C — Usrjph 2
$000D — Nullct 2
$000E — TPos 2
$000F — TWidth 3
$0010 — Iclim 3
$0011 — Itempl 3
$0012 — Itemph 3
$005B — Srchc / Temp3 4
$005C — Scnquo / Asrch 4
$005D — Ibptr / Dimcnt / Tindx 4
$005E — Defdim 4
$005F — Dtypef 4
$0060 — Oquote / Gclctd 4
$0061 — Sufnxf 4
$0062 — Imode 4
$0063 — Cflag 5
$0064 — TabSiz 5
$0065 — next_s 5
$0066 — last_sl 5
$0067 — last_sh 5
$0068-$0070 — des_sk (Descriptor Stack) 5
$0071 — ut1_pl 5
$0072 — ut1_ph 5
$0073 — ut2_pl 6
$0074 — ut2_ph 6
$0075 — FACt_1 6
$0076 — FACt_2 / dims_l 6
$0077 — FACt_3 / dims_h 6
$0078 — TempB 6
$0079 — Smeml 6
$007A — Smemh 6
$007B — Svarl 6
$007C — Svarh 7
$007D — Sarryl 7
$007E — Sarryh 7
$007F — Earryl 7
$0080 — Earryh 7
$0081 — Sstorl 7
$0082 — Sstorh 7
$0083 — Sutill 7
$0084 — Sutilh 7
$0085 — Ememl 7
$0086 — Ememh 7
$0087 — Clinel 8
$0088 — Clineh 8
$0089 — Blinel 8
$008A — Blineh 8
$008B — Cpntrl 8
$008C — Cpntrh 8
$008D — Dlinel 8
$008E — Dlineh 9
$008F — Dptrl 9
$0090 — Dptrh 9
$0091 — Rdptrl 9
$0092 — Rdptrh 9
$0093 — Varnm1 9
$0094 — Varnm2 9
$0095 — Cvaral 9
$0096 — Cvarah 9
$0097 — Frnxtl 9
$0098 — Frnxth 9
$0099 — prstk 9
$009B — comp_f 10
$009C — func_l / garb_l 10
$009D — func_h / garb_h 10
$009E — des_2l 10
$009F — des_2h 10
$00A0 — g_step 10
$00A1 — Fnxjmp 10
$00A2 — Fnxjpl / g_indx 10
$00A3 — FAC2_r 10
$00A4 — Adatal / Nbendl 10
$00A5 — Adatah / Nbendh 11
$00A6 — Obendl 11
$00A7 — Obendh 11

$00A8 — numexp / numbit 11
$00A9 — expcnt 11
$00AA — numdpf / Astrtl 11
$00AB — expneg / Astrth 11
$00AC — FAC1_e 11
$00AD — FAC1_1 11
$00AE — FAC1_2 11
$00AF — FAC1_3 11
$00B0 — FAC1_s 12
$00B1 — negnum / numcon 12
$00B2 — FAC1_o 12
$00B3 — FAC2_e 12
$00B4 — FAC2_1 12
$00B5 — FAC2_2 12
$00B6 — FAC2_3 12
$00B7 — FAC2_s 12
$00B8 — FAC_sc 12
$00B9 — FAC1_r 12
$00BA — csidx / Asptl 13
$00BB — Aspth 13
$00BC-$00C1 — LAB_IGBY (Increment and Get BYte) 13
$00C2-$00C4 — LAB_GBYT (Get BYTe) 13
$00C3 — Bpntrl 13
$00C4 — Bpntrh 13
$00D8 — Rbyte4 13
$00D9 — Rbyte1 13
$00DA — Rbyte2 14
$00DB — Rbyte3 14
$00DC — NmiBase 14
$00DD — NMI Address Low 14
$00DE — NMI Address High 14
$00DF — IrqBase 14
$00E0 — IRQ Address Low 14
$00E1 — IRQ Address High 14
$00E2 — help_len 15
$00EF-$00FF — Decss (Decimal String Buffer) 15
$0100-$01FF — Hardware Stack 17
$0200 — VgcBase (low) 21
$0200-$027F — System Vector Table . 21
$0201 — VgcBase (high) 21
$0202 — RegCmdAddr (low) 22
$0203 — RegCmdAddr (high) 22
$0204 — CharRamBase (low) 22
$0205 — CharRamBase (high) 22
$0206 — ColorRamBase (low) 22
$0207 — ColorRamBase (high) 22
$0208 — SidBase (low) 22
$0209 — SidBase (high) 22
$020A — FileIoBase (low) 22
$020B — FileIoBase (high) 22
$020C — XmcBase (low) 22
$020D — XmcBase (high) 22
$020E — TimerBase (low) 23
$020F — TimerBase (high) 23
$0210 — Sid2Base (low) 23
$0211 — Sid2Base (high) 23
$0212 — NicBase (low) 23
$0213 — NicBase (high) 23
$0214 — DmaBase (low) 23
$0215 — DmaBase (high) 23
$0216 — BlitterBase (low) 23
$0217 — BlitterBase (high) 23
$0280-$9FFF — BASIC Program Space . 25
$A000 — RegMode 36
$A001 — RegBgCol 36
$A002 — RegFgCol 36
$A003 — RegCursorX 36
$A004 — RegCursorY 37
$A005 — RegScrollX 37
$A006 — RegScrollY 37
$A007 — RegFont 37
$A008 — RegStatus 37
$A009 — RegSpriteCount 37
$A00A — RegCursorEnable 38
$A00B — RegColSt 38
$A00C — RegColBg 38
$A00D — RegBorder 38
$A00E — RegCharOut 38
$A00F — RegCharIn 38
$A010 — RegCmd 39
$A011-$A01E — RegP0–RegP13 (Parameter Registers) 39
$A01F — RegIrqCtrl 48
$A020 — RegHelp 48
$A021-$A030 — HelpSearchBuffer 49
$A031 — CmpCmd 49
$A032 — CmpStatus 49
$A033 — CmpErrCount 49
$A034 — CmpWarnCount 49
$A035-$A036 — CmpErrLine 49
$A037-$A038 — CmpCodeSize 50
$A039 — CmpErrMsg 50
$A03A — CmpErrIdx 50
$A03B-$A03D — CmpSrcAddr 50
$A03F — RegRomSwap 50
$A100 — NicCmd 53
$A101 — NicStatus 54
$A102 — NicSlot 54

$A103 — NicIrqCtrl 55
$A104 — NicIrqStatus 55
$A108-$A109 — NicRemotePort 55
$A10A-$A10B — NicLocalPort 55
$A110-$A111 — NicDmaAddr 55
$A112 — NicDmaLen 55
$A113 — NicMsgLen 56
$A118-$A11B — NicSlotStatus0–3 56
$A120-$A13F — NicNameBuf 56
$A140 — WtsVoiceNote 59
$A141 — WtsVoiceVelocity 60
$A142 — WtsVoiceInstrument 60
$A143 — WtsVoiceVolume 60
$A144 — WtsVoicePanning 60
$A145 — WtsVoicePitchBendLo 60
$A146 — WtsVoicePitchBendHi 60
$A147 — WtsVoiceStatus 60
$A180 — WtsReverbLevel 61
$A181 — WtsChorusLevel 61
$A182 — WtsMasterVolume 61
$A183 — WtsSoundfontStatus 61
$A184 — WtsInstrumentCount 61
$A185 — WtsCommand 62
$A186 — WtsCommandArg 62
$A187 — WtsActiveVoices 62
$A1A0 — WtsEnumIndex 62
$A1A1 — WtsEnumBank 62
$A1A2 — WtsEnumProgram 62
$A1A3-$A1DF — WtsEnumName 62
$AA00-$B1CF — Character RAM 65
$B1D0-$B99F — Color RAM 66
$B9A0 — FioCmd 70
$B9A1 — FioStatus 70
$B9A2 — FioErrCode 70
$B9A3 — FioNameLen 70
$B9A4 — FioSrcL 70
$B9A5 — FioSrcH 70
$B9A6 — FioEndL 70
$B9A7 — FioEndH 70
$B9A8 — FioSizeL 71
$B9A9 — FioSizeH 71
$B9AA — FioGSpace 71
$B9AB — FioGAddrL 71
$B9AC — FioGAddrH 71
$B9AD — FioGLenL 71
$B9AE — FioGLenH 71
$B9AF — FioDirType 71
$B9B0-$B9EF — FioName 71
$BA00 — XmcCmd 82
$BA01 — XmcStatus 82
$BA02 — XmcErrCode 82
$BA03 — XmcCfg 83
$BA04 — XmcAddrL 83
$BA05 — XmcAddrM 83
$BA06 — XmcAddrH 83
$BA07 — XmcRamL 83
$BA08 — XmcRamH 83
$BA09 — XmcLenL 83
$BA0A — XmcLenH 83
$BA0B — XmcData 83
$BA0C — XmcBank 83
$BA0D — XmcBanks 84
$BA0E — XmcPagesUsedL 84
$BA0F — XmcPagesUsedH 84
$BA10 — XmcPagesFreeL 84
$BA11 — XmcPagesFreeH 84
$BA12 — XmcNameLen 84
$BA13 — XmcHandle 84
$BA14 — XmcDirCountL 84
$BA15 — XmcDirCountH 84
$BA16 — XmcWinCtl 84
$BA18-$BA1A — XmcWin0A 85
$BA1B-$BA1D — XmcWin1A 85
$BA1E-$BA20 — XmcWin2A 85
$BA21-$BA23 — XmcWin3A 85
$BA24-$BA3F — XmcName 85
$BA40 — TimerCtrl 89
$BA41 — TimerStatus 89
$BA42 — TimerDivL 89
$BA43 — TimerDivH 89
$BA50 — MusicStatus 90
$BA51-$BA5E — MusicNote1–14 90
$BA5F-$BA60 — MusicElapsed 91
$BA61-$BA62 — MusicTotal 91
$BA63 — DmaCmd 92
$BA64 — DmaStatus 92
$BA65 — DmaErrCode 92
$BA66 — DmaSrcSpace 92
$BA67 — DmaDstSpace 92
$BA68 — DmaSrcL 92
$BA69 — DmaSrcM 93
$BA6A — DmaSrcH 93
$BA6B — DmaDstL 93
$BA6C — DmaDstM 93
$BA6D — DmaDstH 93
$BA6E — DmaLenL 93
$BA6F — DmaLenM 93
$BA70 — DmaLenH 93
$BA71 — DmaMode 93
$BA72 — DmaFillValue 93
$BA73 — DmaCountL 93
$BA74 — DmaCountM 93

$BA75 — DmaCountH 94
$BA83 — BltCmd 95
$BA84 — BltStatus 95
$BA85 — BltErrCode 95
$BA86 — BltSrcSpace 96
$BA87 — BltDstSpace 96
$BA88 — BltSrcL 96
$BA89 — BltSrcM 96
$BA8A — BltSrcH 96
$BA8B — BltDstL 96
$BA8C — BltDstM 96
$BA8D — BltDstH 96
$BA8E — BltWidthL 96
$BA8F — BltWidthH 96
$BA90 — BltHeightL 96
$BA91 — BltHeightH 96
$BA92 — BltSrcStrideL 96
$BA93 — BltSrcStrideH 96
$BA94 — BltDstStrideL 96
$BA95 — BltDstStrideH 97
$BA96 — BltMode 97
$BA97 — BltFillValue 97
$BA98 — BltColorKey 97
$BA99 — BltCountL 97
$BA9A — BltCountM 97
$BA9B — BltCountH 97
$BAB0-$BB1F — File Metadata Buffer 77
$BC00-$BFFF — XMC Window 98
$C000-$FFF9 — EhBASIC ROM 101
$C000 — LAB_COLD 102
$C003 — LAB_WARM 102
$D400 — FreqLo — Frequency Low . 106
$D401 — FreqHi — Frequency High . 106
$D402 — PwLo — Pulse Width Low 106
$D403 — PwHi — Pulse Width High 106
$D404 — WfCtrl — Waveform Control .. 106
$D405 — AD — Attack / Decay 107
$D406 — SR — Sustain / Release ... 107
$D415 — FiltLoFreq — Filter Cutoff Low 108
$D416 — FiltHiFreq — Filter Cutoff High 108
$D417 — FiltCtrl — Filter Resonance & Voice Select 108
$D418 — VolMode — Filter Mode & Master Volume 108
$D41D — Voice 1 Volume 109
$D41E — Voice 2 Volume 109
$D41F — Voice 3 Volume 109
$D420-$D43F — SID Chip 2 109
$FF80-$FFF9 — Monitor Code 102
$FFFA-$FFFF — Hardware Vectors .. 103